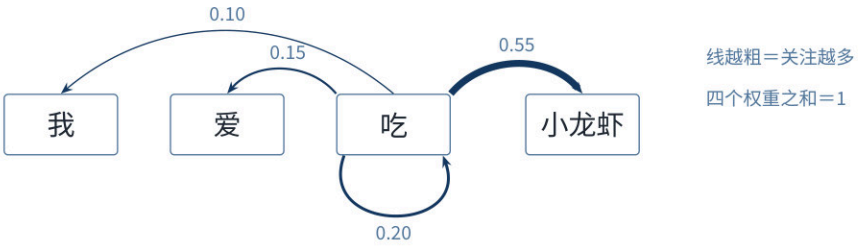
语言大模型

从 RNN 到 Transformer，再到 GPT

一句话读懂本专题：**让每个词去“看”句子里的其他词，再把看到的信息加权混合**——这就是注意力；把它一层层叠起来，就是今天所有语言大模型的骨架。



$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right)V$$

打分 → 缩放 → 归一化 → 加权提取信息



讲义内容　知识讲解与图解 · 公式推导与证明 · 生活应用 · 学科小史 · 38 道分层练习与逐选项图解

排版方式　LaTeX + XeLaTeX，全部图形为 TikZ / PGFPlots 原生矢量图

阅读说明

这份讲义把一次课的课件、课堂笔记和一篇经典论文重新组织成一条可以自学的路线。课件负责“讲了什么”，论文负责“原文怎么说”，笔记里留下的十几个问号负责“哪里还没懂”——讲义的任务是把这三者接起来，并把课件为了节省篇幅而省掉的定义、推导、例子补全。

颜色与标签的固定含义

蓝色框：定义 / 核心公式 / 结论　　米色框：例子 / 生活应用 / 历史　　红色：易错 / 原图纠错　　绿色：小结 / 自检

原 · 课件 p.27 表示这段内容能在原始资料的对应位置找到；教学补全 表示原始资料没有写出、由讲义按论文原文或标准教材表述补上的内容；类比 表示只用于帮助直观理解的比喻，不能当作严格定义。三者在全文中始终分开标记。

建议的读法

1. **第一遍**只读正文和图，跳过所有“证明”与“教学补全”，目标是能把图 34 的完整架构从左到右讲一遍。
2. **第二遍**补上证明与推导，重点是第 6 章的注意力公式：每个符号是什么、为什么除以 $\sqrt{d_k}$、为什么最后乘 V。
3. **第三遍**合上讲义做第 11 章的 38 道题，做完再翻答案。答案集中放在第 12 章，题目区的图不含任何提示。
4. 课后作业是通读论文原文。第 10.5 节给了一张“论文各节 $\leftrightarrow$ 讲义各章”的对照表，可以对着读。

需要的前置知识

向量的点积、矩阵乘法、指数与对数、概率分布的基本概念，以及“神经网络是带参数的函数、靠梯度下降调整参数”这一点认识。凡是用到而容易遗忘的地方，正文都会就地复习一句。

目　录

阅读说明 1

1　导读：这一章要解决什么问题 4
1.1　本专题学什么 . . . 4
1.2　它在课程体系中的位置 . . . 4
1.3　前置知识地图：四件工具，够用就行 . . . 4
1.4　原始资料清单与重新编码 . . . 5
1.5　原始资料纠错 . . . 6

2　学科小史与现实应用 7
2.1　学科小史：这个知识是怎样出现的? . . . 7
2.2　从课本到现实：这个知识到底有什么用? . . . 8

3　大模型：为什么叫“打地基” 8
3.1　它解决什么问题 . . . 8
3.2　三个比喻与五个关键特征 . . . 9
3.3　与传统小模型的对比 . . . 10

4　语言建模：标签从哪里来，分数怎样变成概率 11
4.1　语言建模：预测下一个词 . . . 11
4.2　Softmax：把分数变成比例 . . . 12

4.3 交叉熵损失：猜得好不好，用一个数来衡量 . . . 14

5 循环神经网络与 LSTM：带着记忆逐词读 16
5.1 为什么需要一种新结构 . . . 16
5.2 RNN 的定义：循环在哪里 . . . 16
5.3 RNN 怎样训练 . . . 17
5.4 RNN 的两个根本缺点 . . . 18
5.5 LSTM：给记忆修一条直通车道 . . . 20
5.6 LSTM 仍然留下的问题 . . . 21

6 注意力机制：让每个词直接看见所有词 22
6.1 换一个思路：为什么非要排队? . . . 22
6.2 注意力的直觉：词的意思由关系决定 . . . 23
6.3 文字怎样变成神经网络能计算的数字 . . . 23
6.4 点积：衡量两个向量有多“对得上” . . . 25
6.5 缩放点积注意力：公式的五步 . . . 26
6.6 为什么要除以 $\sqrt{d_k}$ . . . 27
6.7 为什么最后要乘 V . . . 29
6.8 自注意力：一句话里的每个词都重新认识自己 . . . 30
6.9 多头注意力：多组不同的观察角度 . . . 31
6.10 位置编码：告诉模型“谁在前、谁在后” . . . 32

7 Transformer：把零件装配成完整架构 35
7.1 Transformer 一开始要解决什么问题 . . . 35
7.2 全貌：一张图看完整架构 . . . 35
7.3 Add & Norm：残差连接与层归一化 . . . 36
7.4 前馈网络：每个词回到座位上独立加工 . . . 38
7.5 编码器的一层：把输入读成上下文向量 . . . 39
7.6 解码器：一边理解，一边生成 . . . 40
7.7 交叉注意力：生成时回头看原文 . . . 42
7.8 输出层：从向量回到词 . . . 42
7.9 为什么 Transformer 能堆很深 . . . 43
7.10 论文中的关键数字 . . . 43

8 BERT 与 GPT：把 Transformer 拆开来用 44
8.1 从 Transformer 到三种用法 . . . 44
8.2 BERT：只用编码器，做“完形填空” . . . 44
8.3 GPT：只用解码器，专门做生成 . . . 46
8.4 GPT 如何训练：不断预测下一个 token . . . 46
8.5 GPT 生成文本的过程 . . . 47
8.6 Temperature：控制采样的随机程度 . . . 49
8.7 相近概念辨析：BERT 与 GPT . . . 51

9 从“会续写”到“会聊天”：微调、对齐与蒸馏 51
9.1 三个阶段：会说话、会听话、更像助手 . . . 51
9.2 今天的大模型：不只是聊天 . . . 53
9.3 蒸馏：让一个模型向另一个模型学习 . . . 53
9.4 下节课预告与课后作业 . . . 56

10 总结：知识地图、易错总表与论文阅读路线 56
10.1 全章知识地图 . . . 57
10.2 相近概念总对比 . . . 57
10.3 高频易错点总表 . . . 58
10.4 基础解题步骤总结 . . . 58
10.5 后续课程连接与论文阅读路线 . . . 58
10.6 学完全章的总自检 . . . 59

11 分层练习：38 道 **59**
11.1 A 基础巩固训练（25 道） 59
11.2 B 中等偏下提升训练（10 道） 64
11.3 C 跨学科迁移训练（3 道） 66

12 答案速查与详细解析 **68**
12.1 全部答案速查 68
12.2 基础单选题解析（第 1–10 题） 68
12.3 基础多选题解析（第 11–15 题） 71
12.4 基础填空题解析（第 16–20 题） 72
12.5 基础解答题解析（第 21–25 题） 74
12.6 提升单选题解析（第 26–28 题） 75
12.7 提升多选题解析（第 29–30 题） 76
12.8 提升填空题解析（第 31–33 题） 77
12.9 提升综合大题解析（第 34–35 题） 78
12.10 跨学科迁移题解析（第 36–38 题） 80

参考资料 **81**

第一章　导读：这一章要解决什么问题

1.1　本专题学什么

一句话：**计算机怎样学会“接着往下写”，以及为什么“接着往下写”这一件事足以支撑起今天的聊天助手**。

沿着课件的顺序，这条线分成六段，每一段都由上一段留下的问题引出：

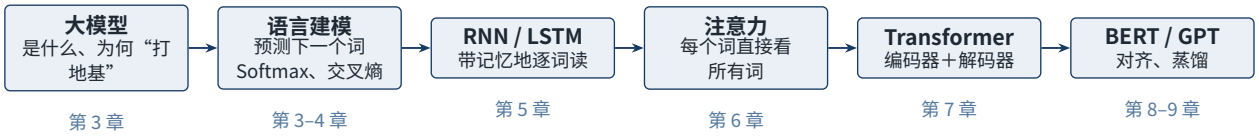


图 1　本专题的六段路线。箭头表示“前一段留下的问题由后一段解决”

学完之后应当能回答：大模型为什么叫 Foundation Model；语言模型的标签从哪里来；RNN 慢在哪里、忘在哪里；注意力公式里每个字母是什么、为什么这样算；Transformer 的每个零件各管什么；BERT 与 GPT 各取了 Transformer 的哪一半；同一个问题为什么每次回答不同；蒸馏到底抄的是什么。

1.2　它在课程体系中的位置

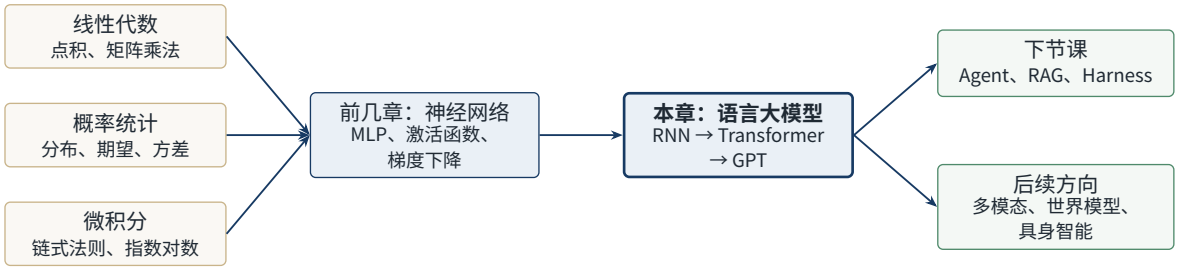
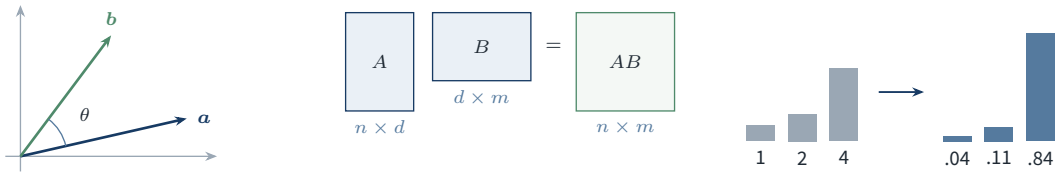


图 2　本章的前置与后续。左列是数学工具，中间是本课程前几章，右侧是本章之后的去向

1.3　前置知识地图：四件工具，够用就行

下面四件事在后文反复出现。这里先各用一句话和一幅小图唤醒记忆，细节在用到的地方再展开。



点积 $\boldsymbol{a}\cdot\boldsymbol{b}=\sum_i a_i b_i$
$=|\boldsymbol{a}||\boldsymbol{b}|\cos\theta$
方向越一致，值越大

矩阵乘法 $(AB)_{ij}$
$=A$ 的第 i 行与 B 的第 j 列的点积
内侧维度必须相等

Softmax
把任意一组实数变成
非负且和为 1 的比例

图 3　三件数学工具：点积量相似、矩阵乘法批量算点积、Softmax 把分数变比例

第四件工具是**“神经网络＝带参数的函数”**：输入一个向量，经过若干次“乘权重矩阵、加偏置、过激活函数”，输出一个向量；权重一开始是随机的，训练时用**梯度下降**一点点调整，使**损失函数**变小。课件第 9 页的多层感知机（MLP）就是这样的函数；本章所有模型——RNN、LSTM、Transformer——都只是在“怎样把这些运算搭起来”上各有不同。

1.4 原始资料清单与重新编码

三份资料的分工如下。讲义的章节顺序基本沿用课件，只把“Softmax 与交叉熵”提前单列一章，因为它在 RNN、注意力、GPT 三处都要用。

资料	可靠识别出的内容	在讲义中的位置
课件 p.3–7	大模型定义、三个比喻、关键特征、发展节点、与小模型对比	第 3 章
课件 p.9–15	MLP、RNN 定义与展开图、语言建模的标签、Softmax 与交叉熵、优缺点	第 4、5 章
课件 p.17–19	LSTM 的记忆单元与三个门、RNN/LSTM 的三个问题	第 5 章
课件 p.21–37	注意力直觉、关系表、Token/Embedding、公式五步、$\sqrt{d_k}$、乘 V、自注意力、多头、位置编码	第 6 章
课件 p.38–45	完整架构、Encoder、FFN、Decoder、掩码、交叉注意力、为什么能堆深	第 7 章
课件 p.47–55	机器翻译、三种用法、GPT 的含义与训练、生成过程、Temperature	第 8 章
课件 p.57–62	预训练—微调—对齐、模型家族、蒸馏	第 9 章
手写笔记 4 页	课件要点的提纲，外加十余个课堂提问（见表 1）	各章就地回答
论文原文	公式 (1)(2)(3)、图 1–2、表 1–3、超参数、实验结果	第 6、7 章逐处引用

课件第 14、18 页是视频截图，没有教学文字，讲义不作转写。

笔记里留下的问题。　手写笔记最有价值的部分是那些只写了问题、没写答案的行。它们恰好是课堂上口头讲过而课件没有写全的地方，讲义逐一作答：

序号	笔记中的问题（按出现顺序）	回答位置
1	RNN 的“循环在哪”？缺点“量大会忘”是什么意思？	5.2、5.4 节
2	向量维数如何确定？编号怎么变成向量？	6.3 节
3	Q、K 是什么？QK^{T} 在算什么？	6.4、6.5 节
4	为什么除以 $\sqrt{d_k}$？	6.6 节
5	V 是什么？为什么乘 V？	6.7 节
6	prompt 有什么作用？	6.8 节
7	为什么 Transformer 能堆很深？	7.9 节
8	Transformer 一开始解决什么问题？	7.1 节
9	Encoder-only 的目标函数是什么？“最终向量状态”怎么确定？	8.2 节
10	为什么 GPT 只用 Decoder？	8.3 节
11	GPT 如何训练？只是预测下一个 token，为什么学到这么多？	8.4 节
12	GPT 生成文本的过程？为什么每次回答可能不一样？	8.5、8.6 节
13	蒸馏是不是复制老师模型？“要真抄作业该抄什么”？开源的是什么？	9.3 节

无法可靠辨认或无法核验的内容。　以下几处不作猜测，如实列出：

- 笔记第 3 页“7. Decoder”下第 ① 行，字迹只能辨认出“……今天 → Decoder 就可多了”，前半句无法可靠辨认，讲义不采用。
- 笔记第 4 页“一些网页版模型 temperature＝0.2”。各家产品的默认采样参数并不统一，也大多不公开，这个数值无法核验，讲义只把它当作“网页版产品通常使用偏低温度”的课堂口述例子。

- 笔记第 4 页“例：deepseek V4.1 pro 开源的是什么？”——型号名称无法核验，讲义在 9.3 节只回答其中不依赖具体型号的部分：开放权重的模型，公开的究竟是什么。
- 课件 p.6 时间线最后一行（2024–2026）列出的具体型号，讲义未作核验，不在正文中引用；模型迭代很快，请以各厂商的官方发布为准。

1.5 原始资料纠错

对照论文原文并重新计算后，发现下列几处笔误或不严谨的表述。它们都不影响课堂主线，但其中两处是数值错误，会影响做题，所以集中列出；正文用到时一律采用修正后的写法。

原图纠错 · 课件 p.25：注意力关系表“吃”这一行之和不等于 1

原始写法　“吃”一行为 $0.1,\ 0.1,\ 0.1,\ 0.8$。
存在的问题　$0.1+0.1+0.1+0.8=1.1$。注意力权重由 Softmax 得到，每一行之和必须恰好等于 1（证明见 4.2 节），其余三行都满足，只有这一行不满足。
规范写法　讲义改为 $0.1,\ 0.1,\ 0.1,\ 0.7$。
是否影响后续　不影响结论（“吃”最关注“小龙虾”），但“每行和为 1”是基础题的常考点。

原图纠错 · 课件 p.30：Softmax 算例的两个数值

原始写法　分数 $(1.0,\ 1.5,\ 0.5,\ 4.0)\ \to$ 权重 $(0.04,\ 0.07,\ 0.02,\ 0.87)$。
存在的问题　重新计算：$e^{1.0}\approx 2.718$，$e^{1.5}\approx 4.482$，$e^{0.5}\approx 1.649$，$e^{4.0}\approx 54.598$，总和 ≈ 63.447，四个比值为 $0.0428,\ 0.0706,\ 0.0260,\ 0.8605$。
规范写法　保留两位小数应为 $(0.04,\ 0.07,\ 0.03,\ 0.86)$。
是否影响后续　不影响“分数越大占比越高”的结论；讲义 4.2 节的例题采用修正值。

原图纠错 · 笔记第 2 页：多头注意力公式与“向量距离”

原始写法　$\mathrm{MultiHead}(Q,K,V)=\mathrm{Concat}(W_1\cdots W_n)W^0$；“$QK^\mathsf{T}$：矩阵运算 → 向量距离计算”。
存在的问题　（1）被拼接的是各个头的**输出** $\mathrm{head}_1,\ldots,\mathrm{head}_h$，不是权重矩阵；末尾的上标是字母 O（Output），不是数字 0。（2）QK^T 算的是点积，点积衡量的是**相似度**：越大越相关；“距离”则是越小越接近，两者方向相反。
规范写法　$\mathrm{MultiHead}(Q,K,V)=\mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_h)\,W^O$（论文 3.2.2 节）；“$QK^\mathsf{T}$：一次算出所有 query 与所有 key 的点积相似度”。

原图纠错 · 其余几处表述，按论文原文校正

- **笔记第 3 页“最早用于文本翻译（中—英）”**。论文的实验是 WMT 2014 的**英语→德语**和**英语→法语**翻译；课件 p.47 的中英例句只是课堂示意。
- **课件 p.29“梯度消失或爆炸”**。论文 3.2.1 节的原话是：点积过大时 Softmax 会被推到“梯度极小”的区域。这里只有梯度变小这一个方向，没有“爆炸”。
- **课件 p.21 的架构示意图**。图中编码器内“前馈神经网络”画在“多头自注意力”下方，与论文图 1（先自注意力、后前馈）的顺序相反。学习时以课件 p.38、p.45 和讲义图 34 为准。
- **课件 p.43 的交叉注意力示意图**。图中把 Encoder 画成 $h_1\to h_2\to\cdots$ 的链，Decoder 各列之间也有横向箭头，这是 RNN 时代的画法。Transformer 里不存在“前一个位置把状态传给后一个位置”的连线，位置之间的信息交流全部通过（带掩码的）自注意力完成。

- **课件 p.50“模型选择概率最高的 token”与 p.55“从分布中采样”**。两者并不矛盾：前者是**贪心解码**，后者是**采样解码**，贪心是温度趋于 0 时的极限情形，见 8.6 节。
- **笔记第 3 页“Encoder-only 的目标函数：让每个词达到最终向量状态”**。训练并不存在一个事先规定好的“最终向量”。BERT 的目标函数是**掩码语言模型**的交叉熵，向量长成什么样是训练的结果而不是目标，见 8.2 节。

到这里，我们已经知道了什么？ 知道了要学哪六段、需要哪四件工具、三份资料各管什么、原始资料有哪几处需要修正。下一章先花两页看看这条线在历史上是怎样一步步长出来的——知道前人卡在哪里，后面每个设计才不显得突兀。

第二章　学科小史与现实应用

2.1　学科小史：这个知识是怎样出现的？

语言有一个让早期神经网络很头疼的特点：**长度不固定，而且顺序重要**。课件第 9 页的 MLP 只接受固定长度的输入，一句话可长可短，没法直接塞进去。此后三十年的主线，就是围绕“怎样让网络读懂一个序列”展开的。

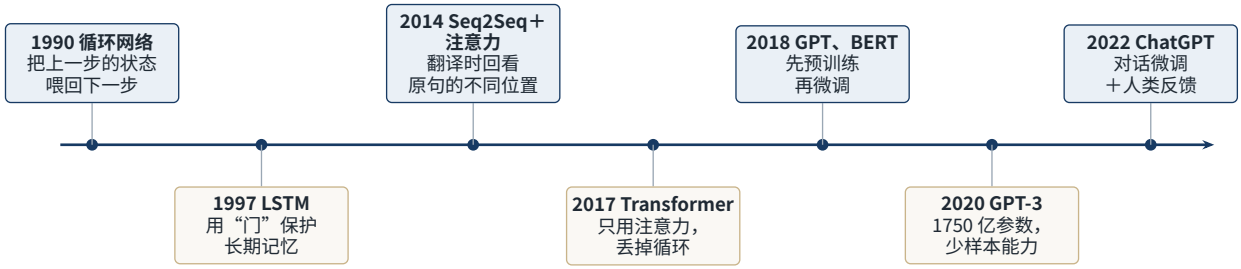


图 4　从循环网络到对话助手的七个节点（横轴仅示意先后，不按比例）

四个关键转折，各自解决了前一步的什么困难

① **让网络有“记忆”（约 1990 年）**。循环神经网络把上一时刻的隐藏状态作为这一时刻的输入之一，于是同一组权重可以读任意长的序列。Elman 在 1990 年的论文是这类结构的代表性工作之一。困难随即出现：序列一长，早先的信息在一次次传递中衰减，训练时梯度也跟着衰减。

② **给记忆加上“门”（1997 年）**。Hochreiter 与 Schmidhuber 提出 LSTM，用一条专门的记忆通道和几个可学习的开关，让信息可以长时间保留。此后近二十年，LSTM 是处理语言、语音的主力。但它仍然必须一个词一个词地算，无法并行。

③ **让解码器“回头看”（2014 年）**。用 RNN 做翻译时，整句原文要先被压进一个固定长度的向量，句子越长损失越多。Bahdanau 等人让解码器在生成每个词时对原句各位置分配不同的权重——这就是注意力机制，起初只是 RNN 的一个辅助部件。

④ **只留下注意力（2017 年）**。Vaswani 等八位作者的论文 Attention Is All You Need 提出 Transformer：完全不用循环，也不用卷积，只靠注意力建模词与词的关系。训练可以并行，任意两个词之间只隔一步。论文在英德翻译上取得 28.4 BLEU，英法翻译上取得 41.8 BLEU，训练成本只是此前最好模型的一小部分。

2018 年，同一种结构被拆成两半各自发展：OpenAI 的 GPT 只用解码器做生成，Google 的 BERT 只用编码器做理解，二者都采用“先在海量文本上预训练、再针对任务微调”的做法。此后参数规模迅速增长，2020 年的 GPT-3 达到 1750 亿参数；2022 年底的 ChatGPT 在预训练模型上加入对话微调与基于人类反馈的强化学习，语言模型第一次成为普通人日常使用的工具。 原 · 课件 p.6

这段历史想说明的只有一件事：**Transformer 里的每个零件都不是凭空设计的**。注意力是为了解决“长句信息被压扁”，去掉循环是为了能并行，残差连接是为了能堆得深——后面每学一个零件，都可以回到这里问一句“它是冲着哪个困难来的”。

2.2 从课本到现实：这个知识到底有什么用？

三个每天都在发生的应用

机器翻译。这是 Transformer 的出生地。把一句英文变成中文，正好对应“编码器读懂原文、解码器逐词写出译文”的完整结构（图 34）。如今的在线翻译系统普遍以这类结构为基础。

搜索与理解。判断一条查询和一篇网页是否相关、一段评论是正面还是负面，都需要先把文字变成“带上下文含义的向量”。这正是编码器（BERT 一类模型）擅长的事：同一个“苹果”，在“苹果手机”和“吃苹果”里会得到不同的向量（6.8 节）。

对话与写作助手。聊天、写代码、总结长文档，本质上都是“给定前文，一个词一个词往下写”，对应只用解码器的 GPT 一类模型（第 8 章）。课件 p.58 列出的写作、翻译、代码生成、长文档理解、工具调用等能力，都建立在这一个动作之上。

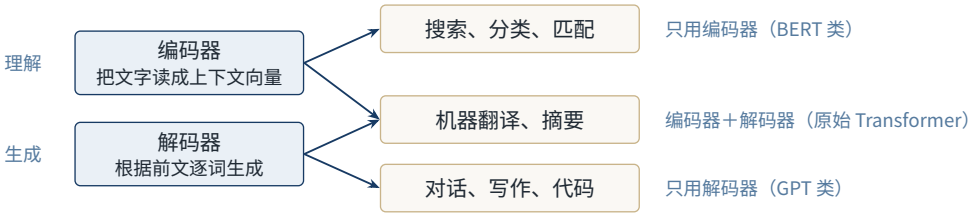


图 5　课堂知识在真实系统中的位置：三类应用对应 Transformer 的三种用法

注意力机制后来也走出了文字：把图片切成小块当作“词”，可以做图像识别；把氨基酸序列当作“句子”，可以预测蛋白质结构。这些属于**拓展联想**，当前阶段不要求掌握，只需知道本章学的是一种通用的“处理序列”的方法。

> **下一步还需要解决什么？**　历史告诉我们方向，但还没有回答第一个具体问题：这些“大模型”到底大在哪里，它和以前为单一任务训练的小模型有什么本质不同？下一章从课件的开头讲起。

第三章　大模型：为什么叫“打地基”

本章回答三个问题：大模型的正式说法是什么？它和以前的模型差在哪里？为什么一个模型能做那么多事？

3.1 它解决什么问题

在大模型出现之前，做一个情感分析系统要收集情感数据、训练一个情感模型；做翻译要另收集翻译数据、另训练一个翻译模型。每个任务从零开始，数据少的任务就做不好，模型之间也无法共享“懂语言”这件最基本的事。

人们很自然地想到：**能不能先让一个模型把“语言本身”学透，再让它去适应各个具体任务？**这就是大模型的出发点。

定义 · 大模型（Foundation Model，基础模型） 原 · 课件 p.4

在**海量数据**上**预训练**、拥有**数十亿甚至万亿**个参数、可以通过**微调**或**提示**适应多种下游任务的通用型人工智能模型。

“Foundation”的意思是“地基”：它不是为了完成某一个特定任务而训练的，而是其他下游模型和任务共同的底座——问答系统、情感分析、多模态生成都可以建在它上面。

定义里有四个词需要逐个解释，它们在后面各章都会再出现：

关键词	含义	后文展开
参数	模型里所有可以被训练调整的数（权重矩阵和偏置的每个元素）。“1750 亿参数”就是有 1750 亿个这样的数	6.9 节会亲手数一次
预训练	在不针对任何具体任务的大规模数据上先训练一遍，学到通用的语言能力	8.4 节
微调	在预训练好的参数基础上，用较少的任务数据继续训练	9.1 节
下游任务	相对于“预训练”这个上游而言的各种具体应用：翻译、摘要、问答……	第 8 章

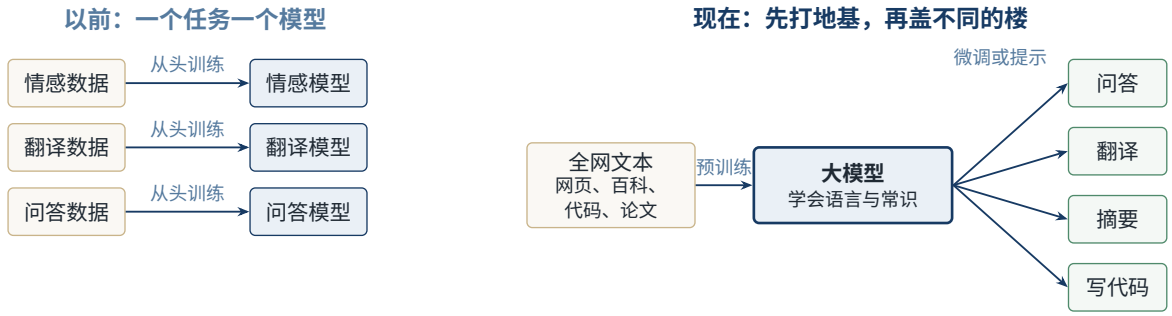


图 6　“打地基”的含义：预训练一次，底座被许多下游任务共用

图中左边每条箭头都要重新训练一个模型；右边只有“预训练”这一条粗箭头是昂贵的，之后通往各个任务的细箭头只需少量数据的微调，甚至只需换一句提示词。

3.2　三个比喻与五个关键特征

课件用三个比喻帮助建立直觉 原 · 课件 p.4 类比：**通才学生**——先接受通识教育，学会语言、知识、常识，再被训练去做写作、问答、翻译；**超级大脑**——不专攻某一科，而是带着世界知识，通过提示就能快速上手；**通用工程平台**——像一套绘图软件既能画建筑也能画桥梁，大模型是各种文本任务的通用平台。

比喻之外，课件 p.5 给出五个可以检验的特征：

特征	描述
超大参数规模	参数数亿到万亿，能表达更复杂的语言与知识结构
大规模预训练数据	通常使用网页、百科、代码、论文等大规模文本
统一训练范式	通过同一个“语言建模任务”学到可迁移的能力（第 4 章）
多任务适应性	一个模型可处理问答、翻译、摘要、对话等任务
可微调	可在特定数据或任务上继续训练以提升性能

其中第三条最值得留意：这么多能力，竟然来自**同一个**训练任务。这个任务是什么，正是下一章的主题。

3.3 与传统小模型的对比

比较维度	传统小模型	大模型
训练数据	一种任务的少量数据	大规模、多来源的文本
参数规模	万 ~ 千万	十亿 ~ 万亿
应用范围	单一任务	多任务、多语言、多模态
适应方式	从头训练	微调、提示工程、检索增强
泛化能力	弱，易过拟合	强，可零样本 / 少样本泛化

表中内容来自课件 p.7

教学补全 表中有两个术语课件没有解释。**零样本**（zero-shot）指不给任何示例，只用自然语言描述任务，模型就能完成；**少样本**（few-shot）指在提示里给几个示例，模型照着做。两者都**不改动模型参数**，这与微调不同。**过拟合**指模型把训练数据背了下来，换一批新数据就表现变差；参数多而数据少时最容易发生。

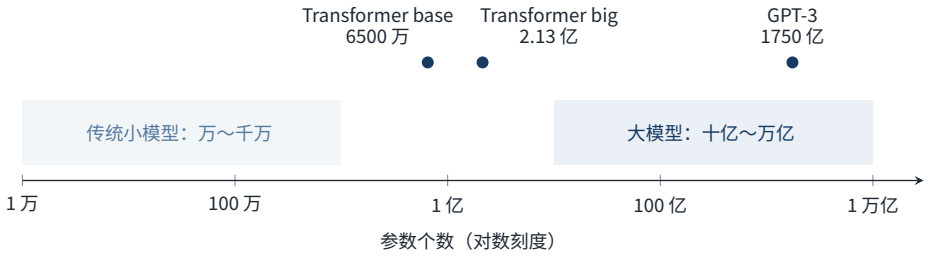


图 7　参数规模的对数轴。两段色带是课件 p.7 给出的范围；三个点分别取自论文表 3 与课件 p.6

横轴每一大格是 100 倍。2017 年论文里的两个 Transformer（6500 万、2.13 亿参数）按今天的标准还算不上“大”，三年后的 GPT-3 已经比它们大了近三个数量级——结构没有本质变化，变的主要是规模。

例 · 用定义判断：哪个算大模型？

（1）一个只用 5000 条影评训练、只会判断“好评 / 差评”的网络，有 20 万个参数。　**（2）**一个在大规模网页文本上预训练、有 70 亿参数、换一句提示就能做翻译或写摘要的模型。

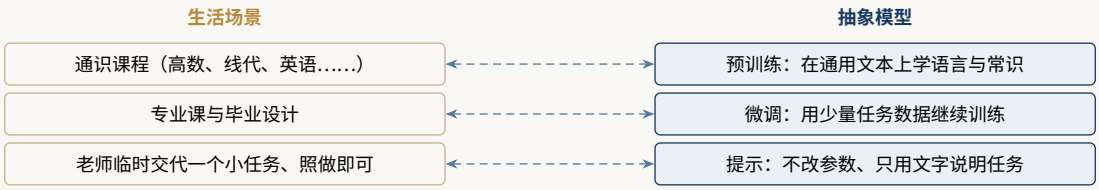
判断　对照定义的三个条件：海量数据预训练、超大参数规模、可适应多种下游任务。（1）三条都不满足，是典型的传统小模型；（2）三条都满足，是大模型。

本例想说明　“大”不只是参数多，更在于“先通用预训练、后适应多任务”这种用法。一个参数很多但只为单一任务从头训练的模型，不符合 Foundation 的本意。

	（1）影评小网络	（2）70 亿参数模型
海量数据预训练	×	✓
超大参数规模	×	✓
适应多种下游任务	×	✓

生活中的它 · 先上通识课，再分专业

生活场景 大一大二所有人都学高数、线代、英语、程序设计；到了高年级才分方向，同一批基础课支撑起控制、通信、计算机等不同专业。



一次小判断 一位同学转专业，需要把高数重学一遍吗？不需要，通识部分可以直接复用——对应“换下游任务不必重新预训练”。

边界说明 人的通识教育有课程设计；模型的预训练没有人逐条教，它只是在做下一章要讲的“预测下一个词”，通用能力是这个任务的副产品。

易错 · 三个常见误解 ①“**大模型＝参数多的模型**”。参数规模只是特征之一，关键是预训练加多任务适应。②“**微调就是重新训练**”。微调从预训练好的参数出发，只做小幅调整，所需数据和算力远小于预训练。③“**提示也会改变模型**”。提示只改变输入，模型参数一个都不变。

学完这里，你应该能够回答 （1）Foundation 一词强调的是什么？（2）预训练、微调、提示三者中，哪些会改动参数？（3）大模型与传统小模型在训练数据、适应方式上有什么不同？（4）课件说的“统一训练范式”指的是哪一个任务？——最后一问现在还答不出，下一章解决。

第四章　语言建模：标签从哪里来，分数怎样变成概率

本章回答三个问题：“语言建模”这个训练任务到底是什么？模型输出的一堆分数怎样变成概率？怎样衡量它猜得好不好？　**知识路线：**下一个词预测 → Softmax → 交叉熵。

上一章留下一个问题：那个“统一的训练任务”是什么。答案出奇地简单。

4.1　语言建模：预测下一个词

定义 · 语言建模（Language Modeling） 原 · 课件 p.12

给定一段文字的前面部分，预测**下一个字或词**是什么。完成这个任务的模型叫**语言模型**。更准确地说，模型输出的是“词表中每个候选词作为下一个词的**概率**”。

这个任务有一个极大的好处：**标签不需要人来标**。任何一段现成的文字，把它错开一位，前面的部分是输入，紧跟着的那个词就是正确答案。课件用“我想要吃饭”举例：

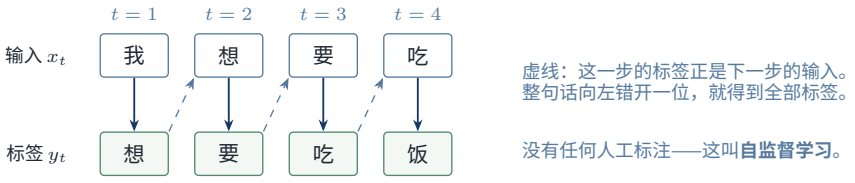
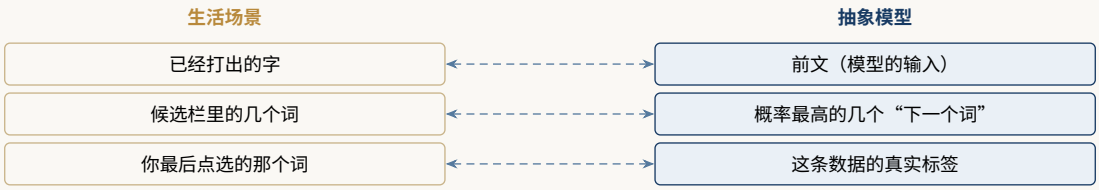


图 8　“我想要吃饭”的输入与标签：标签就是原句错位一格

生活中的它 · 输入法的联想词

生活场景 在手机上打出“今天天气”，输入法在候选栏里给出“很好”“不错”“怎么样”。 原 · 课件 p.12



一次小判断 打出“祝你生日”之后，“快乐”和“桌子”哪个应当排在前面？显然是“快乐”——语言模型要学的就是把这种直觉变成数值：给“快乐”很高的概率，给“桌子”接近 0 的概率。
边界说明 真实输入法还会结合你个人的输入历史和词频统计；这里只取“根据前文猜下一个词”这一点对应关系。

现在的问题是：神经网络最后一层输出的是一组**任意实数**（可正可负、加起来也不是 1），怎样把它变成“概率”？

4.2 Softmax：把分数变成比例

先看图

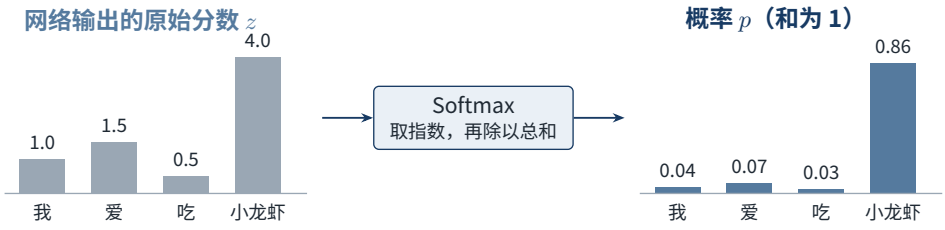


图 9　Softmax 的输入与输出（数值为课件 p.30 的例子，已按 1.5 节修正）

左图是四个可以随便取值的分数；右图是四个非负、加起来等于 1 的比例。大的分数得到大的比例，而且差距被**放大**了：4.0 只是 1.5 的不到 3 倍，对应的比例却是 0.86 比 0.07。

核心公式 · Softmax 函数

对一组实数 $z = (z_1, z_2, \ldots, z_n)$，

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}, \qquad i = 1, \ldots, n$$

每个符号的含义。 z_i：第 i 个候选的原始分数，深度学习里常叫 **logit**，可以是任意实数。e^{z_i}：对分数取自然指数，$e \approx 2.718$。分母：所有候选的指数之和，起“归一化”的作用。$\mathrm{softmax}(z)_i$：第 i 个候选得到的比例，下文常记作 p_i。n：候选的个数——在预测下一个词时是词表大小，在注意力里是句子长度。

为什么要先取指数？ 两个理由。其一，分数可能是负的，直接“除以总和”会得到负的比例，甚至分母为 0；而 e^z 对任何实数 z 都是正的。其二，指数函数增长很快，能把“分数高一点”放大成“比例高很多”，让模型的选择更明确。

结论 · Softmax 的四条基本性质

设 $p = \mathrm{softmax}(z)$，则：
（1）非负　每个 $p_i > 0$。　　**（2）归一**　$\sum_i p_i = 1$。
（3）保序　若 $z_i > z_j$，则 $p_i > p_j$。

（4）平移不变　给所有分数加上同一个常数 c，结果不变：$\mathrm{softmax}(z+c)=\mathrm{softmax}(z)$。

证明　记 $S=\sum_j e^{z_j}$。因为指数函数恒为正，所以 $S>0$。

（1）$p_i=e^{z_i}/S$，分子分母都为正，故 $p_i>0$。

（2）$\sum_i p_i=\sum_i e^{z_i}/S=S/S=1$。

（3）指数函数严格单调递增，$z_i>z_j\Rightarrow e^{z_i}>e^{z_j}$；两边同除以正数 S，得 $p_i>p_j$。

（4）每个分数加 c 后，第 i 项变为

$$\frac{e^{z_i+c}}{\sum_j e^{z_j+c}}=\frac{e^c\,e^{z_i}}{e^c\sum_j e^{z_j}}=\frac{e^{z_i}}{\sum_j e^{z_j}}=p_i,$$

其中第一个等号用了 $e^{a+b}=e^a e^b$，第二个等号约去公因子 e^c。 ■

性质（4）还带出一个很有用的观察：**两个候选的比例之比只取决于分数之差**，

$$\frac{p_i}{p_j}=\frac{e^{z_i}}{e^{z_j}}=e^{\,z_i-z_j}.$$

分数差 1，比例差 $e\approx 2.7$ 倍；差 2，约 7.4 倍；差 4，约 55 倍。所以**把所有分数同乘一个大于 1 的数**会把差距拉大、让分布变尖，**同乘一个小于 1 的数**会让分布变平。这一点是后面“为什么除以 $\sqrt{d_k}$”（6.6 节）和“温度”（8.6 节）的共同基础。

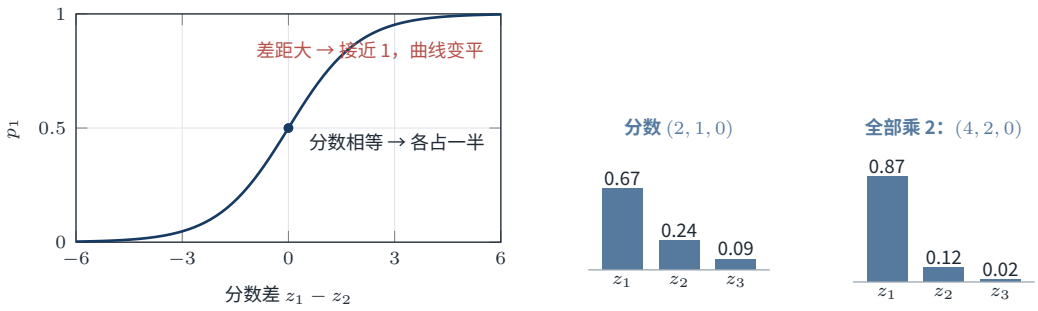


图 10　左：只有两个候选时，p_1 随分数差变化的曲线。右：分数同乘 2，分布变尖

左图是只有两个候选时的情形：$p_1=e^{z_1}/(e^{z_1}+e^{z_2})=1/(1+e^{-(z_1-z_2)})$，正是前面课程学过的 Sigmoid 函数。横轴是分数差，纵轴是第一个候选的概率。曲线中段陡、两端平——两端这段“平”的区域叫**饱和区**，在那里分数再变，概率几乎不动，梯度接近 0。记住这幅图，6.6 节要用。

例 · 亲手算一次 Softmax

已知　“吃”对四个词的原始分数：我 1.0，爱 1.5，吃 0.5，小龙虾 4.0。　**求**　四个注意力权重。

第一步，取指数。$e^{1.0}\approx 2.718$，$e^{1.5}\approx 4.482$，$e^{0.5}\approx 1.649$，$e^{4.0}\approx 54.598$。

第二步，求和。$S\approx 2.718+4.482+1.649+54.598=63.447$。

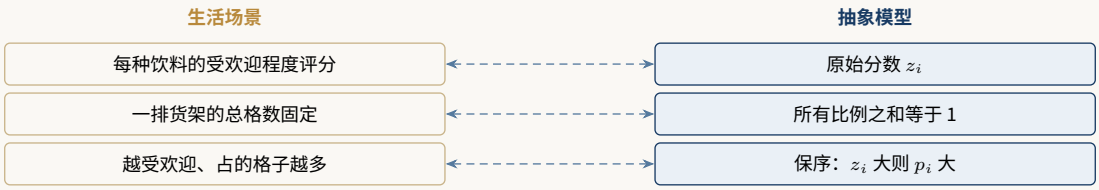
第三步，逐个相除。$\frac{2.718}{63.447}\approx 0.04$，$\frac{4.482}{63.447}\approx 0.07$，$\frac{1.649}{63.447}\approx 0.03$，$\frac{54.598}{63.447}\approx 0.86$。

检查　$0.04+0.07+0.03+0.86=1.00$；最大的分数对应最大的权重，顺序没有改变。结果就是图 9 右侧的四根柱子。

本例想说明　Softmax 只有三步：取指数、求和、相除。

生活中的它 · 按“热度”分配货架

生活场景 便利店只有一排货架，店长按各饮料的受欢迎程度分配位置：卖得好的多摆几格，卖得差的少摆，但每种至少留一点，所有格子加起来正好是一排。



一次小估算 两种饮料评分为 2 和 0。若按 Softmax 分配，比例之比是 $e^{2-0} \approx 7.4$，即约 88% 对 12%；若只按“评分＋1 再相除”这种线性办法，则是 75% 对 25%。Softmax 的分配更偏向强者。

边界说明 现实里没有人真的用指数函数排货架，这里只对应“把一组分数变成加起来为 1 的份额”这一点；取指数是数学上为了保证非负并放大差距。

易错 · Softmax 的三个高频错误 ① **忘记取指数，直接“除以总和”**。分数 $(1,-1)$ 直接相除会得到分母 0。② **以为输出与输入成正比**。分数翻倍，比例并不翻倍，而是差距被指数放大。③ **以为分数的绝对大小重要**。由性质（4），$(1,2,3)$ 与 $(101,102,103)$ 的 Softmax 结果完全相同，起作用的只有分数之间的**差**。

有了概率，下一个问题是：模型给出的这组概率，离“正确答案”有多远?

4.3 交叉熵损失：猜得好不好，用一个数来衡量

训练需要一个数来告诉模型“这次猜得有多差”，这个数就是**损失**（loss）。课件 p.13 只写了“交叉熵损失函数”六个字，这里把它补全。

核心公式 · 交叉熵损失（单个样本） 教学补全

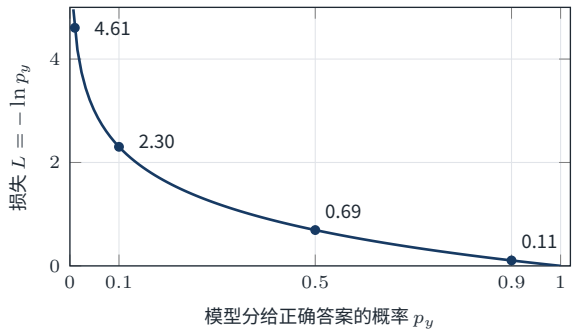
设模型给词表中每个词的概率为 $p_1,\dots,p_n$，真实的下一个词是第 y 个，则

$$\boxed{L = -\ln p_y}$$

即：**只看模型分给正确答案的那个概率，取自然对数，再加负号。**

每个符号的含义。 p_y：模型认为“正确答案是下一个词”的概率，在 0 与 1 之间。ln：自然对数。负号：因为 $0 < p_y \le 1$ 时 $\ln p_y \le 0$，加负号使损失非负。L：损失，越小越好，没有单位。

这个式子从哪里来? 教学补全 交叉熵的一般形式是 $L = -\sum_{i=1}^{n} t_i \ln p_i$，其中 $t = (t_1,\dots,t_n)$ 是“标准答案”的分布。在语言建模里，标准答案是确定的某一个词，所以 t 是**独热向量**：$t_y = 1$，其余全为 0。代入后求和号里只剩下 $i = y$ 这一项，就得到 $L = -\ln p_y$。



读图
横轴越靠右，模型越有把握地猜对；
纵轴是受到的“惩罚”。
$p_y = 1$：完全猜对，损失为 0。
$p_y = 0.5$：损失 $\ln 2 \approx 0.69$。
$p_y \to 0$：自信地猜错，损失趋于无穷。

图 11　交叉熵损失曲线。四个标出的点分别对应 $p_y = 0.9,\ 0.5,\ 0.1,\ 0.01$

曲线的形状回答了“为什么用对数”：猜对时惩罚很小，猜错得越离谱惩罚增长得越快，而且在 p_y

接近 0 时惩罚没有上限——这会迫使模型**永远不要给正确答案一个接近 0 的概率**。

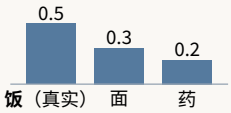
例 · 一步语言建模的损失

已知　前文是“我想要吃”，模型给出的概率为：饭 0.5，面 0.3，药 0.2。真实的下一个字是“饭”。　**求**　这一步的交叉熵损失。

解　正确答案是“饭”，$p_y = 0.5$，所以 $L = -\ln 0.5 = \ln 2 \approx 0.69$。

对比　如果模型给“饭”的概率是 0.9，则 $L = -\ln 0.9 \approx 0.11$；如果只给 0.1，则 $L \approx 2.30$。

本例想说明　损失只与“分给正确答案的概率”有关；其余概率怎样在错误答案之间分配，不影响这一步的损失（它们已经通过“和为 1”间接受到约束）。

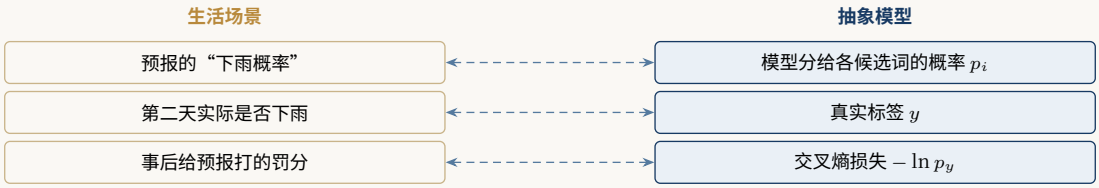


$$\Longrightarrow \quad L = -\ln 0.5 \approx 0.69$$

对一整句话，每个位置都有一个这样的损失，训练时取平均：$L = -\dfrac{1}{T}\sum_{t=1}^{T} \ln p_{y_t}^{(t)}$，其中 T 是位置个数。训练的全部内容，就是用梯度下降调整参数，让这个平均损失变小。

生活中的它 · 给天气预报员打分

生活场景　预报员每天说“明天下雨的概率是多少”。第二天真的下雨了，我们怎样评价他昨天的预报？

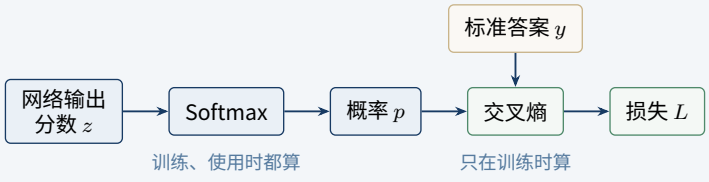


一次小估算　实际下了雨。甲昨天说“90% 下雨”，罚分 $-\ln 0.9 \approx 0.11$；乙说“10% 下雨”，罚分 $-\ln 0.1 \approx 2.30$，是甲的 20 多倍。

边界说明　气象部门评估概率预报时有多种评分规则，对数评分只是其中一种；这里只对应“越自信地说错，罚得越重”这一点。

易错 · 关于交叉熵的两个误解　①**“损失是 0 才算训练好”**。对语言来说下一个词本来就不唯一（“我想要吃”后面可以是饭、面、火锅……），好的模型应当把概率合理地分给多个可能，损失不可能、也不应该降到 0。②**“损失算的是猜错的比例”**。它算的是对数概率，不是对错计数；同样是“最高概率的词猜对了”，给 0.9 和给 0.4 的损失不同。

相近概念辨析：Softmax 与交叉熵　两者总是成对出现，但做的是不同的事：**Softmax 是模型的最后一步**，把分数变成概率，训练和使用时都要算；**交叉熵是训练时的“评分标准”**，把概率与标准答案比较得到损失，只在训练时用。前者输出一组数，后者输出一个数。



到这里，我们已经解决了什么？下一步呢？ 任务有了（预测下一个词）、标签有了（原文错位）、输出有了（Softmax 概率）、评分标准也有了（交叉熵）。还缺最关键的一块：**中间那个“读前文、给出分数”的网络长什么样？**前文可长可短，网络必须能处理任意长度的序列。历史上第一个答案是循环神经网络。

自检：（1）语言模型的标签是怎样得到的？（2）$\mathrm{softmax}(0,0)$ 等于多少？（3）$(1,2,3)$ 与 $(4,5,6)$ 的 Softmax 结果相同吗，为什么？（4）模型给正确答案的概率从 0.5 提高到 1，损失从多少变到多少？

第五章　循环神经网络与 LSTM：带着记忆逐词读

本章回答三个问题：RNN 的“循环”到底循环在哪里？它为什么又慢又健忘？LSTM 用什么办法缓解了健忘，又留下了什么没解决？　**知识路线：**RNN 定义 → 训练与共享权重 → 两大缺点 → LSTM → 遗留问题。

5.1　为什么需要一种新结构

上一章把任务、标签、输出、评分标准都准备好了，只缺“读前文”的网络。前面课程学过的多层感知机（MLP，课件 p.9）不能直接胜任，原因有两个：它的输入层神经元个数是固定的，而句子长短不一；它不区分输入的先后，而“狗咬人”和“人咬狗”含义完全不同。

需要的是这样一种网络：**一次读一个词，读的时候带着“前面读过什么”的记忆。**

5.2　RNN 的定义：循环在哪里

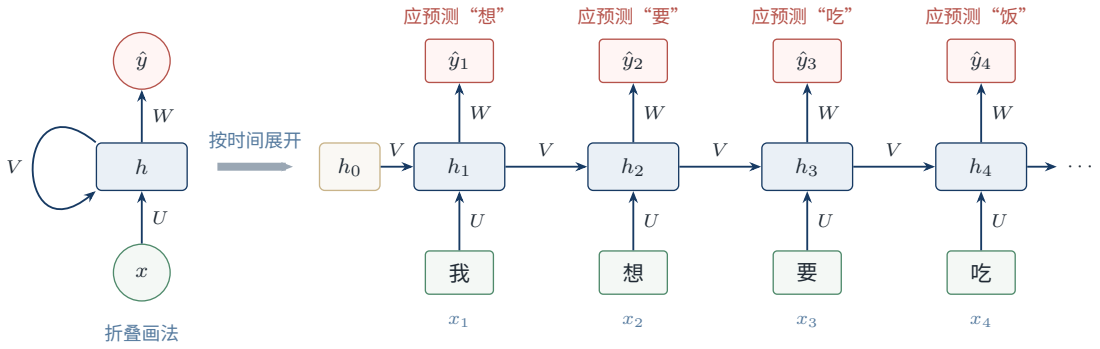


图 12　RNN 的两种画法。左：一个单元带一条指回自己的箭头；右：把这条箭头沿时间拉直（重绘自课件 p.10）

左图那条指回自己的箭头就是“循环”。右图把它沿时间展开后看得更清楚：**同一个单元（同一组权重 U、V、W）在每个时间步被重复使用一次**，上一步算出的 h_{t-1} 作为这一步的输入之一。图中有四个蓝色方框，但它们是同一个方框的四次使用，不是四个不同的网络。原 · 课件 p.11

定义 · 循环神经网络（Recurrent Neural Network，RNN）

专门处理**序列数据**（文本、语音、时间序列）的神经网络。它在每个时间步重复使用同一个单元，并把上一个时间步的**隐藏状态**传给下一个时间步，因而具有“记忆”能力。原 · 课件 p.10–11

核心公式 · RNN 的两个计算式 教学补全

$$\boxed{h_t = \tanh\left(Ux_t + Vh_{t-1} + b\right)}, \qquad \hat{y}_t = \mathrm{softmax}\left(Wh_t + c\right).$$

课件只写了 $h_t = f(x_t, h_{t-1})$，上式是这个 f 最常见的具体形式，字母沿用课件 p.10 图中的 U、V、W。

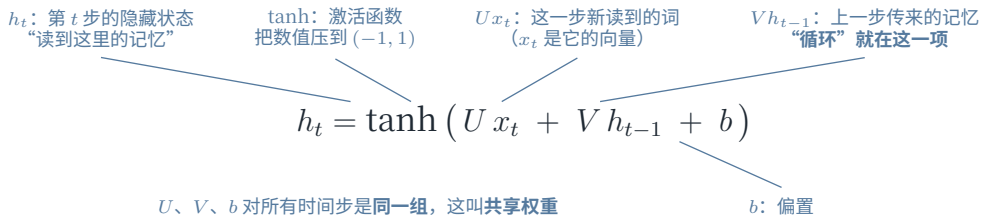


图 13　RNN 状态更新式的逐项标注

读法。这一步的记忆 h_t 由两部分混合而成：新读到的词 x_t 和旧记忆 h_{t-1}。初始记忆 h_0 通常取零向量（课件 p.13）。算出 h_t 后，再经过一个输出层和 Softmax，得到“下一个词是谁”的概率 $\hat{y}_t$。

使用条件。x_t、h_t 都是向量，U、V、W 是形状匹配的矩阵；tanh 对向量逐元素作用。因为每一步用的是同一组权重，所以序列多长都可以算——这解决了 MLP 输入长度固定的问题。原 · 课件 p.15“处理任意长度输入”

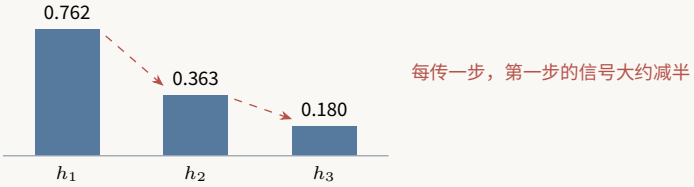
例 · 用一个标量 RNN 看“记忆”怎样传递

为了能手算，把所有量都取成一个数：$U = 1$，$V = 0.5$，$b = 0$，$h_0 = 0$。输入序列为 $x_1 = 1$，$x_2 = 0$，$x_3 = 0$——只有第一步有“信号”，后面两步什么也没读到。

第一步　$h_1 = \tanh(1 \times 1 + 0.5 \times 0) = \tanh(1) \approx 0.762$。

第二步　$h_2 = \tanh(1 \times 0 + 0.5 \times 0.762) = \tanh(0.381) \approx 0.363$。

第三步　$h_3 = \tanh(1 \times 0 + 0.5 \times 0.363) = \tanh(0.182) \approx 0.180$。



本例想说明　第一步的输入确实被“记住”了（h_2、h_3 都不为 0），这就是循环的作用；但它每传一步就衰减一次——这也预告了 RNN 的主要毛病。

5.3　RNN 怎样训练

把上一章的三件事接到图 12 上即可：每个时间步的 $\hat{y}_t$ 是 Softmax 概率，标签是原句的下一个字，损失是交叉熵，整句的损失是各步损失的平均。训练时调整的是那**同一组**共享权重。原 · 课件 p.13

时间步	输入 x_t	标签 y_t	这一步在算什么
$t = 1$	我	想	$h_1 = f(x_1, h_0)$，由 h_1 预测“想”
$t = 2$	想	要	$h_2 = f(x_2, h_1)$，由 h_2 预测“要”
$t = 3$	要	吃	$h_3 = f(x_3, h_2)$，由 h_3 预测“吃”
$t = 4$	吃	饭	$h_4 = f(x_4, h_3)$，由 h_4 预测“饭”

注意每一行的 h_t 都依赖上一行的 h_{t-1}：**不算完第 3 步，就无法开始第 4 步。**这句话是下面第一个缺点的根源。

5.4 RNN 的两个根本缺点

课件 p.15 列了三条缺点：不能并行训练、记忆力有限、训练难收敛。前两条是结构决定的，这里把原因讲透。

缺点一：不能并行

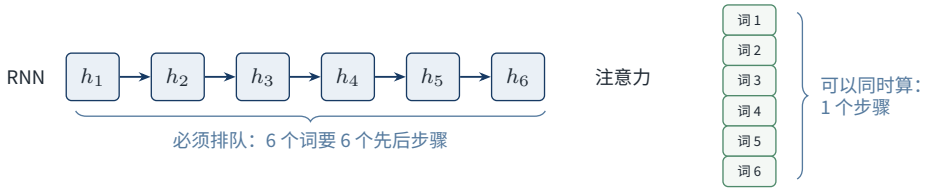


图 14　顺序计算与并行计算。RNN 的第 t 步必须等第 $t-1$ 步；下一章的注意力没有这种等待

现代计算硬件（GPU）擅长同时做大量互不依赖的运算。RNN 的各步环环相扣，长度为 n 的序列需要 n 个先后步骤，硬件再多也只能等。论文表 1 把这一点量化为“顺序操作数”：循环层是 $O(n)$，自注意力层是 $O(1)$。

缺点二：长距离依赖难以学习（梯度消失）

笔记里写的是“量大会忘”。“忘”有两层意思：前向计算时，早先的信息在一次次传递中被冲淡（上面的标量例子）；更要命的是**训练**时，损失对早先时间步的梯度也会一路衰减，网络根本学不会“让很久以前的词影响现在”。

结论 · 标量 RNN 的梯度连乘 教学补全

对标量 RNN $h_t = \tanh(u\,x_t + v\,h_{t-1})$，有

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^{T} v\,(1-h_t^2), \qquad \text{因而} \qquad \left|\frac{\partial h_T}{\partial h_1}\right| \le |v|^{\,T-1}.$$

当 $|v|<1$ 时，这个量随距离 $T-1$ **指数衰减**。

推导　**要证什么**　第 T 步的状态对第 1 步状态的导数是 $T-1$ 个因子的乘积，且每个因子的绝对值不超过 $|v|$。

第一步，相邻两步的导数。记 $a_t = u\,x_t + v\,h_{t-1}$，则 $h_t = \tanh(a_t)$。由 $\tanh'(a) = 1-\tanh^2(a)$ 和链式法则，

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(a_t)\cdot\frac{\partial a_t}{\partial h_{t-1}} = (1-h_t^2)\cdot v.$$

第二步，跨多步连乘。h_T 通过 h_{T-1} 依赖 h_{T-2}，如此一直到 h_1。反复使用链式法则，

$$\frac{\partial h_T}{\partial h_1} = \frac{\partial h_T}{\partial h_{T-1}}\cdot\frac{\partial h_{T-1}}{\partial h_{T-2}}\cdots\frac{\partial h_2}{\partial h_1} = \prod_{t=2}^{T} v\,(1-h_t^2).$$

第三步，估计大小。因为 $-1<h_t<1$，所以 $0<1-h_t^2\le 1$，每个因子满足 $|v(1-h_t^2)|\le|v|$。共 $T-1$ 个因子，故乘积的绝对值不超过 $|v|^{T-1}$。 ■

一句话解释　梯度每往回传一步就要乘一个数；这个数小于 1，传得越远乘得越多，结果就趋于 0。

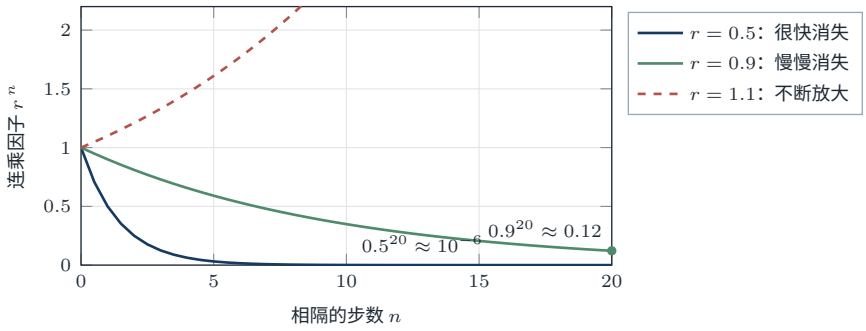


图 15　同一个因子连乘 n 次。横轴是两个词相隔的步数，纵轴是梯度被缩放的倍数

横轴是“隔了多少步”，纵轴是梯度被乘上的倍数。蓝线（每步乘 0.5）在 10 步之后就贴到了横轴上：相隔 20 个词时，梯度只剩约百万分之一，远处的词实际上“教不到”。红色虚线是另一个极端——因子大于 1 时梯度越乘越大，称为**梯度爆炸**，表现为训练震荡。课件说 RNN“对超参数敏感，容易震荡”，根源正是这种连乘。

边界说明。真实 RNN 的 h 是向量，v 换成矩阵 V，“乘一个数”换成“乘一个矩阵”，结论的形状相同（看矩阵把向量放大还是缩小），严格分析需要矩阵范数的工具，超出本课范围。

生活中的它 · 传话游戏

生活场景 十个人排成一排，第一个人把一句话悄悄告诉第二个人，依次往后传。传到最后，原话往往面目全非。

生活场景	抽象模型
每个人只听前一个人说的话	h_t 只直接依赖 h_{t-1}
每传一次丢一点细节	每步乘一个小于 1 的因子
队伍越长、最后越走样	相隔越远、梯度越接近 0

一次小估算 假设每传一次能保留 90% 的信息，传 20 次后剩下 $0.9^{20} \approx 12\%$；若每次只保留一半，20 次后剩约百万分之一。

边界说明 传话丢的是内容，梯度消失丢的是“训练信号”；两者机制不同，但“逐级传递、逐级衰减”的结构是一样的。下一章的注意力相当于让最后一个人**直接去问第一个人**。

易错 · 关于 RNN 的三个误解 ①**“展开图里有几个方框就有几组权重”**。只有一组，被重复使用。②**“RNN 完全记不住前文”**。记得住，只是越远越淡；短句上 RNN 工作得不错。③**“梯度消失是指梯度整体变成 0”**。消失的是损失对**远处时间步**的梯度分量，近处的梯度依然正常，所以模型会“只学会短距离规律”。

既然问题出在“信息每一步都要被 tanh 和权重加工一遍”，一个自然的想法是：**能不能给信息修一条不被反复加工的“直通车道”？**这就是 LSTM。

5.5 LSTM：给记忆修一条直通车道

定义 · 长短期记忆网络（Long Short-Term Memory，LSTM） 原 · 课件 p.17

在 RNN 的基础上引入两样专门的结构：**记忆单元**（Cell，记作 c_t）——一条贯穿各时间步的信息通道；**门控机制**（Gates）——三个取值在 0 到 1 之间的“阀门”，分别控制忘掉什么、写入什么、输出什么。

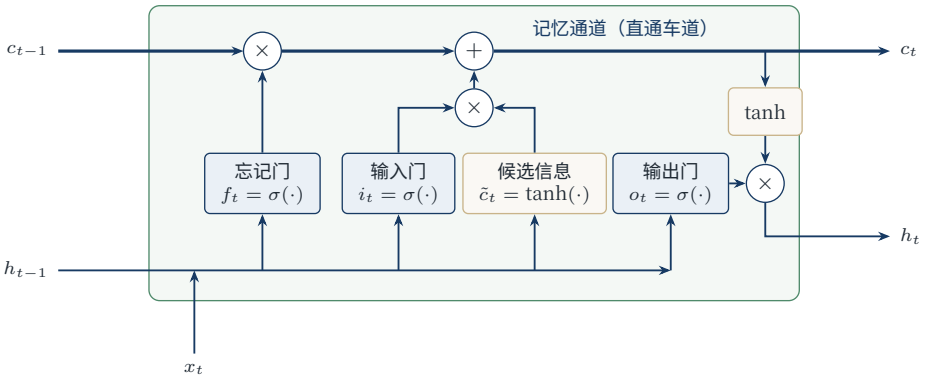


图 16　LSTM 单元的结构（按课件 p.17 的图重绘）。上方粗线是记忆通道，下方四个方框由 h_{t-1} 与 x_t 共同驱动

图中最重要的是上方那条粗线：c_{t-1} 从左到右只经过一次“乘”和一次“加”就变成 c_t，中间**没有**被 tanh 压缩，也没有乘权重矩阵。下方的四个方框都读取同样的输入（上一步的输出 h_{t-1} 和这一步的词 x_t），各自算出一个向量。σ 是 Sigmoid 函数，输出在 0 到 1 之间，正好可以当作“阀门开度”。

门	符号	作用	类比 类比
忘记门	f_t	决定旧记忆 c_{t-1} 的每一维保留多少（1 全留，0 全忘）	清理无用记忆
输入门	i_t	决定候选新信息 $\tilde{c}_t$ 的每一维写入多少	学进来新的知识
输出门	o_t	决定记忆 c_t 中有多少要作为这一步的输出 h_t	说出口的内容

表中内容来自课件 p.17

核心公式 · LSTM 的六个计算式 教学补全

记 $[h_{t-1}, x_t]$ 为两个向量首尾拼接，$\odot$ 为逐元素相乘。

$$
\begin{aligned}
f_t &= \sigma\big(W_f[h_{t-1}, x_t] + b_f\big) && \text{忘记门}\\
i_t &= \sigma\big(W_i[h_{t-1}, x_t] + b_i\big) && \text{输入门}\\
\tilde{c}_t &= \tanh\big(W_c[h_{t-1}, x_t] + b_c\big) && \text{候选新信息}\\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \textbf{记忆更新（核心）}\\
o_t &= \sigma\big(W_o[h_{t-1}, x_t] + b_o\big) && \text{输出门}\\
h_t &= o_t \odot \tanh(c_t) && \text{本步输出}
\end{aligned}
$$

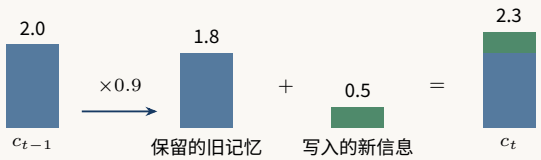
课件只给出了三个门的名字，上面六式是教材中最常见的标准写法。六个式子里真正要记的只有第四个：**新记忆＝保留下来的旧记忆＋允许写入的新信息**。其余五个都在为它准备“开度”和“原料”。

为什么这样能缓解梯度消失？　沿着记忆通道看，c_t 对 c_{t-1} 的直接依赖只有 $f_t \odot c_{t-1}$ 这一项，所以沿这条路每回传一步，梯度乘的是 f_t，而不是 RNN 里的 $v(1-h_t^2)$。f_t 是网络**自己学出来的**：需要长期记住某件事时，它可以把对应维度的 f_t 学到接近 1，于是连乘 20 次仍有 $0.9^{20} \approx 0.12$ 甚至更多（即

图 15 中绿线与蓝线的差别）。
以下为直观解释，不替代严格证明：完整的梯度还包含经由各个门的间接路径，严格分析较复杂；这里只说明“存在一条不必衰减的通路”。

例 · 手算一步 LSTM（标量）

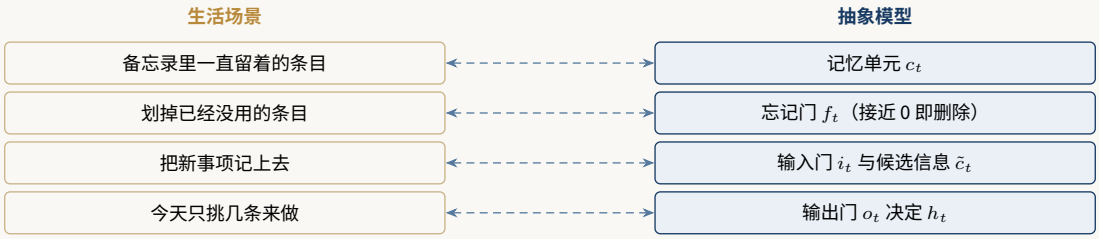
已知　旧记忆 $c_{t-1}=2$；三个门和候选值已经算好：$f_t=0.9$，$i_t=0.5$，$\tilde{c}_t=1$，$o_t=0.5$。　**求**　c_t 和 h_t。
第一步，记忆更新。 $c_t = f_t\,c_{t-1} + i_t\,\tilde{c}_t = 0.9\times 2 + 0.5\times 1 = 1.8 + 0.5 = 2.3$。
第二步，本步输出。 $h_t = o_t\tanh(c_t) = 0.5\times\tanh(2.3) \approx 0.5\times 0.980 = 0.490$。



本例想说明　旧记忆的 90% 被原样带到了下一步，没有经过 tanh 的压缩；这就是“直通车道”的含义。对比上一个例子里 RNN 每步大约减半。

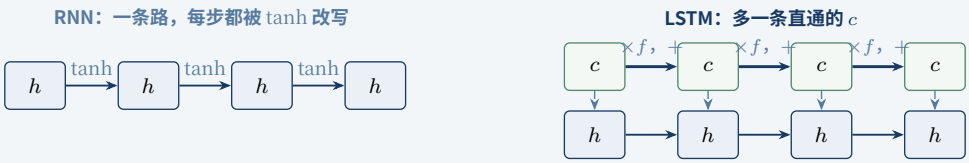
生活中的它 · 手机里的备忘录

生活场景 每天打开备忘录：先划掉已经办完的事，再添上今天新来的事，最后挑出几条“今天要做的”念给自己听。



一次小判断　“下个月交论文”这条要保留很多天。对应到 LSTM，它的忘记门应当接近 1 还是接近 0？接近 1——每天原样带到下一天。
边界说明 备忘录的条目要么留要么删；LSTM 的门是 0 到 1 之间的连续值，可以“留下七成”，而且哪些该留是训练学出来的，不是人规定的。

相近概念辨析：RNN 的 h_t 与 LSTM 的 c_t、h_t　RNN 只有一个状态 h_t，既当“长期记忆”又当“本步输出”，每步都被整体改写。LSTM 把两个角色分开：c_t 负责长期保存，沿直通车道传递；h_t 是从 c_t 里“挑出来给外面看”的部分。



5.6 LSTM 仍然留下的问题

LSTM 让“记得更久”成为可能，在 Transformer 出现之前的很多年里一直是处理语言的主力。但它没有改变“一步接一步”的基本结构，课件 p.19 总结了三个遗留问题：

问题	描述	根源
序列计算慢	无法并行，只能一个时间步一个时间步处理	h_t、c_t 仍依赖上一步的结果
长依赖难捕捉	相距很远的词之间传递信息仍然困难（如前文主语与后文动词）	信息仍要“路过”中间每一个位置
难以扩展	对很长的文本效果受限	前两条叠加：又慢、又远

易错 · 不要把 LSTM 理解成“解决了”梯度消失　课件和笔记的用词是“引入专门结构”，更准确的说法是**缓解**。忘记门只要小于 1，记忆仍会逐步衰减；而且相距 n 的两个词，信息仍要走 n 步。LSTM 改善的是“每一步丢多少”，没有改变“要走多少步”。

到这里，我们已经解决了什么？下一步呢？　RNN 用“循环”读任意长度的序列；LSTM 用记忆通道和三个门让信息保留得更久。但两者都要排队计算，远处的词之间都隔着许多步。Transformer 的做法是**完全抛弃 RNN 的时间结构，转而用注意力机制直接建模句子中任意两个词之间的依赖关系**原 · 课件 p.19。下一章就来拆开“注意力”这三个字。

自检：（1）RNN 展开图中的多个单元共用几组权重？（2）RNN 为什么不能并行？（3）梯度每回传一步乘 0.5，回传 10 步后剩多少？（4）LSTM 的记忆更新式是什么？哪个门接近 1 时旧记忆被保留？（5）LSTM 没有解决 RNN 的哪个问题？

第六章　注意力机制：让每个词直接看见所有词

本章回答四个问题：注意力想做什么？文字怎样变成可以计算的向量？公式 $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})V$ 的每一步在算什么、为什么这样算？只有注意力还缺什么？　**知识路线：**直觉 → Embedding → 点积 → 缩放点积注意力 → 自注意力 → 多头 → 位置编码。

6.1　换一个思路：为什么非要排队？

上一章结尾留下两个问题：RNN 必须排队计算；相距很远的词之间隔着许多步。论文的回答很干脆——**让每个词直接“看见”句子里的其他所有词**。

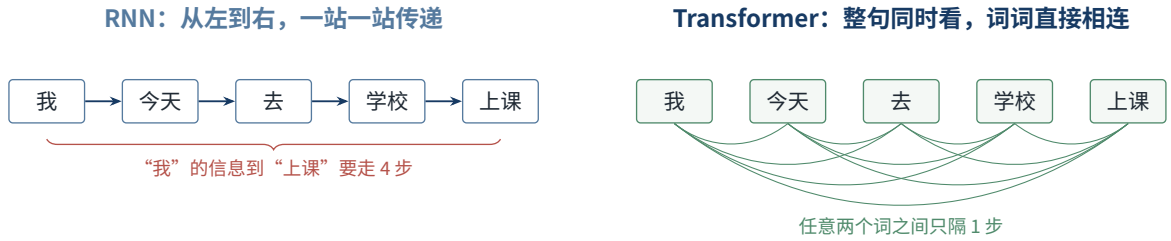


图 17　两种读句子的方式（重绘自课件 p.22）。左：信息沿链条传递；右：任意两个词直接相连

论文表 1 把这两种结构的差别量化成三个指标。这张表课件没有放，它却是“为什么要换结构”的定量依据：教学补全

层的类型	每层计算量	顺序操作数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环（RNN）	$O(n \cdot d^2)$	$O(n)$	$O(n)$

摘自论文表 1。n 为序列长度，d 为向量维数

三列怎么读。顺序操作数是“必须先后进行、不能并行的步骤数”，越小越适合并行：RNN 是 n，自注意力是常数。**最大路径长度**是“任意两个位置之间，信息最多要走几步”，越短越容易学到长距离依

赖：RNN 是 n，自注意力是 1。**每层计算量**上自注意力并不总占便宜：它与 n^2 成正比，论文指出当句子长度 n 小于维数 d 时它比循环层快——句子很长时，n^2 这一项就成了负担。

6.2 注意力的直觉：词的意思由关系决定

先看课件的两个句子：原 · 课件 p.23

① 小明把**苹果**放进**书包**，因为它很**重**。

② 小明把**苹果**放进**书包**，因为它很**甜**。

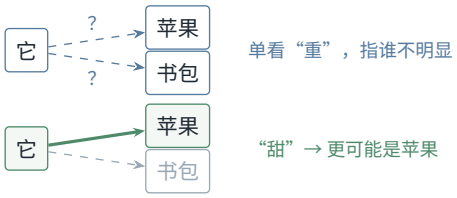


图 18　“它”指谁，取决于句子里的其他词

只换了最后一个字，“它”的指向就变了。可见**一个词的意思不是孤立决定的，而是由它和其他词的关系决定的**。注意力机制做的就是这件事：理解一个词时，让模型自己判断“句子里哪些词对我最重要”，并给出一组权重。

把一句话里所有词两两之间的权重排成一张表，就是**注意力权重矩阵**：



图 19　注意力权重矩阵（数值取自课件 p.25，“吃”一行已按 1.5 节修正）

这张表有一条必须满足的性质：**每一行加起来等于 1**，因为每一行都是一次 Softmax 的结果。列则没有这个要求。接下来的任务，就是弄清这张表里的数是怎样算出来的。第一步是把文字变成数字。

6.3 文字怎样变成神经网络能计算的数字

神经网络只会做数值运算，不认识汉字。从文字到数字要走三步：原 · 课件 p.26



图 20　从文本到向量的三步。Token ID 的具体数值为课件示例

定义 · Token、Token ID 与 Embedding

Token：模型切分文本后得到的基本单位，可以是一个字、一个词，也可以是词的一部分（子词）。

Token ID：事先建立一张**词表**，给每个 token 一个固定的整数编号；这个编号本身没有含义。

Embedding（词嵌入）：把每个 Token ID 对应到一个长度为 d_{model} 的实数向量。**它不是文字本身，而是文字在高维空间中的“数字坐标”**。

笔记在这里留了两个问题，课件都没有写答案。

问题一：编号怎么变成向量？ 教学补全 答案是**查表**。模型里有一个矩阵 E，有 $|\mathcal{V}|$ 行（$|\mathcal{V}|$ 是词表大小）、d_{model} 列，第 k 行就是编号为 k 的 token 的向量。把编号变成向量，只是把这一行取出来：

$$\text{token 的编号为 } k \quad \Longrightarrow \quad \text{它的向量} = E \text{ 的第 } k \text{ 行.}$$

表里的数最初是随机的，和网络的其他权重一样，在训练中靠梯度下降一点点学出来。没有人规定“苹果”的向量应该是多少——训练会让“用法相近的词”得到相近的向量，因为这样有利于预测下一个词。

问题二：向量的维数怎么确定？ 教学补全 d_{model} 是一个**超参数**：训练前由人选定，训练中不变。论文的基础模型取 $d_{\text{model}} = 512$，大模型取 1024。维数越大，能区分的含义越细，但参数和计算量也越大；它不是算出来的，而是靠实验比较选出来的（论文表 3 比较了多组取值）。

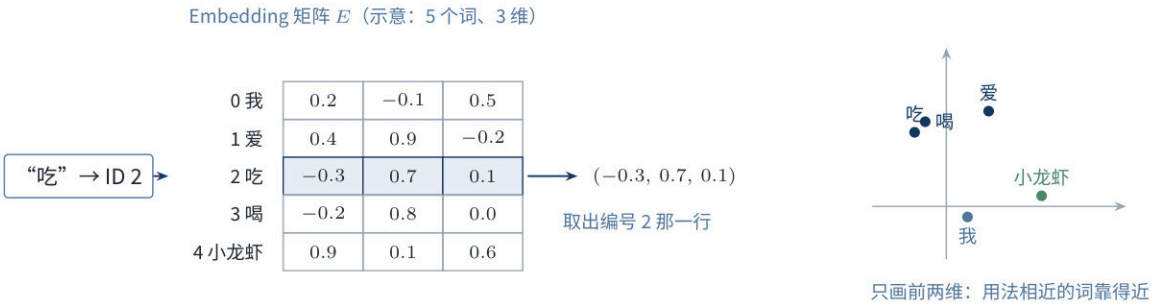


图 21　左：Embedding 就是按编号查表取一行。右：把向量画成点，“吃”和“喝”离得近（数值为示意）

例 · 查表与“独热向量乘矩阵”是同一回事

已知　图 21 中的矩阵 E；“吃”的编号是 2（从 0 数起，即第 3 行）。

查表　直接取出该行：$(-0.3,\ 0.7,\ 0.1)$。

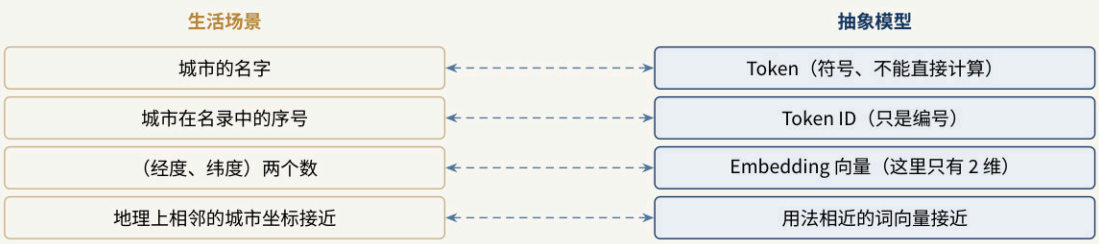
另一种写法　用长度为 5 的**独热向量** $\boldsymbol{e} = (0, 0, 1, 0, 0)$ 表示“第 3 个位置是 1，其余是 0”，那么 $\boldsymbol{e}E = 0 \cdot (\text{第 1 行}) + 0 \cdot (\text{第 2 行}) + 1 \cdot (\text{第 3 行}) + 0 \cdot (\text{第 4 行}) + 0 \cdot (\text{第 5 行}) = (-0.3,\ 0.7,\ 0.1)$。

本例想说明　Embedding 层本质上是一个没有偏置的线性层，输入是独热向量；“查表”只是它的高效实现。所以 E 能像其他权重一样被梯度下降训练。

教学补全 论文 3.4 节还有两个课件未提的细节：Embedding 向量在使用前要乘以 $\sqrt{d_{\text{model}}}$；输入端的 Embedding 矩阵与输出端 Softmax 之前的线性层共用同一个权重矩阵。当前阶段知道有这两件事即可。

生活中的它 · 城市的经纬度

生活场景　“甲地”“乙地”“北京”只是三个名字，名字之间没法做加减。给每座城市配上（经度，纬度）两个数之后，就能计算谁离谁近。



一次小估算　甲地约在（117°E，36.7°N），乙地约在（120.4°E，36.1°N），北京约在（116.4°E，39.9°N）。只看经度差：甲地—乙地约 3.4°，甲地—北京约 0.6°；只看纬度差则分别约 0.6° 和

3.2°。两个维度各自表达了不同方向上的远近——Embedding 的每一维也类似，只是有几百维。
边界说明 经纬度的每一维都有明确含义，而且是测量出来的；Embedding 的各维没有人能直接解释的含义，是训练学出来的。

易错 · 关于 Embedding 的两个误解 ① **“Token ID 越接近，词义越接近”**。ID 只是词表里的排号，101 号和 102 号毫无关系；含义相近体现在向量接近。② **“一个 token 就是一个汉字或一个单词”**。Token 是 tokenizer 切出来的单位，可能比词小（子词），也可能是多个字合在一起。

现在每个词都是一个向量了。下一步：怎样用向量算出“两个词有多相关”？

6.4 点积：衡量两个向量有多“对得上”

核心公式 · 向量的点积

对两个 d 维向量 $\boldsymbol{a}=(a_1,\dots,a_d)$、$\boldsymbol{b}=(b_1,\dots,b_d)$，

$$\boldsymbol{a}\cdot\boldsymbol{b}=\sum_{i=1}^{d}a_ib_i=|\boldsymbol{a}|\,|\boldsymbol{b}|\cos\theta,$$

其中 θ 是两个向量的夹角，$|\boldsymbol{a}|$ 是向量的长度。

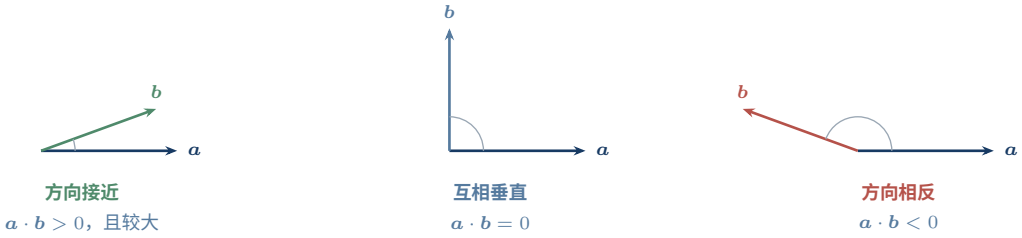


图 22　点积的几何意义：两个向量方向越一致，点积越大

三幅小图里 $\boldsymbol{a}$ 不变，只改变 $\boldsymbol{b}$ 的方向。夹角小，$\cos\theta$ 接近 1，点积大；垂直时为 0；方向相反时为负。所以点积可以当作一种**相似度打分**：分数越高，两个向量越“对得上”。注意它和“距离”相反——距离是越小越接近（1.5 节已就笔记的写法作了纠正）。

例 · 三个点积

设 $\boldsymbol{q}=(2,1)$。　$\boldsymbol{k}_1=(2,1)$：$\boldsymbol{q}\cdot\boldsymbol{k}_1=2\times2+1\times1=5$。　$\boldsymbol{k}_2=(-1,2)$：$\boldsymbol{q}\cdot\boldsymbol{k}_2=-2+2=0$。　$\boldsymbol{k}_3=(-2,-1)$：$\boldsymbol{q}\cdot\boldsymbol{k}_3=-4-1=-5$。
分别对应图 22 的三种情形：同向、垂直、反向。**本例想说明**　点积把“像不像”变成了一个可以比较大小的数。

一次算出所有点积。如果有 n 个 query 向量排成矩阵 Q（每行一个），n 个 key 向量排成矩阵 K（每行一个），那么由矩阵乘法的定义，

$$(QK^{\mathsf{T}})_{ij}=Q\text{ 的第 }i\text{ 行}\cdot K^{\mathsf{T}}\text{ 的第 }j\text{ 列}=\boldsymbol{q}_i\cdot\boldsymbol{k}_j.$$

也就是说，QK^{T} 这个 $n\times n$ 矩阵的第 i 行第 j 列，正是第 i 个 query 与第 j 个 key 的点积。公式里的转置 ^{T}（笔记用红笔标出）只是为了让形状对得上：Q 是 $n\times d_k$，K^{T} 是 $d_k\times n$，乘出来是 $n\times n$。

6.5 缩放点积注意力：公式的五步

核心公式 · 缩放点积注意力（Scaled Dot-Product Attention） 原 · 论文式 (1)，课件 p.27

$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right)V$$

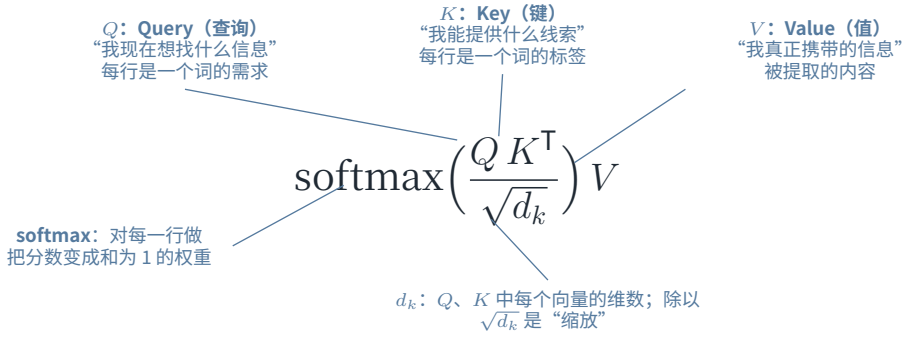


图 23　注意力公式的逐项标注

Q、K、V 这三个名字来自信息检索：拿着查询去和每个条目的键比对，比对上了就取回对应的值。区别在于，这里不是只取回“最匹配的那一个”，而是**按匹配程度把所有的值加权混合**。

Q、K、V 从哪里来？ 在自注意力里，它们都来自同一组输入向量 X（n 行，每行是一个词的 d_{model} 维向量），各自经过一个**可学习的线性变换**：

$$Q=XW^{Q},\qquad K=XW^{K},\qquad V=XW^{V}.$$

W^{Q}、W^{K} 是 $d_{\text{model}}\times d_k$ 的矩阵，W^{V} 是 $d_{\text{model}}\times d_v$ 的矩阵，都是训练出来的参数。同一个词因此有三个“分身”：作为提问者的 $\boldsymbol{q}$、作为被查对象的 $\boldsymbol{k}$、作为信息提供者的 $\boldsymbol{v}$。课件 p.28 的说法是：“Q 来自当前词自己，K 来自句子中所有 token。”

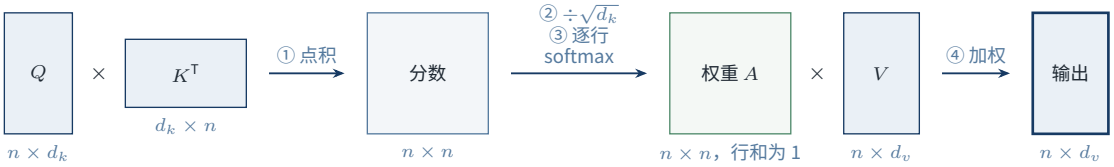


图 24　各个矩阵的形状。输入 n 个向量，输出仍是 n 个向量

使用条件。（1）Q 与 K 的列数必须相同（都是 d_k），否则点积无法计算。（2）K 与 V 的行数必须相同，因为第 j 个键和第 j 个值属于同一个词。（3）Softmax 是对分数矩阵的**每一行**分别做的。（4）输出的行数等于 Q 的行数——有多少个提问者，就有多少个回答。

把矩阵公式拆开，只看第 i 个词得到的输出，会更容易理解：

$$\alpha_{ij}=\frac{\exp\left(\boldsymbol{q}_i\cdot\boldsymbol{k}_j/\sqrt{d_k}\right)}{\sum_{l=1}^{n}\exp\left(\boldsymbol{q}_i\cdot\boldsymbol{k}_l/\sqrt{d_k}\right)},\qquad \boldsymbol{o}_i=\sum_{j=1}^{n}\alpha_{ij}\,\boldsymbol{v}_j.$$

α_{ij} 是“第 i 个词分给第 j 个词的注意力”，$\boldsymbol{o}_i$ 是第 i 个词的输出。课件 p.27 把整个过程概括为五步，核心是三个动作：**打分** → **归一化** → **提取信息**。

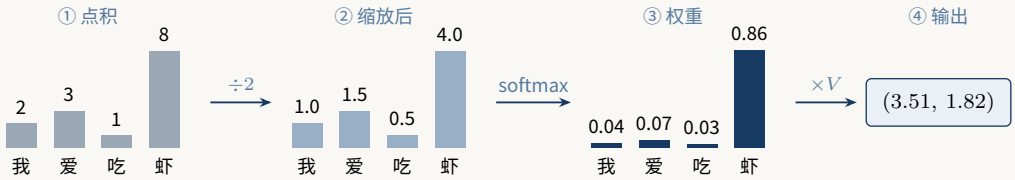
例 · 完整走一遍：计算“吃”的注意力输出

句子“我 爱 吃 小龙虾”，取 $d_k = 4$。假设线性变换之后得到（数值为手算方便而设）：

$$\boldsymbol{q}_{\text{吃}} = (2,1,0,1); \qquad \boldsymbol{k}_{\text{我}} = (1,0,0,0),\ \boldsymbol{k}_{\text{爱}} = (1,1,0,0),\ \boldsymbol{k}_{\text{吃}} = (0,1,0,0),\ \boldsymbol{k}_{\text{虾}} = (2,2,0,2).$$

① **点积打分**。$\boldsymbol{q}\cdot\boldsymbol{k}_{\text{我}} = 2$，$\boldsymbol{q}\cdot\boldsymbol{k}_{\text{爱}} = 2+1 = 3$，$\boldsymbol{q}\cdot\boldsymbol{k}_{\text{吃}} = 1$，$\boldsymbol{q}\cdot\boldsymbol{k}_{\text{虾}} = 4+2+2 = 8$。
② **缩放**。$\sqrt{d_k} = \sqrt{4} = 2$，四个分数变为 1.0, 1.5, 0.5, 4.0。
③ **Softmax**。这正是 4.2 节算过的那组分数，权重为 0.04, 0.07, 0.03, 0.86。
④ **加权求和**。设四个值向量为 $\boldsymbol{v}_{\text{我}} = (1,0)$，$\boldsymbol{v}_{\text{爱}} = (0,1)$，$\boldsymbol{v}_{\text{吃}} = (1,1)$，$\boldsymbol{v}_{\text{虾}} = (4,2)$，则

$$\boldsymbol{o}_{\text{吃}} = 0.04(1,0) + 0.07(0,1) + 0.03(1,1) + 0.86(4,2) = (3.51,\ 1.82).$$



检查　输出 $(3.51, 1.82)$ 很接近 $\boldsymbol{v}_{\text{虾}} = (4,2)$，因为“小龙虾”占了 86% 的权重。**本例想说明**　整个公式只是把前面学过的三件事——点积、Softmax、加权求和——依次做一遍。

生活中的它 · 在图书馆的检索机上找书

生活场景 在检索机里输入“深度学习 入门”，系统把这个查询和每本书的书名、关键词比对，按相关程度排序，然后你去取回最相关的几本来读。

生活场景	抽象模型
你输入的检索词	Query：当前词想找什么
每本书的书名与关键词	Key：每个词可供匹配的线索
书架上那本书的正文	Value：真正被取回的信息
相关度排序	点积打分 → Softmax 权重

一次小判断 检索“深度学习 入门”，《深度学习入门》和《红楼梦》哪一本的“键”与查询的点积大？前者——所以它得到大的权重，它的内容会更多地进入你最后的读书笔记。
边界说明 检索通常只取最相关的几本（“硬”选择）；注意力对**所有**条目都分配一个大于 0 的权重再加权混合（“软”选择），因此整个过程可以求导、可以训练。

公式里还有两处没有解释清楚：为什么要除以 $\sqrt{d_k}$，为什么最后要乘 V。笔记在这两处都打了问号。

6.6 为什么要除以 $\sqrt{d_k}$

课件 p.29 的解释是：维度很大时点积可能很大，Softmax 会变得极端，训练困难。这里把“为什么会很大”和“为什么偏偏除以 $\sqrt{d_k}$ 而不是别的数”补上，依据是论文 3.2.1 节的脚注。

结论 · 点积的方差随维数线性增长 原 · 论文脚注 4

设 $\boldsymbol{q}$、$\boldsymbol{k}$ 的各个分量 $q_1,\dots,q_{d_k},k_1,\dots,k_{d_k}$ 是相互独立的随机变量，均值为 0，方差为 1。则

$$\mathbb{E}[\boldsymbol{q}\cdot\boldsymbol{k}] = 0, \qquad \operatorname{Var}(\boldsymbol{q}\cdot\boldsymbol{k}) = d_k.$$

因而 $\dfrac{\boldsymbol{q}\cdot\boldsymbol{k}}{\sqrt{d_k}}$ 的方差恰好为 1，与维数无关。

证明　**已知条件**　各分量相互独立；$\mathbb{E}[q_i]=\mathbb{E}[k_i]=0$；$\mathrm{Var}(q_i)=\mathrm{Var}(k_i)=1$，于是 $\mathbb{E}[q_i^2]=\mathrm{Var}(q_i)+(\mathbb{E}[q_i])^2=1$，同理 $\mathbb{E}[k_i^2]=1$。

思路　点积是 d_k 项 q_ik_i 之和。先算单独一项的均值和方差，再利用“独立随机变量之和的方差等于方差之和”。

第一步，单项的均值。因为 q_i 与 k_i 独立，$\mathbb{E}[q_ik_i]=\mathbb{E}[q_i]\,\mathbb{E}[k_i]=0\times0=0$。

第二步，单项的方差。$\mathrm{Var}(q_ik_i)=\mathbb{E}[(q_ik_i)^2]-(\mathbb{E}[q_ik_i])^2=\mathbb{E}[q_i^2k_i^2]-0=\mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2]=1\times1=1$，其中倒数第二个等号再次用到独立性。

第三步，求和。不同的 i 对应的 q_ik_i 相互独立，所以

$$\mathbb{E}[\boldsymbol{q}\cdot\boldsymbol{k}]=\sum_{i=1}^{d_k}\mathbb{E}[q_ik_i]=0,\qquad \mathrm{Var}(\boldsymbol{q}\cdot\boldsymbol{k})=\sum_{i=1}^{d_k}\mathrm{Var}(q_ik_i)=d_k.$$

第四步，缩放。由 $\mathrm{Var}(cX)=c^2\,\mathrm{Var}(X)$，取 $c=1/\sqrt{d_k}$，得 $\mathrm{Var}\big(\boldsymbol{q}\cdot\boldsymbol{k}/\sqrt{d_k}\big)=d_k/d_k=1$。 ■

一句话解释　点积是许多项相加，项数越多，和的波动范围越大；波动的典型大小（标准差）是 $\sqrt{d_k}$，所以除以它正好抵消。

这就回答了“为什么是 $\sqrt{d_k}$”：标准差是方差的平方根。若除以 d_k，方差会变成 $1/d_k$，分数被压得太小，Softmax 输出接近均匀，同样不好。笔记里“向量维数太大 → 点积结果太大”的说法可以更准确地表述为：**维数越大，点积的波动范围越大**——它的平均值仍是 0，但典型大小是 $\sqrt{d_k}$。

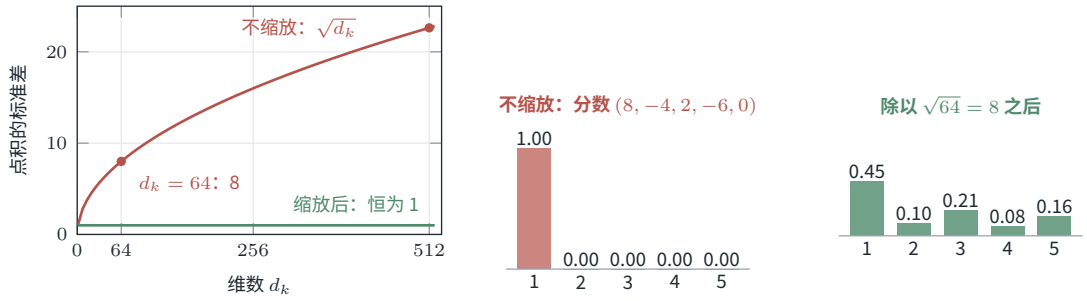


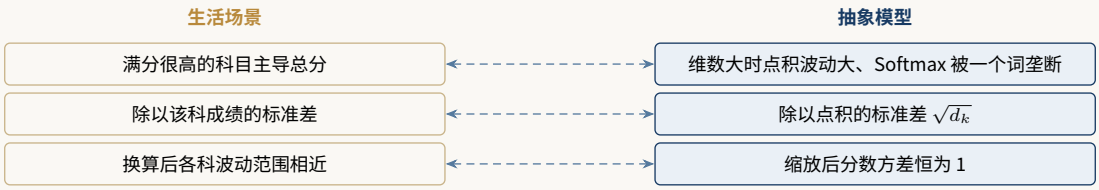
图 25　左：点积的标准差随维数增长（红），缩放后保持为 1（绿）。右：同一组 $d_k=64$ 时的典型分数，缩放前后的 Softmax 结果

右边两幅柱状图是同一组分数。不缩放时分数的典型大小是 8，Softmax 几乎把全部权重给了一个词（第一根柱子精确值约为 0.997）——课件称之为“一票否决”。除以 8 以后，分数回到 1 左右的量级，权重在几个词之间合理分配。

分布太尖为什么不利于训练？ 教学补全　对 Softmax 求导可得 $\dfrac{\partial p_i}{\partial z_i}=p_i(1-p_i)$。当某个 p_i 接近 1（其余接近 0）时，这个导数对所有 i 都接近 0——正是 4.2 节那条 S 形曲线两端的饱和区。梯度几乎为 0，参数就几乎不更新。论文的原话是点积过大会把 Softmax 推到“梯度极小的区域”。

生活中的它 · 把不同满分的成绩换算到同一把尺子上

生活场景　评奖学金要综合多门课。如果一门课满分 150、另一门满分 10，直接相加的话，满分高的那门几乎独自决定排名。常见的做法是先把每门成绩换算成“标准分”（减去平均分，再除以标准差）。



一次小估算 $d_k = 64$ 时点积的标准差是 $\sqrt{64} = 8$；论文基础模型正是 $d_k = 64$，所以每个点积都除以 8。若 $d_k = 256$，则除以 16。

边界说明 标准分还要先减去平均分；点积的平均值本来就是 0，所以只需要“除”这一步。另外定理的前提（各分量独立、方差为 1）只是近似成立，缩放是让量级大致合适，不是精确标准化。

易错 · 关于缩放的三个高频错误 ① **除以 d_k 而不是 $\sqrt{d_k}$**。方差是 d_k，标准差才是 $\sqrt{d_k}$。② **以为缩放会改变“谁的权重最大”**。所有分数除以同一个正数，大小顺序不变（Softmax 保序），改变的只是分布的尖锐程度。③ **把 d_k 当成句子长度** n。d_k 是每个 query/key 向量的维数，与句子有多少个词无关。

6.7 为什么最后要乘 V

算到 Softmax 为止，得到的只是一组权重：模型知道了“吃”最该关注“小龙虾”。但**权重本身不是信息**，它只说明“看谁、看多少”。模型真正想要的是：从被关注的词那里**取回**什么内容。这就是 V 的作用。 原 · 课件 p.31

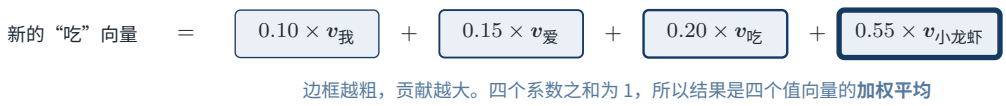


图 26　乘 V 就是按注意力权重对值向量做加权平均（权重取自课件 p.31）

结论 · 注意力输出是值向量的加权平均 教学补全

对每个位置 i，输出 $\boldsymbol{o}_i = \sum_j \alpha_{ij} \boldsymbol{v}_j$，其中 $\alpha_{ij} > 0$ 且 $\sum_j \alpha_{ij} = 1$。因此 $\boldsymbol{o}_i$ 落在各个 $\boldsymbol{v}_j$ “围成的范围之内”（数学上称为凸组合），不会跑到它们之外。

这条结论直接来自 Softmax 的非负性与归一性（4.2 节的性质 1、2）：系数都是正的，加起来是 1。用二维的值向量可以把它画出来。

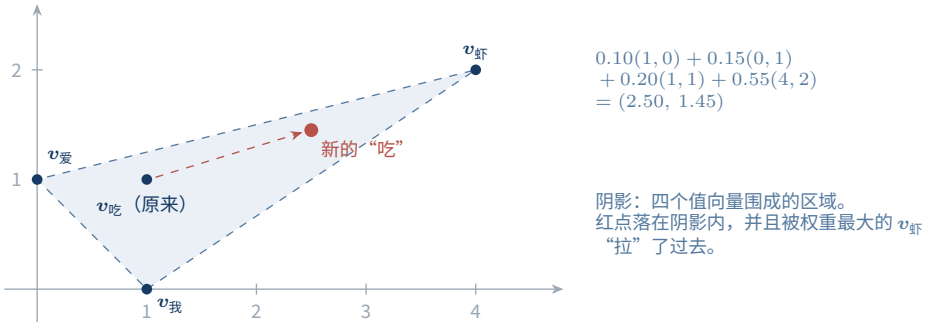


图 27　二维示意：新向量是四个值向量的加权平均，位置偏向权重最大的那个

图中蓝点是四个词的值向量，红点是算出来的新“吃”向量。它从原来的 $(1, 1)$ 被拉向 $(4, 2)$：**“吃”吸收了“小龙虾”的信息**。这就是笔记里那句“融合上下文的新向量”的几何含义。

为什么要把 K 和 V 分开，而不是用同一个向量？　“适合用来被匹配的特征”和“值得被传递的内容”未必相同。比如判断“它”指谁时，匹配靠的是“是不是名词、是不是可以被吃的东西”这类线索（Key）；匹配上之后要取回的，是这个名词的完整含义（Value）。把两者交给两个不同的线性变换 W^K、W^V 去学，模型更灵活。

6.8 自注意力：一句话里的每个词都重新认识自己

前面的例子只算了“吃”这一个词。实际上公式是对所有词**同时**做的：Q 的每一行都是一个提问者，每个词都拿着自己的 query 去和所有词的 key 比对，各自取回一份加权平均。

定义 · 自注意力（Self-Attention）

Q、K、V 都来自**同一个序列**的注意力。句子里的每个词都作为 query 关注这句话里的所有词（包括它自己），输出一组新的向量，每个新向量都融合了整句话的信息。 原 · 课件 p.32–33

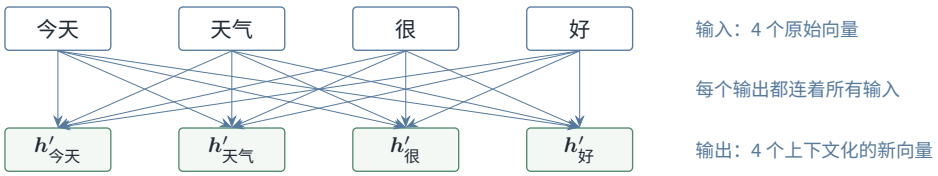


图 28　自注意力的输入与输出（重绘自课件 p.32）：n 个向量进，n 个向量出

自注意力最重要的效果是：**同一个词，在不同的上下文里，会得到不同的新向量**。课件的例子是“苹果”：



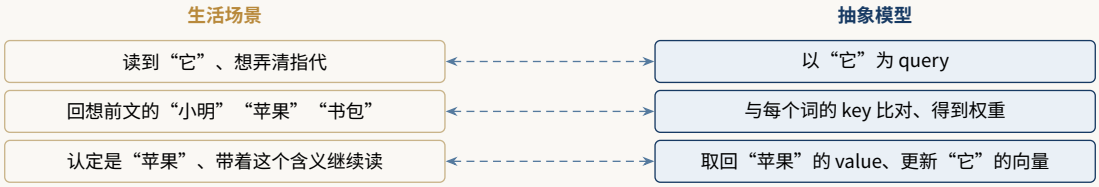
图 29　同一个词在两句话里得到不同的上下文表示（重绘自课件 p.33）

这是 Embedding 做不到的：查表得到的向量对每个 token 是固定的，“苹果”永远是同一行。自注意力之后，向量才带上了“这句话里的含义”。

笔记的问题：prompt 有什么作用？笔记写的是“根据 prompt 重新计算注意力”。结合上面的图就容易理解了：**prompt 也是输入序列的一部分**。用户写的每个词都会进入注意力计算，成为后面所有词可以关注的对象；提示不同，后面每个位置取回的信息就不同，输出自然不同。模型的参数一个也没有变，变的只是“被关注的上下文”——这正是第 3 章说“提示不改参数”的原因。

生活中的它 · 读到“它”时，目光回扫前文

生活场景 读到“小明把苹果放进书包，因为它很甜”中的“它”，你会不自觉地回想前面出现过的名词，判断哪个能和“甜”搭配，然后把“它”理解成苹果。



一次小判断 把“甜”换成“重”，你的判断会更犹豫——对应权重在“苹果”和“书包”之间更平

均。

边界说明 人回想时一次只想一两个候选；自注意力对句中每个词、同时对所有词算权重，而且“它很甜”三个字本身也在同时做同样的事。

相近概念辨析：三种注意力只差在 Q、K、V 的来源

名称	Q 来自	K、V 来自	出现位置
自注意力	本序列	本序列（全部位置）	编码器
掩码自注意力	本序列	本序列（只到当前位置为止）	解码器（7.6 节）
交叉注意力	解码器	编码器的输出	解码器（7.7 节）

三者用的是同一个公式 $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})V$。

6.9 多头注意力：多组不同的观察角度

一次注意力只产生一张权重表。可是词与词之间的关系不止一种：语义上谁和谁相关，语法上谁是谁的主语，指代上“它”指谁……一张表很难同时兼顾。原 · 课件 p.34

办法是**同时做几次注意力，每次用不同的线性变换**，再把结果合起来。

核心公式 · 多头注意力（Multi-Head Attention） 原 · 论文 3.2.2 节

$$\begin{aligned}\mathrm{MultiHead}(Q,K,V) &= \mathrm{Concat}(\mathrm{head}_1,\dots,\mathrm{head}_h)\,W^O,\\ \mathrm{head}_i &= \mathrm{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V), \qquad i=1,\dots,h.\end{aligned}$$

每个符号的含义。h：头的个数。W_i^Q 与 W_i^K（形状 $d_{\text{model}}\times d_k$）、$W_i^V$（形状 $d_{\text{model}}\times d_v$）：第 i 个头自己的三个投影矩阵——“多个头不是简单重复计算，而是每个头有自己的一组 W_Q、W_K、W_V”（课件 p.35）。Concat：把 h 个头的输出左右拼接，得到 $h\,d_v$ 维。W^O（$h\,d_v\times d_{\text{model}}$）：最后的线性层，把拼接结果变回 d_{model} 维；上标 O 是 Output 的首字母。

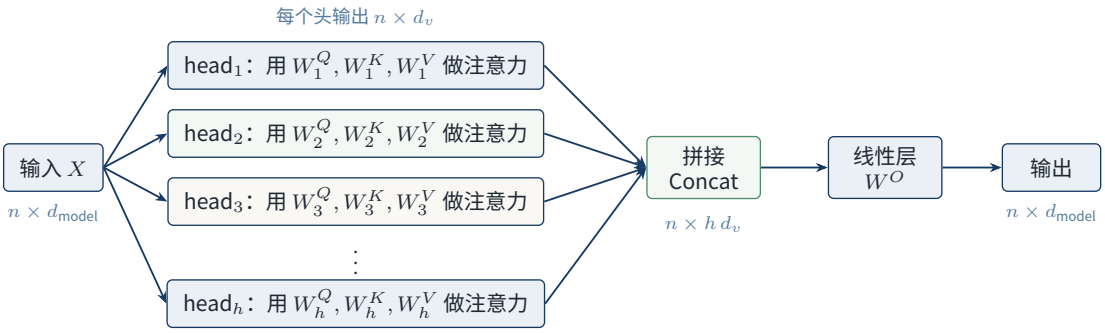


图 30　多头注意力的流程（重绘自课件 p.36 与论文图 2 右）：先分头理解，再合并信息

例 · 论文基础模型的多头配置

已知　$d_{\text{model}}=512$，$h=8$，论文取 $d_k=d_v=d_{\text{model}}/h$。

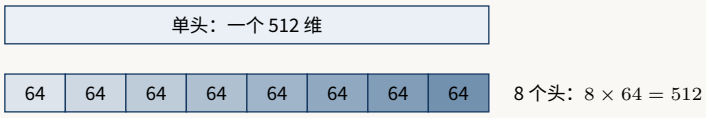
每个头的维数　$d_k=d_v=512/8=64$。

拼接后的维数　$h\times d_v=8\times 64=512$，恰好等于 d_{model}。

一个投影矩阵有多少参数　W_i^Q 是 512×64 的矩阵，共 $512\times 64=32\,768$ 个数。

本例想说明　论文特意让每个头的维数缩小到 $1/h$，所以 8 个头的总计算量与“一个 512 维的单

头”相近。多头不是靠加算力，而是把同样的容量**分给不同的视角**。

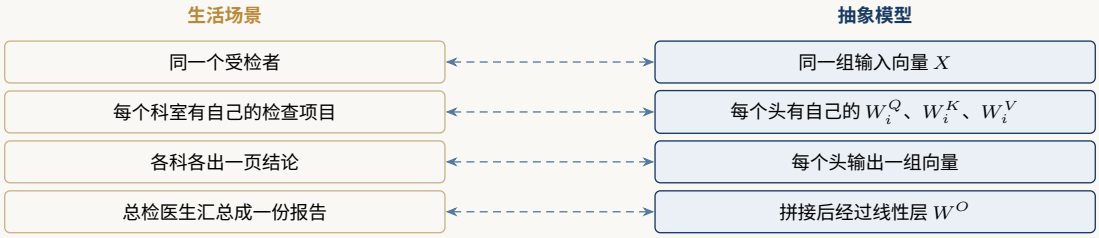


论文表 3 的实验支持这个设计：在总计算量不变的前提下，只用 1 个头比用 8 个头的翻译质量低 0.9 BLEU；头数过多（32 个，每头只有 16 维）效果也会下降。

易错 · “每个头负责一种语法关系”只是比喻　课件 p.35 用“Head 1 看主语和谓语、Head 2 看动词和宾语……”来说明多头的作用。这是类比：没有人给各个头分配任务，每个头学到什么完全由训练决定。论文附录的可视化显示，确实有些头的行为看起来与句法结构或指代关系有关，但并非每个头都能这样清楚地解释。

生活中的它 · 体检时走过的各个科室

生活场景　入学体检时，同一个人依次经过眼科、内科、外科、检验科，每个科室只看自己关心的指标，最后由总检医生把各科结论汇总成一份体检报告。



一次小估算　若体检中心有 8 个科室，每个科室出 64 行结论，汇总前共 $8 \times 64 = 512$ 行——与论文基础模型拼接后的维数算法相同。

边界说明　科室的分工是人预先规定的；注意力头的“分工”是训练中自发形成的，事先并不知道哪个头会学到什么。

6.10 位置编码：告诉模型“谁在前、谁在后”

到目前为止的注意力有一个严重的缺陷，课件 p.37 指出：Transformer 放弃了 RNN 的时间结构，并行处理所有词，因此**不天然知道词的顺序**。这不是一句泛泛的提醒，而是可以证明的性质。

结论 · 自注意力不区分顺序（置换等变性） 教学补全

把输入的 n 个向量按任意方式重新排列（即 X 的各行换位），自注意力的输出只是按**同样的方式**重新排列，每个词得到的新向量本身不变。

证明　**符号**　用置换矩阵 P 表示“重排各行”：PX 就是把 X 的行换了顺序。置换矩阵满足 $P^{\mathsf{T}}P = I$。记 $A = \mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})$。

第一步，Q、K、V 跟着重排。输入换成 PX 后，$Q' = PXW^Q = PQ$，同理 $K' = PK$，$V' = PV$。

第二步，分数矩阵的行和列同时重排。$Q'K'^{\mathsf{T}} = PQ(PK)^{\mathsf{T}} = P\,(QK^{\mathsf{T}})\,P^{\mathsf{T}}$。左乘 P 重排行，右乘 P^{T} 以同样方式重排列。

第三步，Softmax 与重排可交换。逐行 Softmax 只关心“同一行里有哪些数”：行换了位置，每一行内部的计算不受影响；列换了位置，只是同一行里的数换了顺序，结果也跟着换顺序。所以

$A' = PAP^{\mathsf{T}}$。

第四步，乘 V'。$A'V' = PAP^{\mathsf{T}}PV = PA\,(P^{\mathsf{T}}P)\,V = P\,(AV)$。

结论　新输出 $= P\times$（原输出）：只是行的顺序跟着变了，每一行的内容与原来完全相同。 ■

一句话解释　注意力只看“内容之间的匹配”，从不问“你排在第几个”。

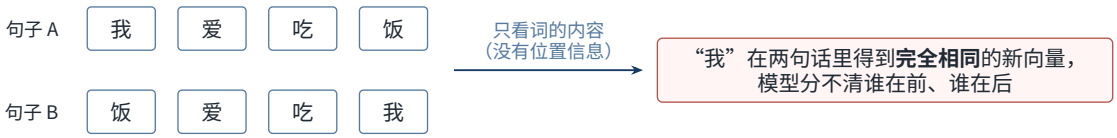


图 31　没有位置信息时，两句词序不同的话在模型眼里是同一个“词的集合”（重绘自课件 p.37）

解决方法是在送入第一层之前，给每个词向量**加上**一个只与位置有关的向量：

$$\boxed{\text{输入向量} = \text{Token Embedding} + \text{Position Embedding}}$$

前者告诉模型“这个词是谁”，后者告诉模型“这个词站在哪里”。加上之后，“我”在第 1 位和在第 4 位的输入向量不再相同，上面的定理就不再适用，顺序信息被带进了后续所有计算。

位置向量怎么取？论文用的是不同频率的正弦和余弦。

核心公式 · 正弦位置编码 原 · 论文 3.5 节

$$PE_{(pos,\,2i)} = \sin\!\left(\frac{pos}{10000^{\,2i/d_{\text{model}}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\!\left(\frac{pos}{10000^{\,2i/d_{\text{model}}}}\right).$$

每个符号的含义。pos：词在句子中的位置，$0, 1, 2, \ldots$。i：维度的“对号”，$i = 0, 1, \ldots, d_{\text{model}}/2 - 1$；第 $2i$ 维（偶数维）用正弦，第 $2i+1$ 维（奇数维）用余弦，两者共用同一个频率。$10000^{2i/d_{\text{model}}}$：随 i 增大而增大的分母，所以**越靠后的维度，波动越慢**。位置编码的维数与词向量相同（都是 d_{model}），这样才能相加。

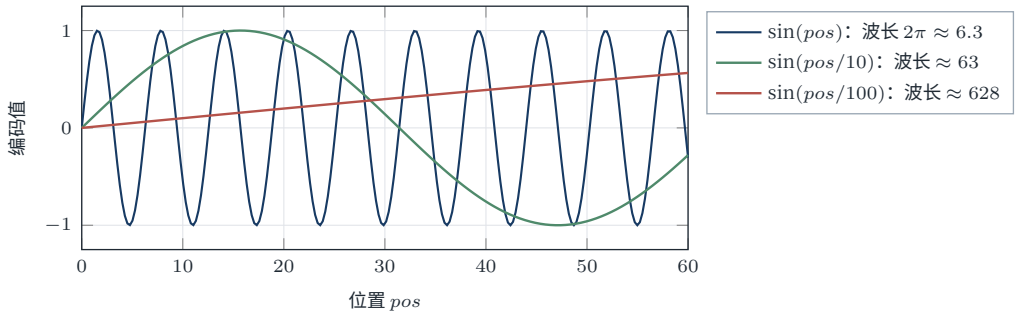


图 32　三个不同维度上的正弦曲线。横轴是位置，每条曲线是位置编码的一个分量

横轴是位置，三条曲线是位置编码中三个不同的偶数维。蓝线变化最快，相邻两个位置的值差别明显，适合区分“紧挨着的词”；红线变化最慢，在这 60 个位置里几乎是直线上升，适合区分“大致在句子的前部还是后部”。在某个位置竖着画一条线，它与各条曲线的交点合起来，就是这个位置的编码。论文说明这些波长从 2π 到 $10000 \cdot 2\pi$ 构成等比数列。

例 · 算出前三个位置的编码（$d_{\text{model}} = 4$）

$d_{\text{model}} = 4$ 时 i 只能取 0 和 1。$i = 0$：分母 $10000^0 = 1$；$i = 1$：分母 $10000^{2/4} = 100$。所以

$$PE_{pos} = \big(\sin(pos),\ \cos(pos),\ \sin(pos/100),\ \cos(pos/100)\big) \qquad \text{（角度以弧度计）}.$$

pos	第 0 维 $\sin(pos)$	第 1 维 $\cos(pos)$	第 2 维 $\sin(pos/100)$	第 3 维 $\cos(pos/100)$
0	0.000	1.000	0.000	1.000
1	0.841	0.540	0.010	1.000
2	0.909	−0.416	0.020	1.000

本例想说明　每个位置得到一个各不相同的向量；前两维变化快，后两维变化慢，与图 32 一致。

为什么用正弦和余弦?　论文给出的理由是：对任何固定的间隔 k，PE_{pos+k} 可以表示成 PE_{pos} 的**线性函数**，这可能让模型更容易学会按相对位置去关注。验证只需要两角和公式。记 $\omega = 1/10000^{2i/d_{\text{model}}}$，则

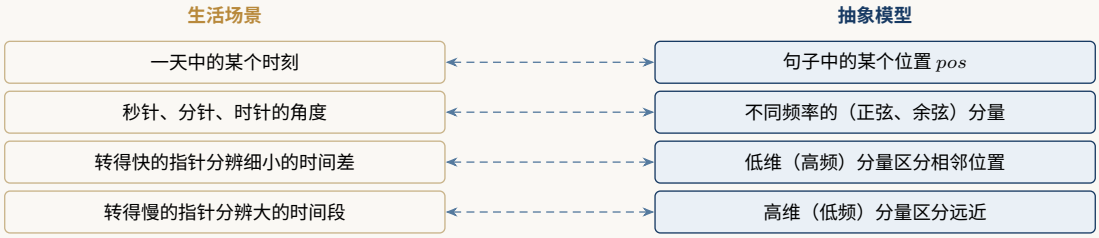
$$\sin\big(\omega(pos+k)\big) = \sin(\omega\, pos)\cos(\omega k) + \cos(\omega\, pos)\sin(\omega k),$$
$$\cos\big(\omega(pos+k)\big) = \cos(\omega\, pos)\cos(\omega k) - \sin(\omega\, pos)\sin(\omega k).$$

右边是 $\sin(\omega\, pos)$ 与 $\cos(\omega\, pos)$ 的线性组合，系数 $\cos(\omega k)$、$\sin(\omega k)$ 只依赖间隔 k，与 pos 无关。换句话说：**“往后挪 k 个位置”对应于对每一对（正弦，余弦）分量做一次固定角度的旋转。**

教学补全　另一种做法是把位置向量也当作可学习的参数（像 Embedding 一样查表），课件的写法“Position Embedding”更接近这种做法。论文表 3 的 (E) 行比较了两者，结果几乎相同；论文选择正弦版本，是因为它有可能外推到比训练时更长的句子。后来的 BERT、GPT 采用的是可学习的位置向量。

生活中的它 · 钟面上的三根指针

生活场景 钟表用秒针、分针、时针三根转速不同的指针表示时间。秒针转得快，分得清相邻两秒；时针转得慢，分得清上午和下午。三根指针的角度合在一起，唯一确定了一个时刻。



一次小估算 秒针 60 秒转一圈，分针 3600 秒转一圈，转速相差 60 倍。位置编码里 $d_{\text{model}} = 4$ 时两对分量的波长是 $2\pi \approx 6.3$ 和 $200\pi \approx 628$，相差 100 倍。
边界说明 钟只有三根指针，转速比是固定的 60 和 12；位置编码有 $d_{\text{model}}/2$ 对“指针”，转速按等比数列排列，并且是把角度的正弦、余弦值加到词向量上，而不是单独显示。

易错 · 关于位置编码的三个误解　①**“位置编码是拼接在词向量后面的”**。是相加，维数不变。②**“位置编码就是把 1、2、3……直接加上去”**。直接用整数会让数值随句长无限增大，盖过词向量本身；正弦余弦的值始终在 -1 到 1 之间。③**“每一层都要重新加位置编码”**。论文只在编码器和解码器的最底部加一次。

到这里，我们已经解决了什么？下一步呢?　文字变成了向量（Embedding）；向量之间的相关性用点积衡量；点积经缩放、Softmax 变成权重；权重对 V 加权平均得到融合上下文的新向量；多头提供

多个视角；位置编码补上顺序。零件都齐了。下一章把它们装配成完整的 Transformer，并补上三个还没出现的零件：残差连接与层归一化、前馈网络、解码器的掩码。

自检：（1）Q、K、V 各自的直观含义是什么？（2）QK^{T} 的第 i 行第 j 列是什么？（3）为什么除以 $\sqrt{d_k}$ 而不是 d_k？（4）注意力权重矩阵的行和是多少？列和呢？（5）$d_{\text{model}}=512$、$h=8$ 时每个头的 d_k 是多少？（6）不加位置编码时，打乱词序会怎样？（7）位置编码与词向量是相加还是拼接？

第七章　Transformer：把零件装配成完整架构

本章回答四个问题：Transformer 最初是为哪个任务设计的？编码器的一层里有什么？解码器比编码器多了什么？为什么它能叠很多层而训练不垮？　**知识路线：**机器翻译 → 全貌图 → Add & Norm → 前馈网络 → 编码器 → 解码器（掩码、交叉注意力） → 输出层 → 为什么能堆深。

7.1　Transformer 一开始要解决什么问题

上一章的注意力把“一组向量”变成“另一组融合了上下文的向量”，但它本身还不是一个能完成任务的模型。论文要解决的具体任务是**机器翻译**：输入一种语言的句子，输出另一种语言的句子。原 · 课件 p.47

翻译有一个特点：输入和输出是**两个长度不同的序列**。于是模型自然分成两半：

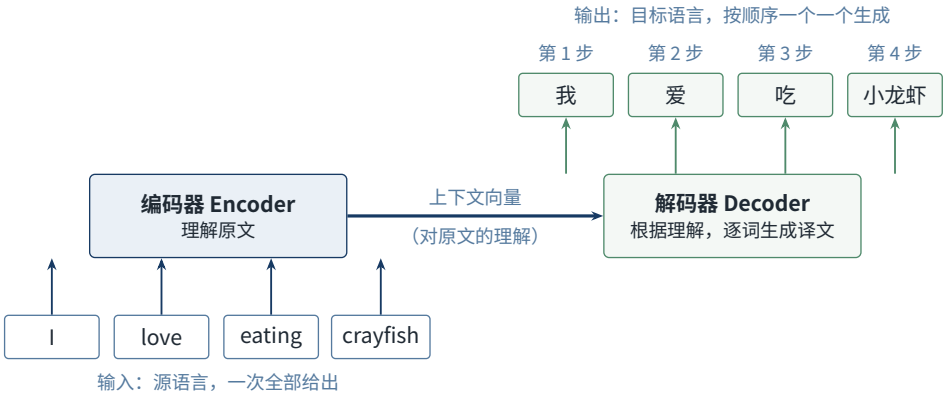


图 33　机器翻译的编码器—解码器分工（重绘自课件 p.47）

课件的比喻是 类比：Encoder 是“读原文的人”，把原文读懂但不负责开口；Decoder 是“根据理解写答案的人”。 教学补全 论文中这个结构的实验是 WMT 2014 的英语→德语和英语→法语翻译（不是中英，见 1.5 节），课件的例句只是示意。

7.2　全貌：一张图看完整架构

图 34 是论文图 1 的重绘，也是本章的地图。**读法：**数据自下而上流动。左边是编码器：输入句子先变成“词向量＋位置编码”，再经过 N 个相同的层；每层有两个子层——多头自注意力、前馈网络，每个子层后面都跟一个“Add & Norm”。右边是解码器：同样先做 Embedding 与位置编码，每层有三个子层，多出来的是中间那个“编码器—解码器注意力”，它的 K、V 来自编码器的最终输出（图中那条横向的粗箭头）。解码器顶端经过 Linear 和 Softmax，输出下一个词的概率。论文取 $N=6$。

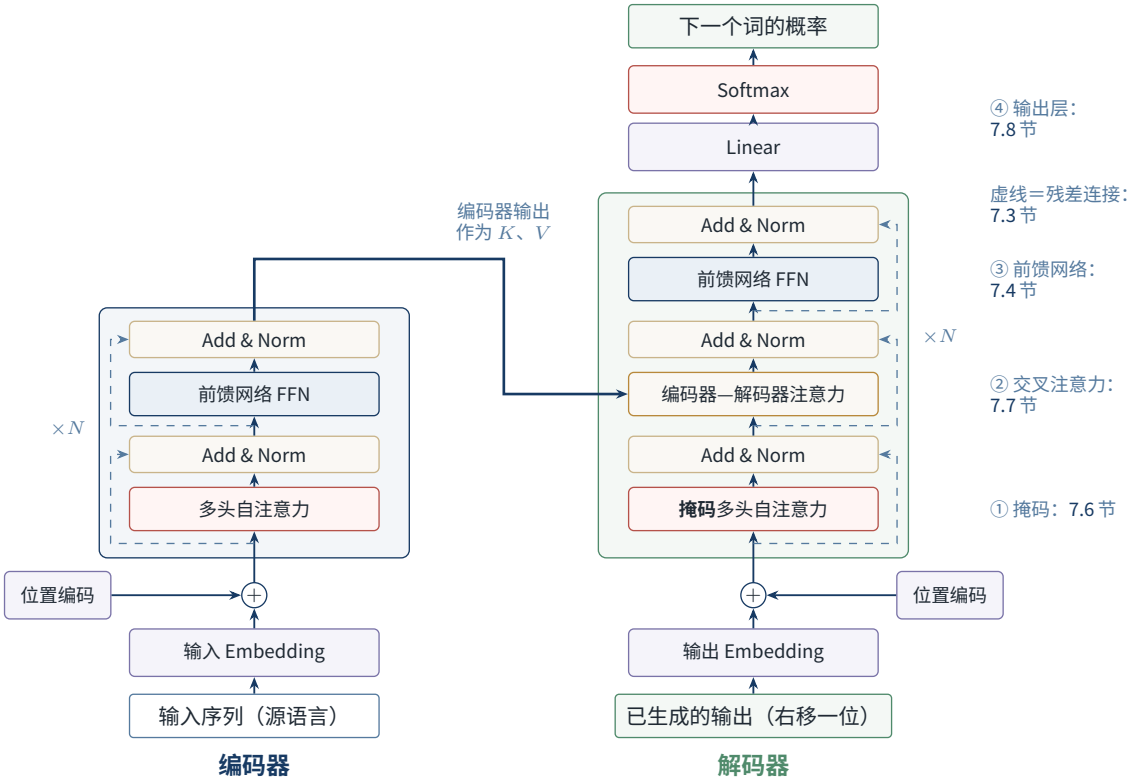


图 34　Transformer 的完整架构（按论文图 1 重绘，对应课件 p.38、p.45）。箭头自下而上表示数据流向

图中三种零件上一章都没有讲：**Add & Norm**、**前馈网络**，以及解码器的**掩码**。下面按这个顺序补齐。

7.3 Add & Norm：残差连接与层归一化

核心公式 · 每个子层的统一写法 原 · 论文 3.1 节

$$\boxed{\text{子层输出} = \mathrm{LayerNorm}\big(x + \mathrm{Sublayer}(x)\big)}$$

其中 Sublayer 是这个子层本身的运算（自注意力或前馈网络），x 是子层的输入。

这个式子包含两件事：“Add”——把子层的输入 x 原样加回到子层的输出上，称为**残差连接**；“Norm”——对相加的结果做**层归一化**。

残差连接：只学“修改量”

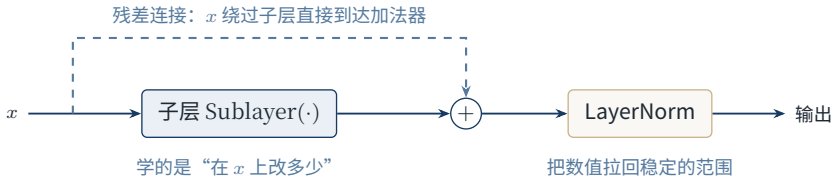


图 35　一个“子层 + Add & Norm”的结构

没有那条虚线时，子层必须**重新生成**完整的输出；有了它，子层只需要学习“在 x 的基础上修改多少”。即使子层什么都没学好（输出接近 0），x 也能原样通过，信息不会丢。这就是课件 p.44 说的“残差连接保留原信息”。

残差连接更重要的作用体现在训练上，它是第 5 章“梯度连乘”问题的解药：

结论 · 残差连接给梯度留了一条直通路 教学补全

设 $y = x + F(x)$，则 $\dfrac{\partial y}{\partial x} = 1 + F'(x)$（向量情形为 $I + \dfrac{\partial F}{\partial x}$）。即使 $F'(x)$ 很小，导数也接近 1 而不是接近 0。

推导与数值对比　对 $y = x + F(x)$ 求导，和的导数等于导数的和：$\dfrac{\partial y}{\partial x} = \dfrac{\partial x}{\partial x} + \dfrac{\partial F(x)}{\partial x} = 1 + F'(x)$。■
把 6 层叠起来看（标量、忽略归一化的简化情形），假设每层 $F' = 0.1$：
没有残差：梯度连乘 $0.1^6 = 10^{-6}$，几乎消失。　　有残差：$(1 + 0.1)^6 \approx 1.77$，保持在 1 附近。
以下为直观解释，不替代严格证明：实际每层后面还有 LayerNorm，导数不会恰好是 $1 + F'$，但“存在一条不经过子层的恒等通路”这一点不变。

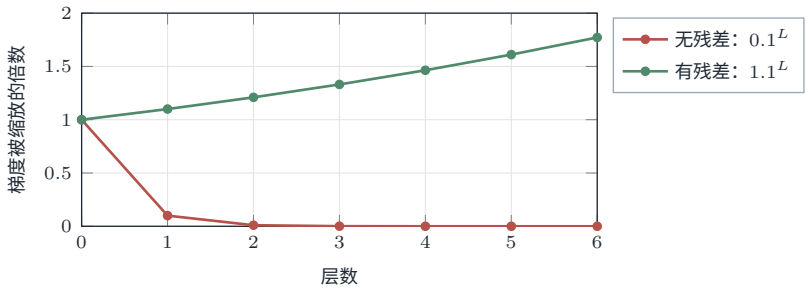


图 36　每层子层导数为 0.1 时，叠 L 层后梯度的大小。横轴是层数

红线一层就掉到 0.1，三层以后贴在横轴上；绿线始终在 1 附近。残差连接并不是 Transformer 的发明（论文引用的是 2015 年图像识别领域的残差网络），但没有它，6 层乃至几十上百层的堆叠就无从谈起。

层归一化：把每个向量拉回同一把尺子

核心公式 · 层归一化（Layer Normalization） 教学补全

对一个 d 维向量 $x = (x_1, \ldots, x_d)$：

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}, \qquad \mathrm{LayerNorm}(x)_i = \gamma_i \cdot \frac{x_i - \mu}{\sigma} + \beta_i.$$

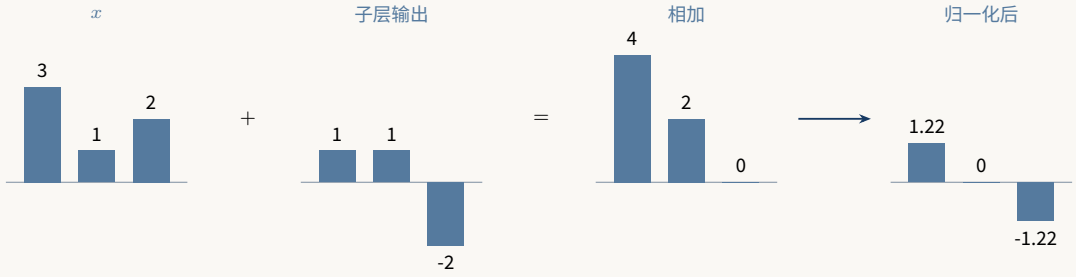
每个符号的含义。μ、σ：这个向量**自己的**各分量的均值与标准差。$(x_i - \mu)/\sigma$：标准化，结果均值为 0、标准差为 1。γ_i、β_i：可学习的缩放与平移参数，初始值通常为 1 和 0。实际实现中分母还会加一个很小的常数以防除以 0。**使用条件**：归一化是对**每个词的向量单独**做的，不同词之间、不同句子之间互不影响。

论文只写了“followed by layer normalization”并给出引用，上面的公式是它所引用的方法的标准形式。

例 · 算一次 Add & Norm

已知　子层输入 $x = (3, 1, 2)$，子层输出 $\mathrm{Sublayer}(x) = (1, 1, -2)$；取 $\gamma = 1$、$\beta = 0$。
Add　$x + \mathrm{Sublayer}(x) = (4, 2, 0)$。
Norm　均值 $\mu = (4 + 2 + 0)/3 = 2$；标准差 $\sigma = \sqrt{\left((4-2)^2 + (2-2)^2 + (0-2)^2\right)/3} = \sqrt{8/3} \approx 1.633$；

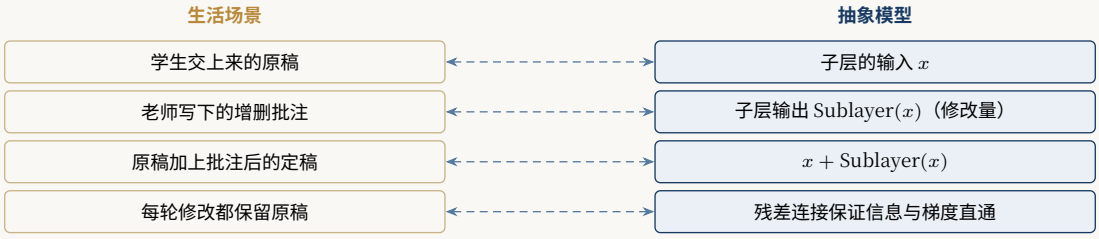
结果 $\left(\frac{4-2}{1.633}, \frac{2-2}{1.633}, \frac{0-2}{1.633}\right) \approx (1.22,\ 0,\ -1.22)$。



检查　归一化后三个数的均值为 0，平方的平均为 $(1.22^2+0+1.22^2)/3 \approx 1$。**本例想说明**　无论相加后的数值有多大，归一化都把它拉回“均值 0、标准差 1”的范围，下一层看到的输入尺度始终稳定。

生活中的它 · 在原稿上批注修改

生活场景 老师改作文，不是把整篇重写一遍，而是在原稿上用红笔批注：这里加一句，那里删两个字。改完的作文＝原稿＋修改。



一次小判断 如果某一轮老师一个字没改，定稿是什么？就是原稿本身——对应 Sublayer$(x)=0$ 时输出等于 x，这一层“什么也没破坏”。
边界说明 这个比喻只对应“Add”。至于“Norm”，它距日常生活较远，最接近的经验是 6.6 节提到的“换算成标准分”：把一组数减去均值、除以标准差，使不同来源的数可以放在一起比较。

易错 · 关于 Add & Norm 的两个误解　① **“Add 是把向量拼接起来”**。是逐元素相加，所以要求子层的输入和输出维数相同——论文为此让所有子层和 Embedding 的输出维数都等于 $d_{\text{model}}=512$。② **“归一化是对一批句子一起做的”**。层归一化只在单个词向量内部求均值和标准差，与句子里的其他词、与其他句子都无关。

7.4 前馈网络：每个词回到座位上独立加工

注意力让词与词交换了信息，但回头看 6.7 节：它的输出只是值向量的加权平均，是一种“混合”，没有对内容本身做更复杂的加工。因此每层在注意力之后还接一个前馈网络（Feed-Forward Network，FFN）。 原 · 课件 p.40

核心公式 · 逐位置前馈网络 原 · 论文式 (2)

$$\boxed{\mathrm{FFN}(x)=\max(0,\ xW_1+b_1)\,W_2+b_2}$$

每个符号的含义。x：一个词的 d_{model} 维向量。W_1（$d_{\text{model}}\times d_{ff}$）、$b_1$：第一个线性层，把向量从 d_{model} 维扩展到 d_{ff} 维。$\max(0,\cdot)$：ReLU 激活函数，逐元素把负数变成 0。W_2（$d_{ff}\times d_{\text{model}}$）、$b_2$：

第二个线性层，把向量压回 d_{model} 维。论文取 $d_{\text{model}}=512$，$d_{ff}=2048$。课件写的“ReLU / GELU”中，论文原文用的是 ReLU，GELU 是后来的模型常用的一种平滑变体。

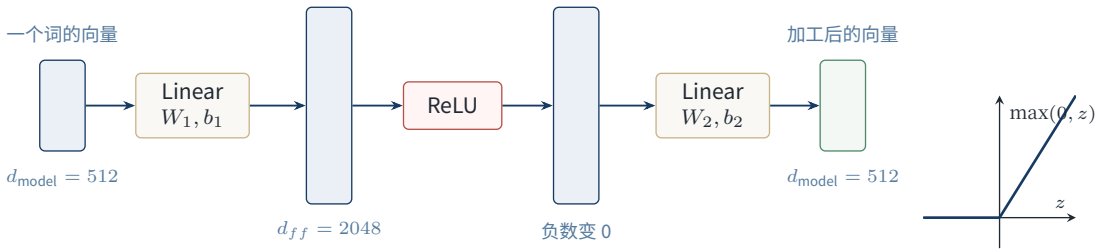


图 37　前馈网络：先扩展、过 ReLU、再压回。右图是 ReLU 函数的图像

“逐位置”是什么意思。句子里每个词的向量**各自独立**地通过**同一个** FFN：词与词之间在这一步没有任何信息交换，但所有词用的是同一组 W_1、b_1、W_2、b_2。课件的类比 类比：Attention 是“开会交流”，FFN 是“每个人回到座位上独立思考”。

为什么中间必须有 ReLU。两个线性层直接相连仍然是一个线性层（xW_1W_2 可以合并成 $x(W_1W_2)$），叠多少都不会增加表达能力。夹在中间的非线性函数才使网络能表示更复杂的变换。

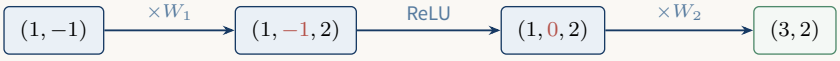
例 · 用一个 2 维 → 3 维 → 2 维的小 FFN 走一遍

已知　$x=(1,-1)$，$b_1=b_2=\mathbf{0}$，$W_1=\begin{pmatrix}1&0&1\\0&1&-1\end{pmatrix}$，$W_2=\begin{pmatrix}1&0\\0&1\\1&1\end{pmatrix}$。

第一步，扩展。$xW_1=\big(1\cdot1+(-1)\cdot0,\ 1\cdot0+(-1)\cdot1,\ 1\cdot1+(-1)(-1)\big)=(1,-1,2)$。

第二步，ReLU。$\max(0,(1,-1,2))=(1,0,2)$，负的那一维被清零。

第三步，压回。$(1,0,2)\,W_2=\big(1\cdot1+0\cdot0+2\cdot1,\ 1\cdot0+0\cdot1+2\cdot1\big)=(3,2)$。



本例想说明　FFN 只有三步：乘矩阵、把负数清零、再乘矩阵。输入输出维数相同，这样才能接残差连接。

FFN 是 Transformer 中参数最多的部分之一：论文配置下，一个 FFN 的权重有 $512\times2048+2048\times512=2\,097\,152$ 个，再加 $2048+512$ 个偏置。

关于生活对应，需要如实说明：**前馈网络这个知识距日常生活较远**，它没有贴切的日常现象可以对应，课件的“回座位独立思考”只是帮助记忆分工的比喻。它最直接的用途在于为每一层提供非线性加工能力；后续课程里，凡是“线性层—激活—线性层”的结构都是同一个想法。

7.5　编码器的一层：把输入读成上下文向量

现在可以把编码器的一层完整地写出来了。设这一层的输入是 n 个向量排成的矩阵 X（$n\times d_{\text{model}}$）：

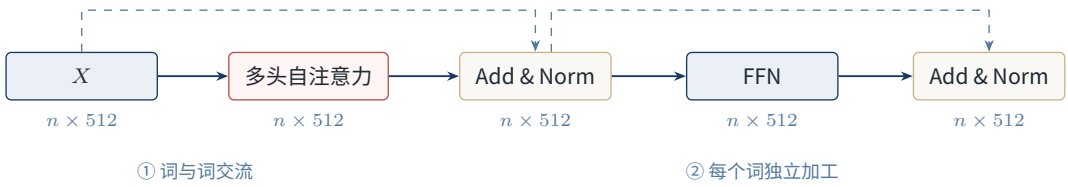


图 38　编码器一层的数据流与形状。每一步的输出形状都与输入相同

每一步的形状都是 $n \times d_{\text{model}}$——进去几个向量，出来还是几个向量，维数不变。这正是层与层可以反复叠加的前提。N 层叠完之后，编码器输出 n 个**上下文向量**：每个词一个，每个都融合了整句话的信息。课件 p.39 的说法是：Encoder 像阅读理解，把原文读懂，但不负责开口说话。

“开口说话”是解码器的事。

7.6 解码器：一边理解，一边生成

解码器要做的是**逐词生成**：已经写出了“我 爱”，现在要决定下一个词。论文称这种方式为**自回归**（auto-regressive）：每一步都把此前已经生成的词作为额外输入，再生成下一个。 原 · 课件 p.41

对照图 34，解码器的每一层有三个子层：

1. **掩码多头自注意力**——在已生成的词之间交换信息，但不能看未来；
2. **编码器—解码器注意力**（交叉注意力）——回头查看编码器对原文的理解；
3. **前馈网络**——与编码器中的完全一样。

每个子层后面同样跟着 Add & Norm。下面讲前两个。

掩码自注意力：不能偷看答案

问题从训练方式来。训练时我们手里有完整的正确译文“我 爱 吃 小龙虾”。为了高效，模型会一次性把整句译文送进解码器，让每个位置同时预测自己的下一个词。可是普通的自注意力让每个词都能看到所有词——第 2 个位置在预测“吃”的时候，就能直接看到后面的“吃”，等于抄答案。

解决方法是在注意力里加一张“可见性表”：位置 i 只允许关注位置 1 到 i，后面的全部遮住。



图 39　因果掩码（重绘自课件 p.42）：绿色可见，红色被遮住

课件 p.42 写“当模型预测‘吃’时，只能看到‘我 爱’”，对应的正是图中第 2 行：**负责预测第 3 个词的，是第 2 个位置**。这一点容易看错行，后面图 40 会再说明“右移一位”的对应关系。

核心公式 · 掩码怎样进入公式 原 · 论文 3.2.3 节

在做 Softmax **之前**，把不允许关注的位置的分数设为 $-\infty$：

$$\text{分数}_{ij} \leftarrow \begin{cases} \boldsymbol{q}_i \cdot \boldsymbol{k}_j / \sqrt{d_k}, & j \le i \quad \text{（允许）} \\ -\infty, & j > i \quad \text{（遮住）} \end{cases}$$

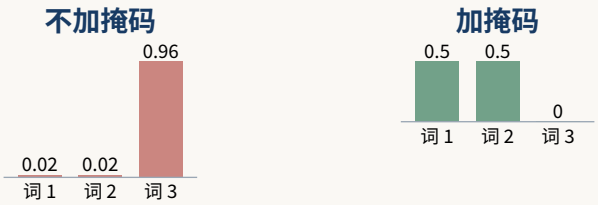
因为 $e^{-\infty} = 0$，这些位置经过 Softmax 后权重恰好为 0，其余位置的权重仍然加起来等于 1。

例 · 带掩码的一行 Softmax

已知　三个词的句子，第 2 个位置对三个词的缩放后分数为 $(1,\ 1,\ 5)$。　**求**　加因果掩码后的注意力权重。

解　第 2 个位置只能看前 2 个词，第 3 个分数改为 $-\infty$：$(1,\ 1,\ -\infty)$。取指数得 $(e,\ e,\ 0)$，归一化后为 $(0.5,\ 0.5,\ 0)$。

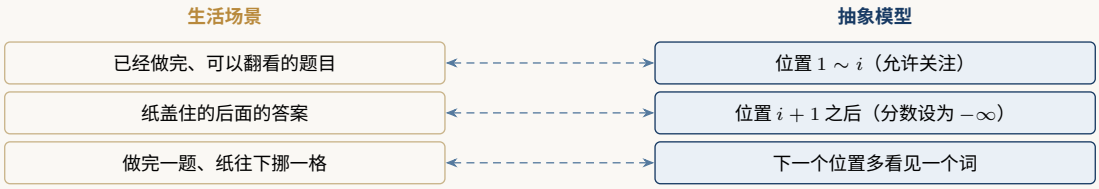
对比　若不加掩码，$\mathrm{softmax}(1,1,5) \approx (0.02,\ 0.02,\ 0.96)$——几乎全部注意力都给了“未来”的第 3 个词。



本例想说明　掩码不是“把权重乘以 0”，而是“在 Softmax 之前把分数改成 $-\infty$”——这样剩下的权重会自动重新归一化。

生活中的它 · 做练习册时用纸盖住后面的答案

生活场景 练习册每道题后面紧跟着答案。自测时用一张纸盖住还没做的部分，做完一题才往下挪一格。 原 · 课件 p.42



一次小估算 一句话有 5 个 token，可见性表共 $5 \times 5 = 25$ 格，其中允许关注的有 $1+2+3+4+5 = 15$ 格，被遮住的有 10 格。

边界说明 人是一题一题往下做；训练时模型是 5 个位置**同时**做，每个位置各自带着不同大小的“遮挡”，所以能并行。

右移一位：解码器的输入与目标

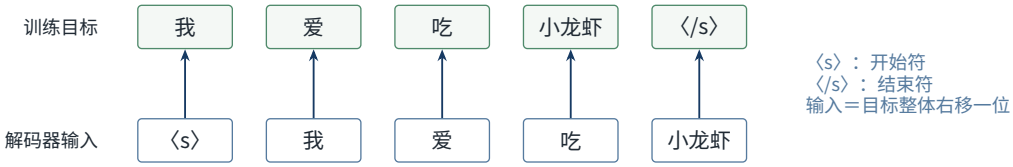


图 40　“右移一位”：每个位置的输入是前一个词，目标是当前词

解码器的输入序列是正确译文**右移一位**并在最前面补一个开始符的结果。这样，每个位置看到的是“到上一个词为止”的内容，要预测的是“当前这个词”。配合因果掩码，就保证了“位置 i 的预测只依赖于位置 i 之前的已知输出”（论文 3.1 节）。

训练与使用时的区别 教学补全　**训练时**：正确译文已知，整句一次送入，所有位置**并行**计算、各自预测下一个词——掩码使这样做不算作弊。

使用时（没有正确答案）：只能**一个一个**生成。先送入〈s〉得到第 1 个词，把它接到输入后面再算第 2 个词……直到生成结束符。

两种情形下，每个位置能看到的信息完全一样，这正是掩码要保证的。

7.7 交叉注意力：生成时回头看原文

掩码自注意力只让译文内部的词互相交流，还没有用到原文。解码器的第二个子层负责这件事。 原 · 课件 p.43

定义 · 编码器—解码器注意力（Cross-Attention，交叉注意力）

公式仍然是 $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})V$，区别只在来源：**$Q$ 来自解码器**上一个子层的输出，**K 和 V 来自编码器**的最终输出。于是解码器的每个位置都可以关注输入序列的**所有**位置。 原 · 论文 3.2.3 节

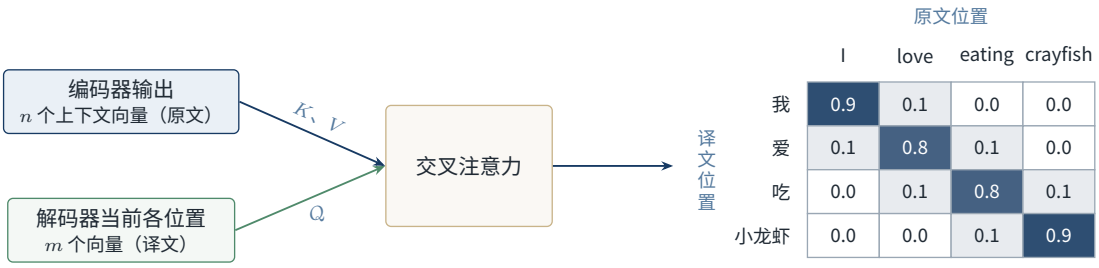


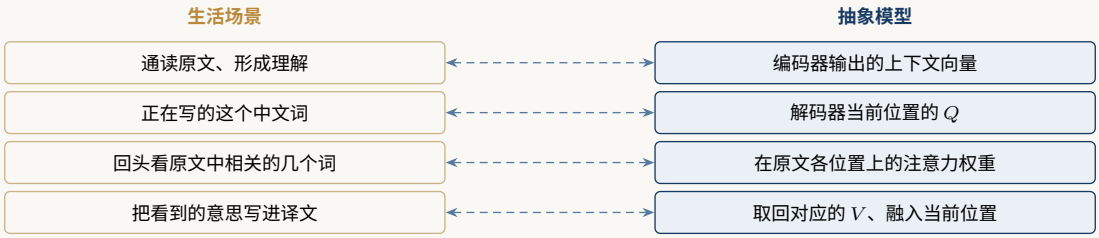
图 41　左：Q、K、V 的来源。右：交叉注意力权重的形状是“译文长度 × 原文长度”（数值为示意）

右边的权重表有 m 行（译文的每个位置一行）、n 列（原文的每个位置一列），每行之和为 1。示意的数值画成了接近对角线的样子：生成“吃”时主要关注原文的“eating”。真实的权重是模型学出来的，不一定这样整齐。

两点容易混淆。（1）交叉注意力**不需要**因果掩码：原文是完整给出的，解码器的任何位置都可以看原文的任何位置。需要掩码的只是解码器**自己内部**的自注意力。（2）编码器只运行一次；它的输出被解码器的**每一层**重复使用。

生活中的它 · 笔译时，写一句、回看一眼原文

生活场景 做英译汉的翻译作业：先把英文原文通读一遍；动笔写中文时，每写一个词都会回头看一眼原文里对应的地方。



一次小判断 写“小龙虾”时，目光该落在原文的哪个词上？crayfish。对应权重表第 4 行在第 4 列最大。

边界说明 人回看原文是有意识的、一次看一处；交叉注意力对原文所有位置同时给出权重，并且是多头、多层重复进行的。

7.8 输出层：从向量回到词

解码器最后一层输出的仍是 d_{model} 维向量，而我们需要的是“词表中每个词的概率”。最后两步完成这个转换：

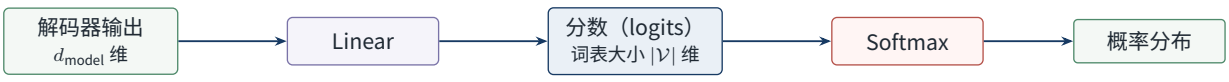


图 42　输出层：线性层把向量映射到词表大小，Softmax 变成概率

线性层把 d_{model} 维向量变成 $|\mathcal{V}|$ 维——词表里有多少个 token，就输出多少个分数；Softmax 把分数变成概率（4.2 节）。训练时用交叉熵把这组概率与正确的下一个词比较（4.3 节）。到这里，第 4 章准备的两件工具各就各位。

7.9 为什么 Transformer 能堆很深

笔记里这个问题只写了题目。把前面几节合起来，答案有两层。原 · 课件 p.44

第一层：每个零件各管一件事，叠起来才有意义。

零件	作用	讲义位置
Attention（全局交互）	每个位置都能看见所有位置，建模长距离依赖	6.5 节
Multi-Head（多角度）	从不同子空间、不同关系角度同时理解信息	6.9 节
FFN（非线性加工）	在每个位置上做非线性变换，提升表达能力	7.4 节
Add & Norm（稳定训练）	残差保留原信息并让梯度直通，归一化稳定数值范围	7.3 节
位置编码（顺序信息）	让模型感知词语的先后关系	6.10 节

第二层：能“堆”的关键是残差与归一化。深层网络的老问题是梯度在层与层之间连乘后消失（第 5 章对时间步、7.3 节对层数分别算过）。残差连接提供直通路，层归一化让每层的输入保持在稳定的尺度上，二者合起来使得叠 6 层、几十层乃至更多层时仍然可以训练。

叠深之后得到什么？课件给了一个直观的说法：浅层学到局部词语之间的直接关联（“吃 ↔ 小龙虾”），中层建立更丰富的关系链，深层抽象出句子级的语义、语气和意图。



图 43　层数越深，表示越抽象（内容取自课件 p.44）

这幅图是类比：真实模型的各层并没有这样清楚的分工，“越深越抽象”是总体趋势的概括，不是逐层可验证的规定。

7.10 论文中的关键数字

把论文的主要配置和结果集中列在这里，方便做作业（通读论文）时对照。

项目	基础模型（base）	说明
层数 N（编码器、解码器各）	6	大模型（big）也是 6
向量维数 d_{model}	512	big：1024
前馈网络中间维数 d_{ff}	2048	big：4096
头数 h；每头维数 $d_k = d_v$	8；64	big：16 个头
参数量	约 6500 万	big：约 2.13 亿
英→德 BLEU（WMT 2014）	27.3	big：28.4，超过此前最好结果 2 BLEU 以上
英→法 BLEU（WMT 2014）	38.1	big：41.8
训练时间（8 块 P100 GPU）	12 小时	big：3.5 天

摘自论文表 2、表 3 与第 5.2、6.1 节。BLEU 是衡量译文与参考译文接近程度的指标，越高越好

教学补全 **拓展联想：以下内容只用于建立知识联系，当前阶段不要求掌握。**论文训练时还用了三项技巧：Adam 优化器配合“先线性升高、再按步数平方根倒数下降”的学习率（前 4000 步为升温阶段）；

每个子层输出处 0.1 的 dropout；0.1 的标签平滑。它们在论文第 5 节，读原文时遇到不必紧张，都属于“让训练更稳”的工程手段。

易错 · 本章四个高频错误　① **认为解码器只比编码器多一个“掩码”**。它每层还多一个完整的交叉注意力子层。② **认为交叉注意力的 Q 来自编码器**。反了：提问的是解码器，被查的是编码器。③ **认为掩码用在编码器里**。原始 Transformer 的编码器可以看到整句，掩码只在解码器的自注意力里。④ **认为“Transformer 没有顺序概念所以不分先后生成”**。它在**理解**时并行，在**生成**时仍然是一个词一个词地输出。

到这里，我们已经解决了什么？下一步呢？　Transformer＝Embedding＋位置编码，接 N 层编码器（自注意力、FFN，各带 Add & Norm），接 N 层解码器（掩码自注意力、交叉注意力、FFN），最后 Linear＋Softmax。它是为“读一句、写一句”的翻译任务设计的。可是今天的聊天助手并不是在做翻译——如果任务只是“接着往下写”，还需要编码器吗？下一章从这个问题出发，引出 GPT 与 BERT。
自检：（1）编码器一层有几个子层？解码器呢？（2）$\mathrm{LayerNorm}(x+\mathrm{Sublayer}(x))$ 中哪部分是残差连接？（3）FFN 是否让词与词交换信息？（4）因果掩码是什么形状？它在 Softmax 之前还是之后起作用？（5）交叉注意力的 Q、K、V 分别来自哪里？（6）残差连接为什么有助于训练深层网络？

第八章　BERT 与 GPT：把 Transformer 拆开来用

本章回答四个问题：如果任务不是翻译，Transformer 的两半还都需要吗？BERT 和 GPT 各自怎样训练？GPT 是怎样一个词一个词写出回答的？同一个问题为什么每次回答可能不同？　**知识路线：**三种用法 → BERT → GPT 的结构与训练 → 生成过程 → 温度。

8.1　从 Transformer 到三种用法

上一章的 Transformer 是为翻译设计的：编码器读原文，解码器写译文。但很多任务并不需要两半都在：

- 判断一条评论是好评还是差评——只需要**读懂**，不需要生成一段话；
- 接着一段开头往下写——没有单独的“原文”，只有已经写出的前文。

于是 2018 年出现了两条路：只保留编码器，或者只保留解码器。课件 p.49 的说法是：**Transformer 是积木，BERT 和 GPT 是不同的组合方式**。



图 44　Transformer 的三种典型用法（重绘自课件 p.49）

8.2　BERT：只用编码器，做“完形填空”

编码器的自注意力没有掩码，每个词都能同时看到左边和右边的全部上下文。BERT 利用的正是这一点，课件 p.6 称之为“双向语言建模”。

但这也带来一个问题：既然每个词都能看到整句话，就不能再用“预测下一个词”来训练了——答案就摆在右边。笔记在这里问：**Encoder-only 的目标函数是什么？**课件没有写，这里补上。

定义 · 掩码语言模型（Masked Language Model，MLM） 教学补全

随机挑出句子里的一部分 token（BERT 论文中约为 15%），把它们遮住，让模型根据**左右两侧**的上下文把被遮住的词猜出来。损失函数是被遮住位置上的交叉熵：

$$L_{\text{MLM}} = -\frac{1}{|M|}\sum_{t\in M} \ln P\big(x_t \mid \text{被遮住后的整句}\big),$$

其中 M 是被遮住的位置的集合，x_t 是该位置原来的词。

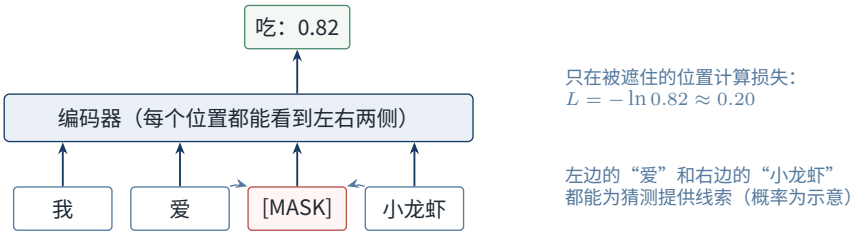


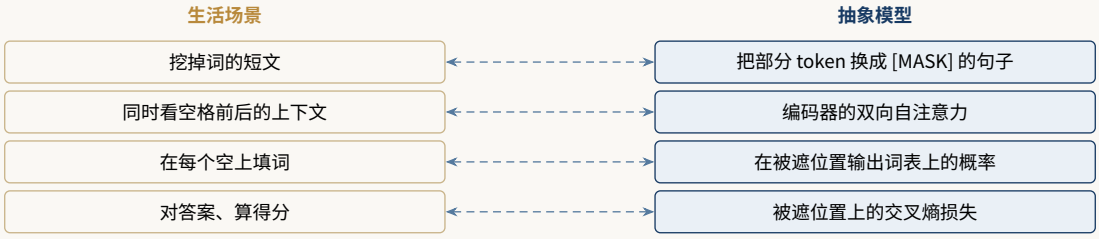
图 45　掩码语言模型：遮住一个词，根据两侧上下文把它猜回来

BERT 的原始论文还搭配了第二个训练任务——判断两个句子是否前后相邻（Next Sentence Prediction）；当前阶段只需掌握 MLM。

再回答笔记的追问：“最终向量状态怎么确定？”笔记写的是“让训练中的每个词达到最终向量状态 → 机器理解了”。需要澄清的是：训练并没有事先规定每个词“应该”变成哪个向量。目标函数只有上面那个交叉熵；参数（包括 W^Q、W^K、W^V 等）朝着让损失变小的方向调整，每个词经过 N 层自注意力之后得到的向量，是这个过程的**结果**。所以笔记里“$\Rightarrow$ Q、K、V 计算”这半句是对的——向量确实是由各层的 Q、K、V 计算一步步算出来的；只是“算成什么样才好”，由“能不能把被遮住的词猜对”来评判。

生活中的它 · 英语考试里的完形填空

生活场景 一段短文挖掉了若干个词，要求根据上下文把每个空填上。做题时你会同时看空格前面和后面的内容。



一次小估算 一句话有 20 个 token，按 15% 的比例大约遮住 $20 \times 15\% = 3$ 个，每句话只在这 3 个位置上产生训练信号。

边界说明 考试的空是出题人精心挑的；BERT 的遮挡位置是随机的，而且同一句话在不同训练轮次里遮的位置可以不同。

读懂了，却“不会开口”：BERT 的每个位置都看得到整句，没法一个词一个词地往下写。生成要靠另一条路。

8.3 GPT：只用解码器，专门做生成

定义 · GPT = Generative Pre-trained Transformer 原 · 课件 p.50

Generative（生成式）：不是只判断类别，而是能够继续生成文本。
Pre-trained（预训练）：先在海量文本上做通识训练。
Transformer：底层使用 Transformer 架构，并且只保留解码器（Decoder-only）。
一句话：GPT 是一个先被海量文本预训练过、基于 Transformer、能够生成内容的模型。

为什么只用解码器？因为生成任务的形式是“根据前文预测下一个 token”，而这正是解码器的工作方式：带因果掩码的自注意力保证每个位置只看前文，然后输出下一个词的概率。课件 p.51 的结论是“生成任务天然匹配 Decoder”。

教学补全 这里有一个细节需要说清。原始解码器的每层有三个子层，中间那个交叉注意力要用到编码器的输出；GPT 没有编码器，所以**这个子层被去掉了**。剩下的每层只有两个子层：掩码多头自注意力、前馈网络。

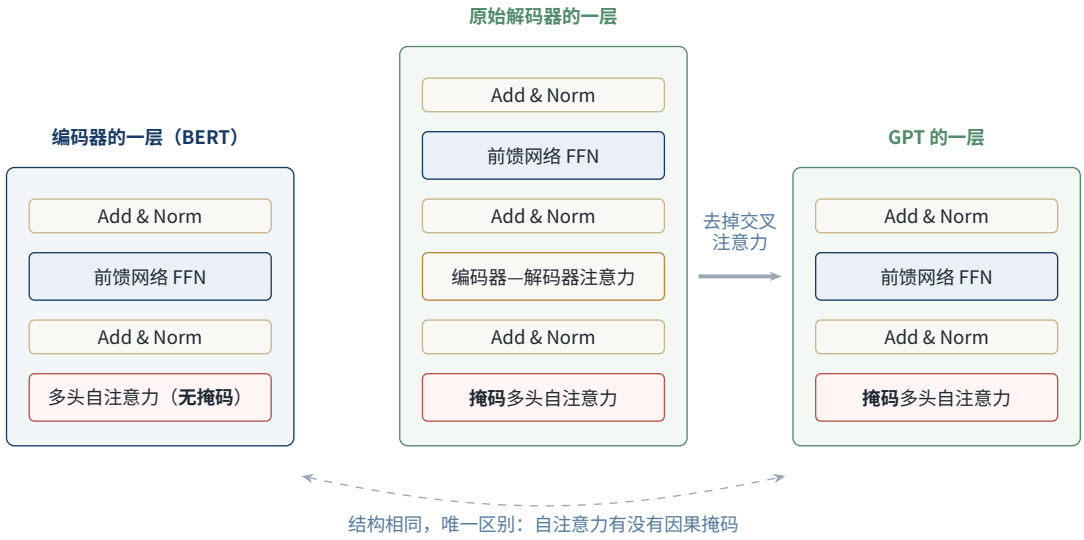


图 46　三种“一层”的对比。GPT 的一层＝原始解码器去掉交叉注意力；它与编码器一层的差别只在掩码

这幅图也解释了笔记里那句“Decoder 包含 encoder 相当大功能”：去掉交叉注意力之后，GPT 的一层和编码器的一层在结构上完全相同，都是“自注意力＋前馈网络”，**唯一的差别是自注意力有没有因果掩码**。有掩码，每个位置只看前文，可以用来生成；没有掩码，每个位置看整句，更适合理解。

8.4 GPT 如何训练：不断预测下一个 token

训练任务就是第 4 章的语言建模，一字未改。课件 p.52 用“我 爱 吃 小龙虾 。”展示了训练样本的构造：

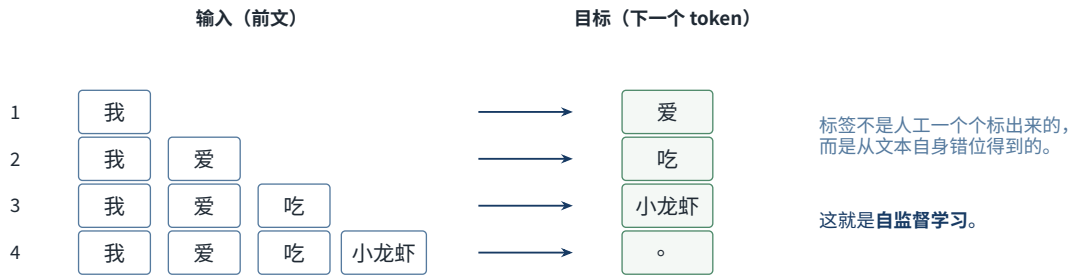


图 47　一句 5 个 token 的话，自动产生 4 组“输入 → 目标”（重绘自课件 p.52）

核心公式 · GPT 的训练目标 教学补全

对一段 token 序列 $x_1, x_2, \ldots, x_T$，

$$L = -\frac{1}{T}\sum_{t=1}^{T} \ln P(x_t \mid x_1, \ldots, x_{t-1})$$

即：每个位置都用前文预测当前 token，取各位置交叉熵的平均。

每个符号的含义。$P(x_t \mid x_1, \ldots, x_{t-1})$：给定前 $t-1$ 个 token 时，模型分给“第 t 个 token 恰好是 x_t”的概率，由最后的 Softmax 输出。T：序列的 token 数。这就是 4.3 节的交叉熵在每个位置上各算一次。

为什么“预测下一个词”就等于“学会整段文字”？ 教学补全 概率论的乘法公式告诉我们，一整句话出现的概率可以拆成一连串条件概率的乘积：

$$P(x_1, x_2, \ldots, x_T) = P(x_1) \cdot P(x_2 \mid x_1) \cdot P(x_3 \mid x_1, x_2) \cdots P(x_T \mid x_1, \ldots, x_{T-1}).$$

右边每一项正是“根据前文预测下一个词”。把每一项都学好，就等于学会了整句话的概率——两者在数学上是同一件事。

一次前向计算，T 个训练信号。由于因果掩码，把整句话送进模型一次，每个位置就同时给出自己对下一个 token 的预测（7.6 节）。上图的 4 组样本不必分 4 次计算，这也是 Transformer 比 RNN 训练快的原因之一。

只是预测下一个 token，为什么能学会这么多？

这是笔记里的又一个问号。课件 p.53 的回答是：**为了预测好下一个 token，模型被迫学习语言和世界的规律。**

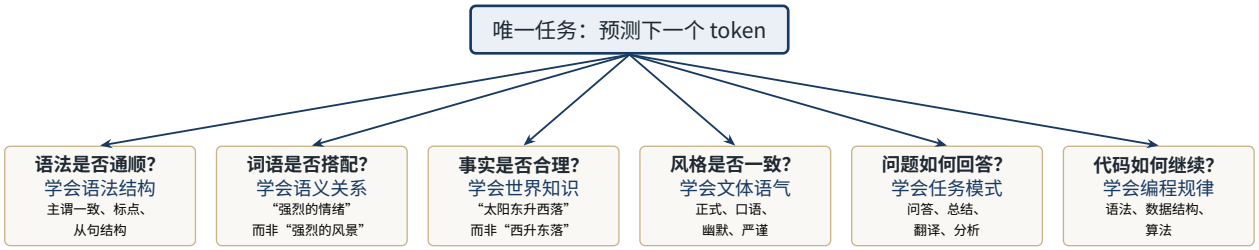


图 48　为了把下一个 token 猜对，模型不得不学会的六类规律（内容取自课件 p.53）

可以这样理解：要在“太阳从___边升起”的空格上给“东”很高的概率，模型的参数里必须以某种方式编码了这条常识；要续写一段代码，必须掌握这门语言的语法。训练目标只有一个，但要把它做好，所需要的知识是全方位的。

8.5　GPT 生成文本的过程

训练好之后怎样用它写东西？课件 p.54：**不是一次写完整答案，而是一个 token 一个 token 地生成。**

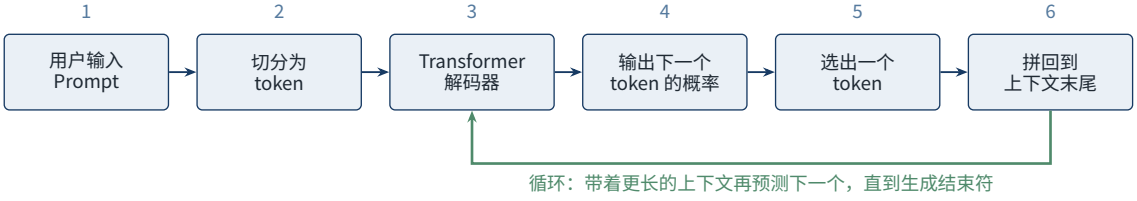


图 49　自回归生成的循环（重绘自课件 p.54）

例 · 续写“人工智能的核心是”

第 1 步　把“人工智能的核心是”送入模型，得到下一个 token 的概率分布，选出“让”。
第 2 步　把“让”接到后面，上下文变成“人工智能的核心是 让”。
第 3 步　把新的上下文再送入模型，选出“机器”。
第 4 步　上下文变成“人工智能的核心是 让 机器”，再预测，选出“学习”……
如此循环，直到模型选出**结束符**，生成完成。 原 · 课件 p.54



本例想说明　模型每次只做一件事：给“下一个 token”一个概率分布。一长段回答是这个动作重复几百上千次的结果。

第 5 个环节“选出一个 token”是关键。怎么选？有两种基本办法：

	贪心解码	采样解码
做法	每次都选概率最高的那个 token	把概率当作“中签率”，随机抽一个
同一输入的结果	每次相同	每次可能不同
特点	稳定、保守，容易千篇一律	多样、有变化，偶尔会抽到不太合适的词
课件出处	p.50“模型选择概率最高的 token”	p.55“从这个分布中采样”

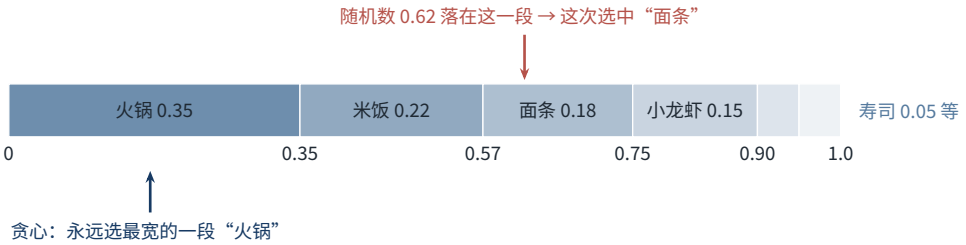
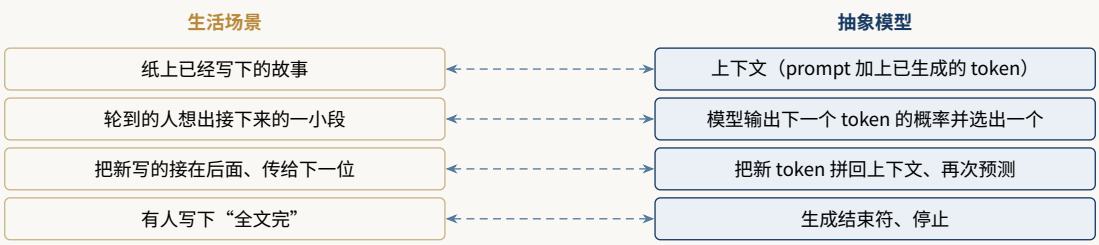


图 50　采样的含义：把 $[0, 1]$ 按概率切成小段，随机落在哪一段就选哪个词（概率取自课件 p.55）

前文是“今天晚上我想吃”。把 0 到 1 的线段按各候选的概率切开：“火锅”占 0.35，“米饭”占 0.22……然后产生一个 0 到 1 之间的随机数，落在哪一段就输出哪个词。“火锅”那段最宽，最容易被选中，但并不是每次都中。**这就是同一个问题每次回答可能不一样的直接原因：模型在概率分布上进行采样。**

生活中的它 · 文字接龙

生活场景 几个人玩故事接龙：每人只能看到前面已经写下的内容，然后接上一小段，下一个人再接着写。 原 · 课件 p.51“接龙写作者”



一次小估算 一段 300 个 token 的回答，模型要把“预测—选择—拼回”的循环执行 300 次；每一次都要重新考虑到目前为止的全部上下文。
边界说明 接龙的人事先大致想好了情节；模型没有预先写好的提纲，每一步都只是在给下一个 token 打概率。

8.6 Temperature：控制采样的随机程度

能不能调节“有多随机”？可以，用一个叫**温度**的参数。 原 · 课件 p.55

核心公式 · 带温度的 Softmax 教学补全

设模型输出的分数为 $z_1, \dots, z_n$，温度为 $T > 0$，则

$$p_i = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}$$

即：先把所有分数除以 T，再做普通的 Softmax。$T = 1$ 时就是原来的 Softmax。

课件只给了“低温更稳定、高温更随机”的结论，上面的公式说明了原因。回忆 4.2 节：两个候选的概率之比只取决于分数之差。加入温度后，

$$\frac{p_i}{p_j} = e^{(z_i - z_j)/T}.$$

T 小，分数差被放大，分布变尖；T 大，分数差被缩小，分布变平。

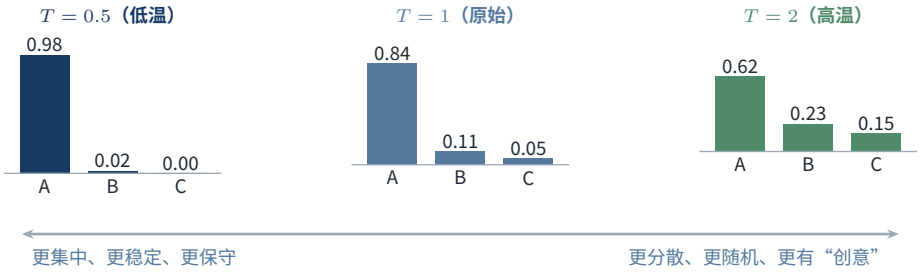


图 51　同一组分数 (3, 1, 0.2) 在三种温度下的概率分布

三幅图是同一组分数，只有温度不同。低温时第一名几乎独占全部概率，采样结果几乎总是它；高温时三根柱子靠近，后两名被抽中的机会明显变大。无论温度多少，**柱子的高低顺序不变**——温度不改变“谁最可能”，只改变“差距有多大”。

例 · 验算图 51 中 $T = 2$ 的一组

已知　分数 $(3,\ 1,\ 0.2)$，$T = 2$。　**求**　三个概率。
第一步，除以温度。 $(3/2,\ 1/2,\ 0.2/2) = (1.5,\ 0.5,\ 0.1)$。
第二步，取指数。 $e^{1.5} \approx 4.482$，$e^{0.5} \approx 1.649$，$e^{0.1} \approx 1.105$；总和 ≈ 7.236。
第三步，相除。 $4.482/7.236 \approx 0.62$，$1.649/7.236 \approx 0.23$，$1.105/7.236 \approx 0.15$。
本例想说明　温度不是新的运算，只是 Softmax 之前多了一步“除以 T”。

结论 · 温度的两个极限 教学补全

设各分数互不相等。**（1）**当 $T \to 0^+$ 时，分数最高的候选的概率趋于 1，其余趋于 0——这就是贪心解码。**（2）**当 $T \to +\infty$ 时，每个候选的概率都趋于 $1/n$——均匀分布，完全随机。

证明思路　设第 m 个分数最大。由 $p_i/p_m = e^{(z_i - z_m)/T}$：对 $i \neq m$ 有 $z_i - z_m < 0$。

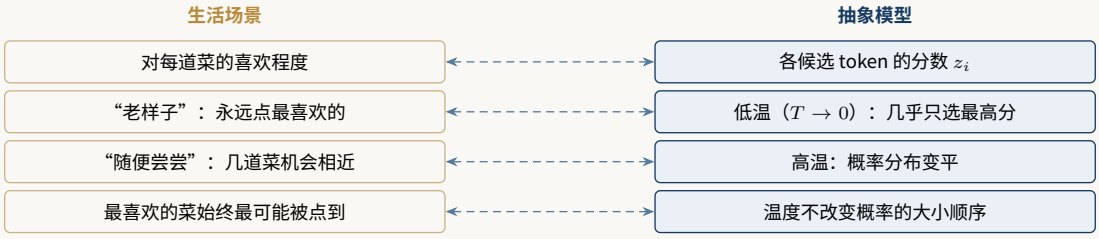
（1）$T \to 0^+$ 时，$(z_i - z_m)/T \to -\infty$，故 $p_i/p_m \to 0$，即其余候选的概率相对于第一名趋于 0；又因为概率之和为 1，所以 $p_m \to 1$。

（2）$T \to +\infty$ 时，$(z_i - z_m)/T \to 0$，故 $p_i/p_m \to e^0 = 1$，所有概率趋于相等；n 个相等的数之和为 1，所以每个都趋于 $1/n$。 ■

这条结论把课件两处看似不同的说法统一了起来：p.50 的“选择概率最高的 token”是温度趋于 0 的极限；p.55 的“采样”是温度大于 0 的一般情形。笔记里记的“一些网页版模型 temperature＝0.2”无法核验具体数值（见第 1 章），但它表达的意思是清楚的：面向问答的产品倾向于用偏低的温度，让回答稳定可靠；需要发散创意时才把温度调高。

生活中的它 · 去常去的餐馆点菜

生活场景　你对这家店的几道菜各有偏好。有时直接说“老样子”，每次都点最喜欢的那道；有时想换换口味，就在几道还不错的菜里随手挑一个。



一次小估算　三道菜的分数为 $(3, 1, 0.2)$。$T = 1$ 时点到第一道的概率是 0.84，十次里大约八九次；$T = 2$ 时降到 0.62，十次里大约六次，另外四次会尝到别的。

边界说明　人的“想换口味”还受上次吃了什么的影响；模型每一步的采样只取决于当前的概率分布和温度，随机数之间互不相关。

易错 · 关于生成与温度的四个误解　①**“模型心里有一个完整答案，只是逐字显示”**。没有。每个 token 都是在前面所有 token 确定之后才被选出来的。②**“温度越高模型越聪明”**。温度不改变模型的知识，只改变抽签方式；太高会抽到不通顺的词。③**“温度会改变哪个词概率最高”**。不会，顺序不变。④**“回答不一样说明模型不稳定或出错”**。这是采样的正常结果；把温度设得很低，回答就会趋于一致。

8.7 相近概念辨析：BERT 与 GPT

比较维度	BERT	GPT
取自 Transformer 的	编码器（Encoder-only）	解码器（Decoder-only，去掉交叉注意力）
自注意力	无掩码，每个词看左右两侧	因果掩码，每个词只看左侧（前文）
训练任务	掩码语言模型：猜被遮住的词	语言建模：预测下一个词
擅长	理解：分类、阅读理解、匹配	生成：续写、对话、写作
类比 类比	完形填空、阅读理解老师	文字接龙、写作助手
常见错误	以为 BERT 可以直接续写长文	以为 GPT 生成时能看到“后面”的词



图 52　两者自注意力的“可见性表”：区别只在右上角那个三角形

到这里，我们已经解决了什么？下一步呢？　只保留编码器、用完形填空训练，得到擅长理解的 BERT；只保留解码器、用预测下一个词训练，得到擅长生成的 GPT。GPT 生成时一次只出一个 token，靠采样和温度决定选哪个。但到此为止的 GPT 只是一个“续写机”：给它一个问题，它可能接着写出另一个问题，而不是回答。从“会续写”到“会聊天”还差几步，这是最后一章的内容。

自检：（1）BERT 和 GPT 各取了 Transformer 的哪一部分？（2）GPT 的一层比原始解码器的一层少了什么？（3）“我 爱 吃 小龙虾 。”能构造出几组“前文 → 下一个 token”？（4）贪心解码与采样解码有什么不同？（5）温度从 1 降到 0.5，分布变尖还是变平？概率最高的词会变吗？（6）BERT 的训练任务叫什么？

第九章　从“会续写”到“会聊天”：微调、对齐与蒸馏

本章回答三个问题：只会续写的 GPT 怎样变成能听指令的助手？“对齐”是在对齐什么？蒸馏是不是把老师模型复制一份？　**知识路线：**预训练 → 指令微调 → 偏好对齐 → 蒸馏。

9.1 三个阶段：会说话、会听话、更像助手

上一章结束时的 GPT 只会一件事：接着往下写。给它“请把下面这句话翻译成英文：我爱吃小龙虾”，它完全可能续写出“请把下面这句话翻译成法文：……”，因为在它读过的文本里，一道题后面跟着另一道题是很常见的。它并不知道“这是一条指令，应该照做”。

课件 p.57：原始 GPT 只会续写，但聊天助手还要**会听指令、会回答问题、会遵守规则**。为此在预训练之后再加两个阶段：

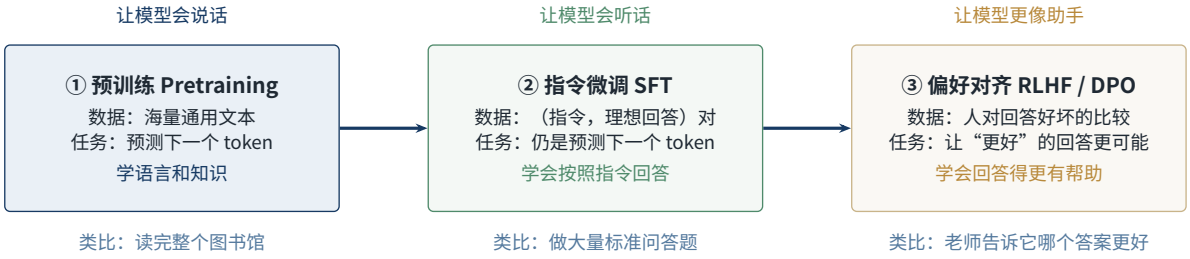


图 53　从预训练模型到聊天助手的三个阶段（重绘自课件 p.57）

课件只给出了三个阶段的名字和类比，下面把每个缩写展开。教学补全

定义 · 三个阶段的规范说法

预训练（Pretraining）：第 8 章讲的全部内容——在大规模文本上用“预测下一个 token”训练。
指令微调（SFT，Supervised Fine-Tuning，有监督微调）：准备大量“指令—理想回答”形式的样本，在预训练好的参数基础上继续训练。训练目标仍然是预测下一个 token，只是数据换成了问答格式，模型由此学会“看到问题之后应当给出回答”。
偏好对齐：让模型的输出更符合人的偏好。常见的两种做法是 **RLHF**（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）和 **DPO**（Direct Preference Optimization，直接偏好优化）。

RLHF 与 DPO 的差别，一句话各说清。两者用的原料相同：对同一个问题，人来比较两个回答哪个更好。RLHF 先用这些比较训练一个“打分模型”（奖励模型），再用强化学习让语言模型朝高分方向调整；DPO 则省去单独的打分模型和强化学习过程，直接用“较好回答—较差回答”这样的成对数据构造损失函数来训练。当前阶段只需知道二者的目的相同、原料相同，路径不同。

关于“对齐”——笔记用红笔写的“安全对齐”。“对齐”（alignment）这个词的意思是让模型的行为与人的意图和价值取向保持一致。它不只是“回答得更讨人喜欢”，更重要的一部分是**安全**：遵守规则、拒绝明显有害的请求、不确定的时候不乱编。课件 p.57 说助手要“会遵守规则”，p.61 列出的“安全拒答方式”，指的都是这一层。

	预训练	指令微调	偏好对齐
数据规模	最大（海量通用文本）	中等（人工整理的问答）	较小（人工比较与评价）
是否改动参数	是	是	是
学到什么	语言、常识、知识	任务格式、照指令做事	什么样的回答更好、更安全
没有它会怎样	根本不会说话	只会续写、不会回答	会回答，但可能啰嗦、不可靠或不守规则

例 · 同一条输入，三个阶段的模型各会怎样接

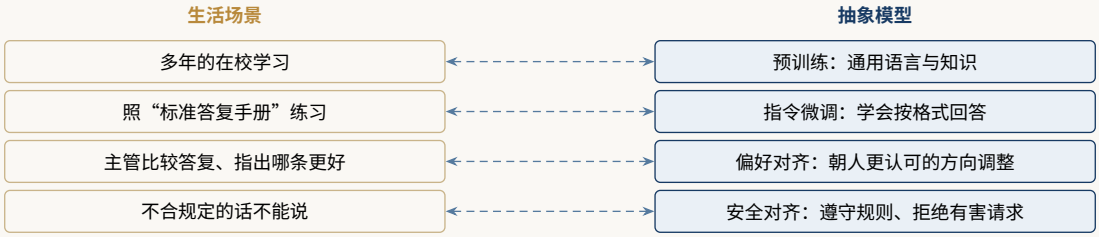
输入　“中国的首都是哪里？”

只做了预训练	“日本的首都是哪里？美国的首都是……”（像在续写一份题库）
加上指令微调	“北京。”（知道该回答问题了）
再加偏好对齐	“中国的首都是北京。”（完整、礼貌、不多不少）

本例想说明　三个阶段模型的“知识”差别不大——首都是北京这件事预训练时就学到了；差别在于**怎样使用**这些知识。（三条输出均为示意。）

生活中的它 · 新人入职的三步

生活场景 一位毕业生进公司做客服：上学时学会了读写和各科常识；入职后拿到一本“常见问题与标准答复”照着练；正式上岗后，主管每周抽查几条答复，指出哪条更好、哪条不合规定。



一次小判断 三步里哪一步用到的材料最多？在校学习——对应预训练的数据量远大于后两个阶段。
边界说明 人能说出“我为什么这样回答”；模型的三个阶段都只是在调整同一组参数，没有哪一部分参数专门负责“守规则”。

9.2 今天的大模型：不只是聊天

经过这三个阶段，同一个模型可以承担课件 p.58 列出的许多角色：写作、翻译、代码生成、长文档理解、问答、总结、数据分析、多模态理解，以及调用工具、分步执行任务（Agent）。课件把它概括为“面向多任务的通用智能底座”——这正好回到第 3 章的定义：Foundation Model。



图 54　课件 p.58 列出的能力（重绘）

课件同时列出了几个常见的模型家族：GPT、Claude、Gemini、Llama、Qwen、DeepSeek。它们的具体型号更新很快，讲义不逐一介绍；对本课程重要的是：**它们的骨架都是本章学的 Transformer，训练流程都大致遵循“预训练—微调—对齐”**。

9.3 蒸馏：让一个模型向另一个模型学习

训练一个大模型要耗费巨大的数据和算力，大模型用起来也又慢又贵。于是有了一个很实际的问题：**能不能让一个小模型直接向已经训练好的强模型学？**课件 p.59 的说法很形象：“训练也可以直接抄作业”。

定义 · 蒸馏（Knowledge Distillation，知识蒸馏） 原 · 课件 p.60

让一个模型向另一个更强的模型学习。更强、更大、知识更丰富的一方叫**老师模型**（Teacher），更小、更轻量、更高效的一方叫**学生模型**（Student）。蒸馏训练不是只看真实答案，而是让学生模型**模仿老师模型的输出**。



图 55　普通训练与蒸馏训练的流程对比（重绘自课件 p.60）

为什么这像“走捷径”？原本训练一个模型，语言规律、知识结构、推理方式都要从海量文本里一点点摸索。有了强老师，学生可以直接学习老师**整理好的输出**：回答格式、推理步骤、语言风格、问题拆解方式……课件 p.61 的说法是：老师模型的输出已经包含了很多“压缩后的经验”。所以学生虽然参数少，也能较快获得一部分能力。蒸馏常用于四种转换：大模型 → 小模型，强模型 → 弱模型，昂贵模型 → 便宜模型，通用模型 → 专用模型。

老师的输出里到底多了什么信息 教学补全

课件说“模仿老师的输出”，这里补充经典蒸馏方法（Hinton 等人，2015 年）中的一个关键观察，它能解释“为什么向老师学比只看标准答案信息更多”。

真实数据里，下一个词的标签是**硬标签**：正确的那个词是 1，其余全是 0。老师模型给出的却是一整个概率分布，称为**软标签**：

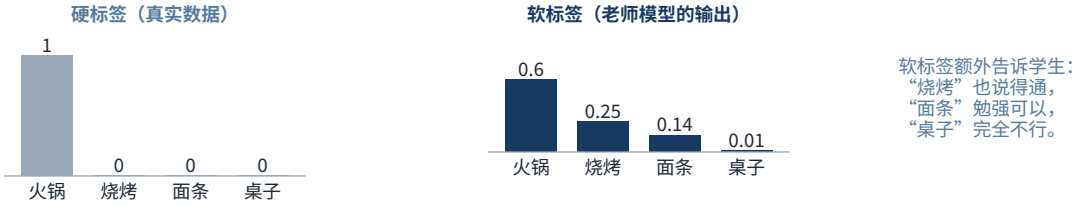


图 56　“今晚我想吃__”的两种标签（数值为示意）

硬标签只说“答案是火锅”；软标签还说明了**其余选项各有多不靠谱**。“烧烤比桌子合理得多”这类信息，正是老师从海量数据中总结出来的经验，学生一次就能学到。训练时把交叉熵里的独热标签换成老师的分布即可：

$$L_{\text{蒸馏}} = -\sum_{i=1}^{n} p_i^{\text{老师}} \ln p_i^{\text{学生}}.$$

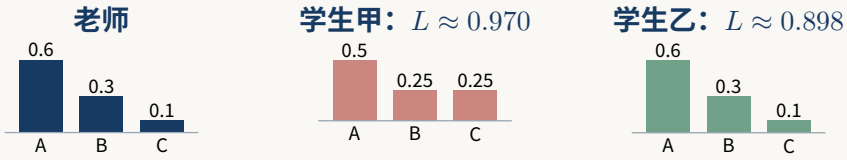
这就是 4.3 节交叉熵一般形式里的 t 由独热向量换成了老师的概率。经典做法还会在老师和学生的 Softmax 里用同一个大于 1 的温度（8.6 节），让软标签更“软”，把小概率选项之间的差别也显露出来。

例 · 软标签下的损失：学生越像老师，损失越小

已知　三个候选，老师的分布为 $(0.6,\ 0.3,\ 0.1)$。学生甲输出 $(0.5,\ 0.25,\ 0.25)$；学生乙输出与老师完全相同。　**求**　两位学生的蒸馏损失。

学生甲　$L = -(0.6\ln 0.5 + 0.3\ln 0.25 + 0.1\ln 0.25) = 0.6\times 0.693 + 0.3\times 1.386 + 0.1\times 1.386 \approx 0.970$。

学生乙　$L = -(0.6\ln 0.6 + 0.3\ln 0.3 + 0.1\ln 0.1) = 0.6\times 0.511 + 0.3\times 1.204 + 0.1\times 2.303 \approx 0.898$。



本例想说明　学生的分布与老师完全一致时损失最小（可以证明这是一般结论，严格证明要用到后续课程中的不等式，这里只用数值验证）。所以最小化这个损失，就是在让学生“回答得像老师一样”。

对今天的语言大模型，很多时候拿不到老师完整的概率分布，只能拿到它生成的文字。这时的做法就是课件 p.60 所画的：准备大量问题，让老师写出高质量回答，把“问题—回答”当作训练数据，对学生做指令微调。两种做法精神一致：**学生学的是老师的输出**。

蒸馏是不是复制老师模型？

这是笔记最后一组问题。课件 p.62 的回答很明确：**不是复制大脑，而是模仿输出结果**。



图 57　“抄权重”与“蒸馏”的区别

笔记：“要真抄作业该抄什么：权重（W）”。这句话说到了点子上。一个模型学到的全部东西，都存放在它的参数——那些权重矩阵 W 里（第 3 章）。真正的“原样照抄”是把这些数拷贝过来，这需要拿到权重文件本身。蒸馏不碰老师的权重，只看老师“对什么输入给出什么输出”，然后训练学生自己的一套参数 W' 去模仿。课件的类比 类比：学生不是复制老师的大脑，而是通过老师讲课、做题、示范，慢慢学会类似的能力。

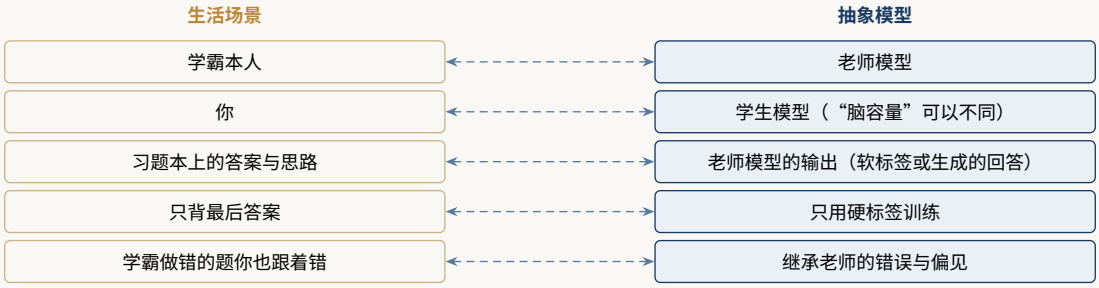
笔记：“开源的是什么？” 教学补全 笔记里记的具体型号讲义无法核验，这里只回答不依赖型号的部分。人们常说某个大模型“开源”，通常指的是**开放权重**：公开训练好的参数文件，一般连同模型结构的说明和运行所需的代码。有了权重，任何人都可以在自己的机器上直接运行或继续微调这个模型——这才是“抄到了 W”。训练数据和完整的训练过程则未必一并公开。相对地，只通过网页或接口提供服务的模型，外界只能看到输入和输出，想借用它的能力就只能走蒸馏这条路。

蒸馏不是万能的。课件 p.62 列出了五条局限和一条提醒，都值得原样记住：

局限	原因
很难超过老师模型的上限	学生的学习目标就是“像老师”
可能继承老师模型的错误	老师答错的地方，学生会照着错
可能学到老师模型的偏见	同上，偏见也在输出里
数据太少时效果有限	只见过老师很少的输出，模仿不全
模型太小时装不下全部能力	参数容量有限
合规提醒	未经授权大量使用别的商业模型的输出做训练，可能涉及版权、服务协议或合规问题

生活中的它 · 借学霸的解题笔记

生活场景 期末复习时，你借来学霸的习题本。上面不只有最后答案，还有每一步怎么想的、哪个选项“差一点就对了”。你照着学，掌握了他的大部分方法——但你并没有、也不可能把他的大脑复制过来。



一次小判断 如果学霸某道题的方法本身是错的，你照着学会不会也错？会——对应“可能继承老师模型的错误”。

边界说明 人借笔记只学几十道题；模型蒸馏通常需要老师在大量问题上的输出。另外，借笔记要经过同学同意，使用别的模型的输出同样要遵守授权与协议。

易错 · 关于蒸馏的三个误解 ①**“蒸馏就是把大模型压缩打包”**。学生的参数是**重新训练**出来的，不是从老师的参数里裁剪出来的。②**“蒸馏出来的学生和老师一样强”**。通常更弱，且很难超过老师。③**“蒸馏需要拿到老师的权重”**。不需要，只需要老师的输入输出；需要权重的是“直接复制”。

9.4 下节课预告与课后作业

笔记最后记了两行：**下节课预告：Agent、RAG、Harness；作业：通读 Transformer 论文（原文）**。

前两个词课件里已经露过面：p.7 的“检索增强”就是 RAG（Retrieval-Augmented Generation）——先检索相关资料，再把资料放进上下文让模型据此生成；p.58 的“Agent 任务执行”指让模型自己规划步骤、调用工具、分步完成任务。Harness 课件没有展开，一般指围绕模型搭建的运行与调度框架，具体含义以下节课为准。三者有一个共同点：**都不改动模型本身，而是在模型外面做文章**——这也是为什么要先把本章的模型内部弄清楚。

通读论文的作业，可以借助下一章的对照表（10.5 节）。

到这里，我们已经解决了什么？ 预训练让模型会说话，指令微调让它会听话，偏好对齐让它更像助手、更安全；蒸馏让小模型通过模仿强模型的输出快速获得一部分能力，但它不是复制权重，也有上限。至此，从“什么是大模型”到“今天的助手是怎样做出来的”这条线走完了。

自检：（1）只做了预训练的模型为什么不会“回答问题”？（2）SFT 的训练目标与预训练有什么相同和不同？（3）RLHF 和 DPO 用的原料是什么？（4）蒸馏时学生模仿的是老师的什么？（5）“复制权重”和“蒸馏”各需要拿到什么？（6）举出蒸馏的两条局限。

第十章　总结：知识地图、易错总表与论文阅读路线

10.1 全章知识地图

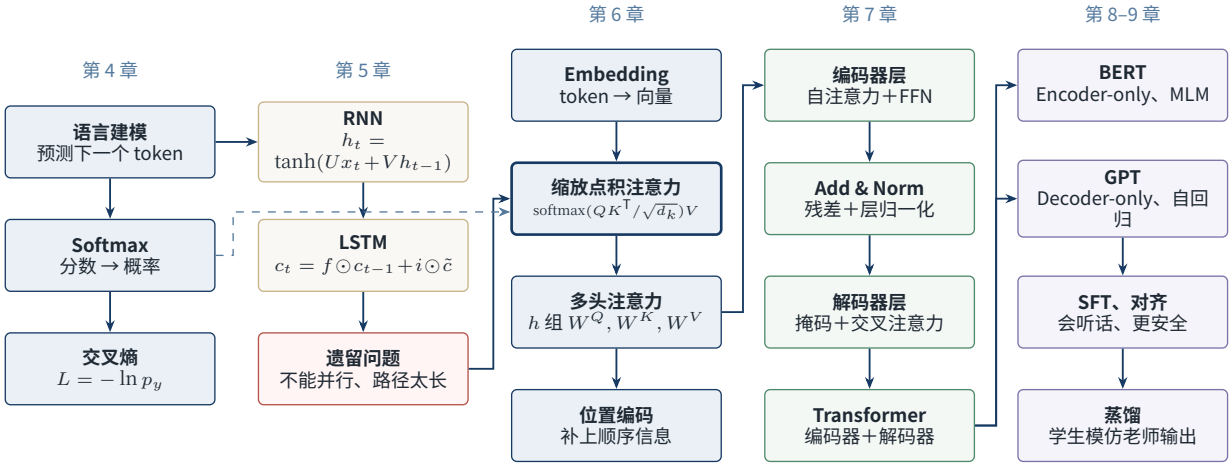


图 58　全章知识地图。实线表示“为后者提供基础或被后者改进”，虚线表示 Softmax 在注意力中被再次使用

五列从左到右就是本讲义的五步：先有任务和工具，再有“逐词读”的老办法及其问题，然后是注意力这组新零件，接着装配成 Transformer，最后拆成 BERT 与 GPT 并走向实用。

10.2 相近概念总对比

容易混淆的一组	到底哪里不同
RNN / LSTM / Transformer	RNN：一条状态链，逐步传递；LSTM：多一条受门控保护的记忆通道，仍逐步传递；Transformer：不传递，任意两词经注意力直接相连，可并行
Token ID / Embedding	ID 是词表里的整数编号，无含义；Embedding 是按 ID 查表得到的向量，含义相近则向量相近
Q / K / V	Q：我想找什么；K：我可供匹配的线索；V：我真正提供的内容。Q 与 K 做点积得到权重，权重作用在 V 上
自注意力 / 掩码自注意力 / 交叉注意力	公式相同。自注意力：Q, K, V 同源、可看全部；掩码：同源、只看到当前位置；交叉：Q 来自解码器，K, V 来自编码器
Softmax / 交叉熵	前者把分数变成概率（一组数），训练与使用时都要算；后者把概率与标准答案比较得到损失（一个数），只在训练时算
残差连接 / 层归一化	残差：把输入加回输出，保信息、保梯度；层归一化：把单个向量标准化，稳数值范围
位置编码 / Embedding	前者只与“在第几位”有关；后者只与“是哪个词”有关；二者相加
编码器 / 解码器	编码器每层 2 个子层、无掩码；解码器每层 3 个子层、自注意力带因果掩码、多一个交叉注意力
BERT / GPT	BERT：编码器、双向、完形填空、擅理解；GPT：解码器、单向、预测下一个词、擅生成
预训练 / 微调 / 提示	前两者改参数（数据量一大一小）；提示不改参数，只改输入
贪心 / 采样 / 温度	贪心每次取最大；采样按概率抽签；温度调节分布尖平，$T \to 0$ 时采样退化为贪心
蒸馏 / 复制权重	蒸馏只看老师的输入输出，学生重新训练自己的参数；复制权重需要拿到参数文件本身

10.3 高频易错点总表

序号	常见错误	正确认识
1	注意力权重矩阵的行和、列和都是 1	只有每一行的和是 1
2	除以 d_k	除以 $\sqrt{d_k}$（标准差）
3	d_k 是句子长度	d_k 是 query/key 向量的维数；句子长度是 n
4	点积越大表示距离越远	点积是相似度，越大越相关
5	位置编码拼接在词向量后面	是相加，维数不变
6	多头是把同一计算重复 h 遍	每个头有各自独立的投影矩阵
7	掩码是把权重乘以 0	是在 Softmax 之前把分数设为 $-\infty$
8	交叉注意力的 Q 来自编码器	Q 来自解码器，K、V 来自编码器
9	RNN 展开图里每个方框有各自的权重	所有时间步共享同一组权重
10	LSTM 解决了不能并行的问题	LSTM 只缓解了长期依赖，仍需逐步计算
11	GPT 生成时可以看到后面的词	因果掩码保证只看前文
12	温度会改变概率最高的词	温度只改变分布的尖平，不改变顺序
13	损失降到 0 才算训练好	语言的下一个词本就不唯一，损失不会也不应为 0
14	蒸馏是复制老师的参数	蒸馏是模仿老师的输出

10.4 基础解题步骤总结

五类基础题的固定步骤　**算 Softmax：** ① 每个分数取 e 的指数；② 求和；③ 逐个相除；④ 检查和为 1。

算一次注意力： ① $\boldsymbol{q}$ 与每个 $\boldsymbol{k}_j$ 做点积；② 除以 $\sqrt{d_k}$；③（如有掩码）把不允许的位置改为 $-\infty$；④ Softmax 得权重；⑤ 用权重对 $\boldsymbol{v}_j$ 加权求和。

判断形状： Q：$n \times d_k$；K：$n \times d_k$；QK^{T}：$n \times n$；V：$n \times d_v$；输出：$n \times d_v$。交叉注意力把第一个 n 换成译文长度 m。

数参数： 一个 $a \times b$ 的权重矩阵有 ab 个参数，偏置另加 b 个；多头时每个头的 $d_k = d_{\text{model}}/h$。

算位置编码： ① 确定这一维属于第几对（i）；② 分母为 $10000^{2i/d_{\text{model}}}$；③ 偶数维取 sin，奇数维取 cos，角度用弧度。

10.5 后续课程连接与论文阅读路线

这章的知识以后在哪里再出现　**本课程下一节**：Agent、RAG 等都建立在“模型根据上下文生成”之上，上下文就是本章的注意力所关注的对象。**线性代数、概率统计**：本章反复使用的矩阵乘法、独立随机变量的方差，是这两门课的直接应用。**最优化**：梯度下降、学习率调度是训练所有这些模型的工具。**信号与系统**：位置编码的“不同频率的正弦分量”与傅里叶分析中用一组频率表示信号的思想相通。**拓展方向**：把图像块、语音帧当作 token，同一套结构可用于多模态模型。

课后作业“通读论文原文”的对照表。论文只有十页正文，结构与本讲义第 6、7 章几乎一一对应。建议按下表的顺序读，遇到看不懂的句子就回到右列的章节：

论文章节	读的时候抓住什么	讲义对应
摘要、1 引言、2 背景	循环网络为何难以并行；本文只用注意力	第 2、5 章
3.1 编码器与解码器堆叠	$N=6$；每层几个子层；$\mathrm{LayerNorm}(x+\mathrm{Sublayer}(x))$	7.3、7.6 节
3.2.1 缩放点积注意力	式 (1)；脚注 4 解释 $\sqrt{d_k}$	6.5、6.6 节
3.2.2 多头注意力	$h=8$，$d_k=d_v=64$	6.9 节
3.2.3 注意力的三种用法	交叉注意力、编码器自注意力、解码器掩码	6.8、7.7 节
3.3 逐位置前馈网络	式 (2)；$d_{ff}=2048$	7.4 节
3.4 Embedding 与 Softmax	权重共享；乘 $\sqrt{d_{\text{model}}}$	6.3、7.8 节
3.5 位置编码	正弦余弦公式；相对位置的线性关系	6.10 节
4 为什么用自注意力	表 1 的三列指标	第 6 章开头
5 训练、6 结果	表 2 的 BLEU；表 3 的消融实验	第 7 章末尾

10.6　学完全章的总自检

> **合上讲义，试着回答**　（1）用一句话说出大模型为什么叫 Foundation Model。　（2）语言模型的标签从哪里来？　（3）写出 Softmax 和交叉熵的公式。　（4）RNN 的两个根本缺点是什么？　（5）写出注意力公式，并说出 Q、K、V、d_k 各是什么。　（6）为什么要除以 $\sqrt{d_k}$？　（7）为什么需要位置编码？　（8）画出编码器一层和解码器一层的子层。　（9）BERT 与 GPT 的训练任务各是什么？　（10）温度升高，输出更确定还是更随机？　（11）蒸馏与复制权重有什么不同？
> 如果其中有答不上来的，回到知识地图找到对应的章节重读，再去做下一章的练习。

第十一章　分层练习：38 道

共 38 道正式练习：基础 基础巩固 25 道（单选 10、多选 5、填空 5、解答 5），提升 中等偏下提升 10 道（单选 3、多选 2、填空 3、综合 2），跨学科 跨学科迁移 3 道（单选、多选、填空各 1）。**请先独立完成，再翻到第 12 章对答案。**题目区的图只呈现题目给出的信息，不含任何提示。计算题需要的指数值：$e\approx 2.718$，$e^2\approx 7.389$，$e^{0.5}\approx 1.649$，$\ln 2\approx 0.693$。

知识点—题目对应表

核心知识点	基础题	提升题	跨学科
大模型、预训练—微调—对齐	1	—	—
语言建模、自监督标签	2、25	35	—
Softmax 及其性质	3、14、16、21	28、35	—
交叉熵损失	19、25	35	—
RNN / LSTM、梯度连乘	4、11、20	33	—
点积、缩放点积注意力、形状	5、6、12、17、22	34	38
为什么除以 $\sqrt{d_k}$	12	26	37
多头注意力、参数量	7	31	—
位置编码、顺序信息	8、23	27	36
Add & Norm、前馈网络、编码器	13、24	—	—
因果掩码、解码器、交叉注意力	9、18	29、32、34	—
复杂度与路径长度（论文表 1）	11	30	38
BERT / GPT、生成与温度	10、25	28、35	—
蒸馏	15	—	—

11.1　A　基础巩固训练（25 道）

一、单项选择题（第 1–10 题）

1. 基础　“大模型”的英文是 Foundation Model，其中 Foundation 一词强调的是（　）。

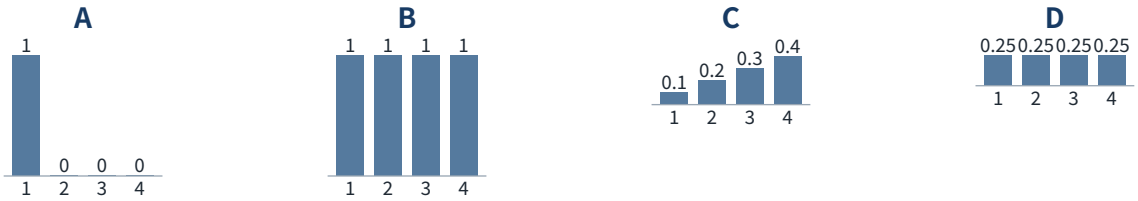
A. 模型的参数全部存放在底层硬件中
B. 模型是多种下游任务可以共同依托的底座
C. 模型只能完成一种最基础的任务
D. 模型的网络结构比传统模型更简单

2. 基础　用句子“今天 天气 很 好”训练语言模型。当输入为“今天 天气”时，这一步的标签是（　）。

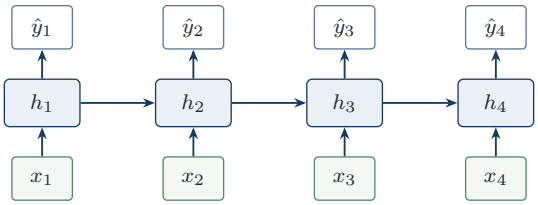


A. 今天　　B. 天气
C. 很　　D. 好

3. 基础　四个候选的原始分数都是 1。经过 Softmax 后得到的分布是下列哪一幅？（　）



4. 基础　下图是一个 RNN 按时间展开 4 步的示意。图中一共有（　）组不同的权重（U、V、W 算作一组）。



A. 1　　B. 2
C. 4　　D. 8

5. 基础　下列四个 2×2 矩阵中，可以作为一句两个词的话的注意力权重矩阵的是（　）。（每个矩阵的行对应“当前词”，列对应“被关注的词”。）

A

0.6	0.6
0.5	0.5

B

1.2	−0.2
0.5	0.5

C

0.5	0.5
0.9	0.1

D

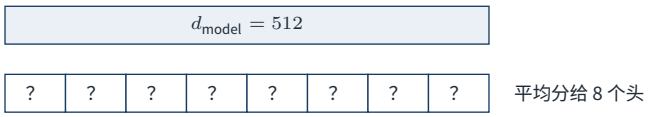
0.9	0.9
0.1	0.1

6. 基础 在注意力机制的 Q、K、V 中，含义为“我能提供什么线索，用来和别人的查询做匹配”的是（ ）。

A. Query　　B. Key

C. Value　　D. 位置编码

7. 基础 论文的基础模型取 $d_{\text{model}} = 512$、头数 $h = 8$，且每个头的维数 $d_k = d_{\text{model}}/h$。则 d_k 等于（ ）。



A. 8　　B. 56

C. 64　　D. 4096

8. 基础 Transformer 需要位置编码，主要原因是（ ）。



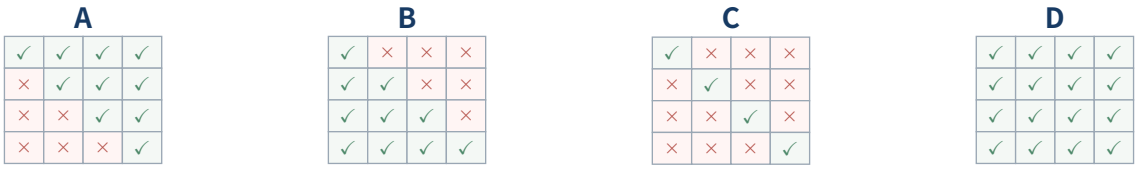
A. 减少模型的参数量

B. 代替 Softmax 完成归一化

C. 让句子变短以节省计算

D. 自注意力本身不区分词的先后，需要额外提供顺序信息

9. 基础 下列四幅图表示 4 个 token 的自注意力“可见性”（行：当前位置；列：可以关注的位置；✓ 可见，× 遮住）。解码器使用的因果掩码是（ ）。

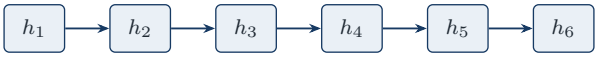


10. 基础 GPT 采用的结构对应下列哪一幅？（ ）



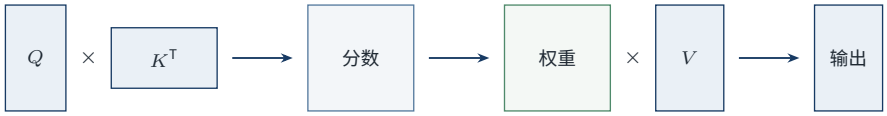
二、多项选择题（第 11–15 题）

11. 基础 下列属于 RNN 的缺点的是（　）。



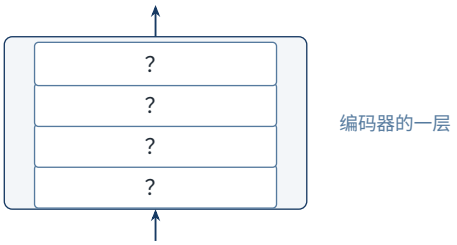
A. 各时间步必须先后计算，不能并行
B. 相距很远的词之间，信息在传递中逐渐丢失
C. 输入序列的长度必须固定
D. 每个时间步使用一组新的权重，参数量随句长增加

12. 基础 关于缩放点积注意力 $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})V$，下列说法正确的是（　）。



A. QK^{T} 的第 i 行第 j 列等于第 i 个 query 与第 j 个 key 的点积
B. Softmax 是对分数矩阵的每一**列**分别做的
C. 除以 $\sqrt{d_k}$ 是为了让点积的波动范围不随维数增大
D. 输出的行数等于 Q 的行数

13. 基础 原始 Transformer 的**编码器**的一层中，包含下列哪些部分？（　）



A. 多头自注意力
B. 前馈网络
C. 编码器—解码器注意力
D. 残差连接与层归一化

14. 基础 设 $p = \mathrm{softmax}(z)$。下列关于 Softmax 的说法正确的是（　）。

A. p 的每个分量都大于 0
B. p 的各分量之和等于 1
C. 给所有分数加上同一个常数，p 不变
D. 把所有分数都乘以 2，p 不变

15. 基础 关于大模型的蒸馏，下列说法正确的是（　）。



A. 蒸馏必须先拿到老师模型的全部权重再拷贝给学生
B. 学生模型学习的是老师模型的输出
C. 学生模型通常比老师模型更小、更高效
D. 学生模型可能继承老师模型的错误和偏见

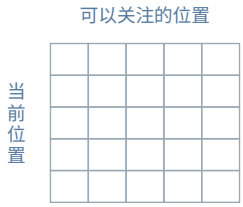
三、填空题（第 16–20 题）

16. 基础 两个候选的分数为 $z_1 = 0$、$z_2 = \ln 3$。经过 Softmax 后，第二个候选的概率 $p_2 =$_____。



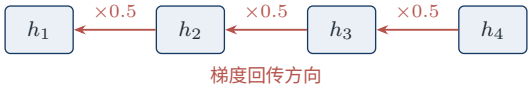
17. 基础 设 $d_k = 4$，$\boldsymbol{q} = (1,1,1,1)$，$\boldsymbol{k} = (2,0,2,0)$。则点积 $\boldsymbol{q}\cdot\boldsymbol{k}$ =_____，缩放后的分数 $\boldsymbol{q}\cdot\boldsymbol{k}/\sqrt{d_k}$ =_____。

18. 基础 一句话有 5 个 token。它的自注意力权重矩阵共有_____个元素；加上因果掩码后，允许取非零值的元素有_____个。



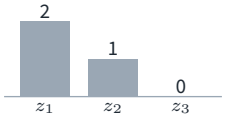
19. 基础 在某一步训练中，模型分给正确的下一个词的概率为 0.25。这一步的交叉熵损失 $L = -\ln p_y$ =_____（保留两位小数）。

20. 基础 某标量 RNN 中，梯度每往回传一个时间步就要乘以 0.5。往回传 3 步后，梯度变为原来的_____倍。

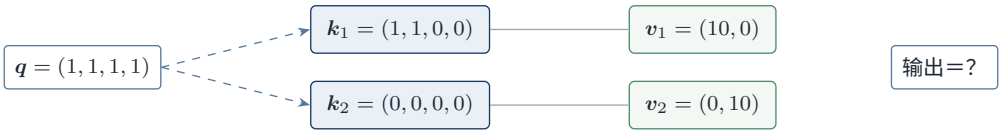


四、解答题（第 21–25 题）

21. 基础 计算 softmax(2, 1, 0)，结果保留三位小数，并验证三者之和为 1。

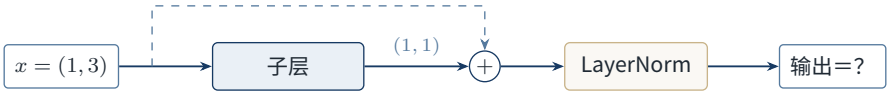


22. 基础　设 $d_k = 4$。某个词的查询向量为 $\boldsymbol{q} = (1,1,1,1)$；句中共有两个词，键向量为 $\boldsymbol{k}_1 = (1,1,0,0)$、$\boldsymbol{k}_2 = (0,0,0,0)$，值向量为 $\boldsymbol{v}_1 = (10,0)$、$\boldsymbol{v}_2 = (0,10)$。求这个词的注意力输出。



23. 基础　取 $d_{\text{model}} = 4$，按论文的正弦位置编码公式，写出 $pos = 0$ 和 $pos = 1$ 两个位置的 4 维编码向量。（已知 $\sin 1 \approx 0.841$，$\cos 1 \approx 0.540$，$\sin 0.01 \approx 0.010$，$\cos 0.01 \approx 1.000$；角度以弧度计。）

24. 基础　某子层的输入为 $x = (1,3)$，子层的输出为 $\text{Sublayer}(x) = (1,1)$。按 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 计算这个子层的最终输出（取 $\gamma = 1$、$\beta = 0$，标准差按 $\sigma = \sqrt{\frac{1}{d}\sum_i (x_i - \mu)^2}$ 计算）。



25. 基础　用句子“猫 喜欢 吃 鱼”（4 个 token）训练一个 GPT 式的语言模型。（1）列出这句话能构造出的全部“前文 → 下一个 token”训练对；（2）说明为什么这种训练被称为自监督学习；（3）若输入“猫 喜欢”时模型给“吃”的概率是 0.8，求这一位置的交叉熵损失（已知 $\ln 0.8 \approx -0.223$）。



11.2　B　中等偏下提升训练（10 道）

一、单项选择题（第 26–28 题）

26. 提升　设 $\boldsymbol{q}$、$\boldsymbol{k}$ 的各个分量相互独立，均值为 0、方差为 1，维数 $d_k = 64$。则点积 $\boldsymbol{q} \cdot \boldsymbol{k}$ 的标准差为（　）。

A. 1　　B. 8

C. 64　　D. 4096

27. 提升　不加位置编码时，把“我 爱 吃 饭”打乱成“饭 爱 吃 我”，送入同一个自注意力层。与打乱前相比，“我”这个词得到的输出向量（　）。



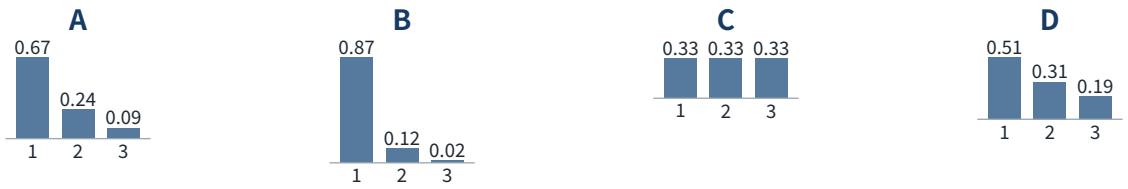
A. 变成了打乱前“饭”的输出向量

B. 变成零向量

C. 与打乱前完全相同，只是它所在的位置换了

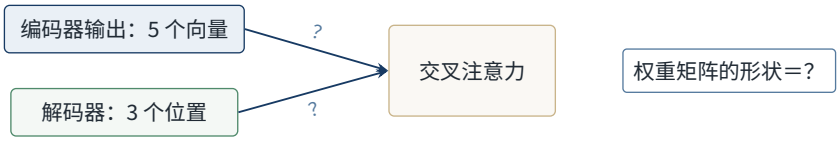
D. 变得与另外三个词的输出向量完全相同

28. 提升　某一步模型输出的分数为 $(2,\ 1,\ 0)$。下列四幅图是不同温度下的概率分布，其中对应 $T = 2$ 的是（　）。



二、多项选择题（第 29–30 题）

29. 提升 翻译任务中，原文有 5 个 token，解码器当前已有 3 个位置。关于解码器中的编码器—解码器注意力（交叉注意力），下列说法正确的是（　）。



A. Q 来自编码器的输出

B. K 和 V 来自编码器的最终输出

C. 注意力权重矩阵的形状是 3×5

D. 权重矩阵每一行之和等于 1

30. 提升 论文表 1 比较了自注意力层与循环层（n 为序列长度，d 为向量维数）。下列说法与该表一致的是（　）。

层的类型	每层计算量	顺序操作数	最大路径长度
自注意力	?	?	?
循环（RNN）	$O(n \cdot d^2)$	?	?

A. 自注意力层的顺序操作数为 $O(1)$

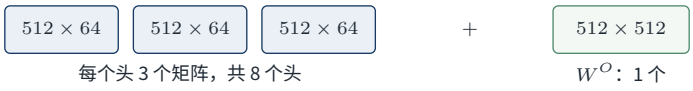
B. 循环层中任意两个位置之间的最大路径长度为 $O(n)$

C. 句子越长，自注意力层每层的计算量相对循环层就越小

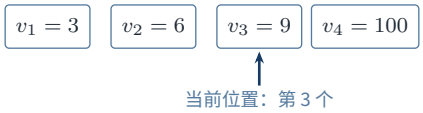
D. 自注意力层的每层计算量为 $O(n^2 \cdot d)$

三、填空题（第 31–33 题）

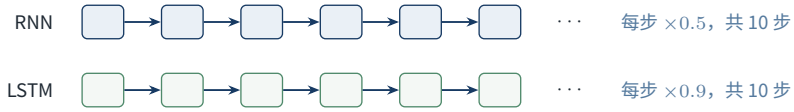
31. 提升 论文基础模型的一个多头注意力模块：$d_{\text{model}} = 512$，$h = 8$，$d_k = d_v = 64$。每个头有 W_i^Q、W_i^K、W_i^V 三个 512×64 的矩阵，另有一个 512×512 的 W^O。忽略偏置，这个模块共有_____个权重参数。



32. 提升 一句话有 4 个 token，各自的值是标量：$v_1 = 3$，$v_2 = 6$，$v_3 = 9$，$v_4 = 100$。在带因果掩码的自注意力中，第 3 个位置对所有**未被遮住**的位置的分数恰好相等。则第 3 个位置的输出为_____。



33. 提升 比较“回传 10 步”后的梯度：标量 RNN 每步乘 0.5，结果约为原来的_____倍；LSTM 沿记忆通道每步乘忘记门 $f = 0.9$，结果约为原来的_____倍。（第一空可写成分数；第二空保留两位小数，已知 $0.9^{10} \approx 0.349$。）



四、综合大题（第 34–35 题）

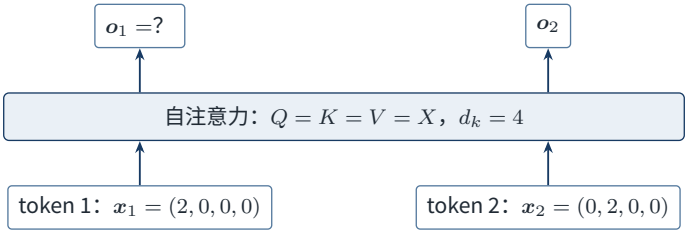
34. 提升 **（一次完整的自注意力）**一句话只有两个 token，输入向量为 $\boldsymbol{x}_1 = (2,0,0,0)$、$\boldsymbol{x}_2 = (0,2,0,0)$。为便于手算，设三个投影矩阵都是单位矩阵，即 $Q = K = V = X$，$d_k = 4$。

（1） 写出 2×2 的点积矩阵 QK^{T}；

（2） 写出缩放并逐行 Softmax 后的注意力权重矩阵（保留三位小数）；

（3） 求第 1 个 token 的输出向量 $\boldsymbol{o}_1$；

（4） 如果这是解码器中的自注意力，需要加因果掩码。此时第 1 个 token 的权重和输出向量各变成什么？



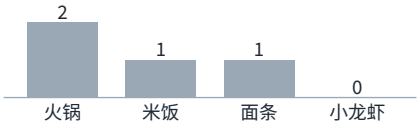
35. 提升 **（GPT 生成的一步）**前文为“今天晚上我想吃”。模型在最后一个位置输出的分数（logits）为：火锅 2，米饭 1，面条 1，小龙虾 0（假设词表里只有这四个词）。

（1） 取温度 $T = 1$，求四个词的概率（保留三位小数）；

（2） 若采用贪心解码，这一步输出哪个词？

（3） 若这是一条训练数据，真实的下一个词是“米饭”，求这一位置的交叉熵损失（保留两位小数）；

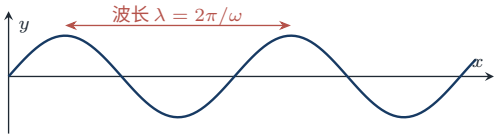
（4） 把温度改为 $T = 2$，重新计算四个概率（保留三位小数），并说明“火锅”的概率如何变化、输出变得更确定还是更随机。



11.3 C　跨学科迁移训练（3 道）

36. 跨学科 **（单选 · 借用大学物理中“波”的背景）**背景：对正弦函数 $y = \sin(\omega x)$，相邻两个波峰之间的距离称为波长，$\lambda = 2\pi/\omega$。

位置编码在某一维上的取值是 $\sin(pos/100)$，其中 pos 是词的位置。把 pos 看作自变量，这条正弦曲线的波长约为（　）个位置。



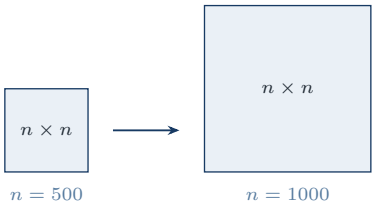
A. 6.28　　B. 100

C. 628　　D. 10000

37. 跨学科 **（多选 · 借用概率统计的背景）背景：**若随机变量 $X_1, \dots, X_m$ 相互独立，则 $\mathrm{Var}(X_1 + \cdots + X_m) = \mathrm{Var}(X_1) + \cdots + \mathrm{Var}(X_m)$；对常数 c，有 $\mathrm{Var}(cX) = c^2\,\mathrm{Var}(X)$；标准差是方差的平方根。
已知点积 $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i$ 中，各项 $q_i k_i$ 相互独立，每一项的方差都是 1，且 $d_k = 16$。下列说法正确的是（　）。

A. $\boldsymbol{q} \cdot \boldsymbol{k}$ 的方差为 16
B. $\boldsymbol{q} \cdot \boldsymbol{k}$ 的标准差为 16
C. $\boldsymbol{q} \cdot \boldsymbol{k}/\sqrt{d_k}$ 的方差为 1
D. $\boldsymbol{q} \cdot \boldsymbol{k}/d_k$ 的方差为 $1/16$

38. 跨学科 **（填空 · 借用数据结构与算法中“时间复杂度”的背景）背景：**如果一个算法的运算次数与 n^2 成正比，那么 n 变为原来的 c 倍时，运算次数变为原来的 c^2 倍；若与 n 成正比，则变为 c 倍。
自注意力要计算每一对 token 之间的分数，每层的计算量与句子长度 n 的平方成正比。句子长度从 500 增加到 1000（维数不变）时，这一层的计算量变为原来的_____倍。



第十二章　答案速查与详细解析

12.1　全部答案速查

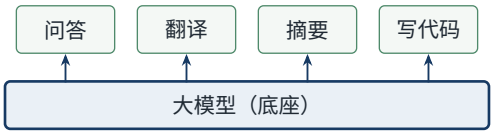
题组	答案
基础单选 1–10	1. B　2. C　3. D　4. A　5. C　6. B　7. C　8. D　9. B　10. C
基础多选 11–15	11. AB　12. ACD　13. ABD　14. ABC　15. BCD
基础填空 16–20	16. 0.75　17. 4；2　18. 25；15　19. 1.39　20. 0.125（即 $1/8$）
基础解答 21–25	21. $(0.665,\ 0.245,\ 0.090)$　22. $(7.31,\ 2.69)$　23. $(0,1,0,1)$；$(0.841,\ 0.540,\ 0.010,\ 1.000)$ 24. $(-1,\ 1)$　25. 3 组训练对；损失 ≈ 0.22
提升单选 26–28	26. B　27. C　28. D
提升多选 29–30	29. BCD　30. ABD
提升填空 31–33	31. 1 048 576　32. 6　33. $1/1024 \approx 0.001$；0.35
提升综合 34	（1）$\begin{pmatrix}4 & 0\\ 0 & 4\end{pmatrix}$　（2）$\begin{pmatrix}0.881 & 0.119\\ 0.119 & 0.881\end{pmatrix}$　（3）$(1.762,\ 0.238,\ 0,\ 0)$　（4）权重 $(1,0)$，输出 $(2,0,0,0)$
提升综合 35	（1）$(0.534,\ 0.197,\ 0.197,\ 0.072)$　（2）火锅　（3）1.63　（4）$(0.387,\ 0.235,\ 0.235,\ 0.143)$，火锅的概率下降，输出更随机
跨学科 36–38	36. C　37. ACD　38. 4

12.2　基础单选题解析（第 1–10 题）

第 1 题　答案：B

难度：基础　|　考查：大模型的定义（第 3 章）

思路 Foundation 意为“地基”。定义的后半句是“可用作其他所有下游模型和任务的基础”，所以强调的是被许多任务共用。



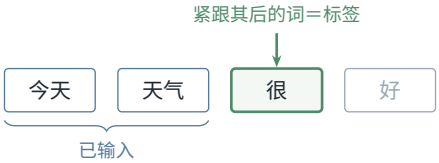
逐项 A **错误**：参数存在哪里与 Foundation 的含义无关。B **正确**：上图中一个底座托起多个任务，正是“地基”。C **错误**：恰好相反，大模型的特点是多任务适应。D **错误**：大模型的结构与规模都比传统小模型复杂得多。

一句话总结 Foundation = 很多任务共用的底座，而不是“基础、简单”。

第 2 题　答案：C

难度：基础　|　考查：语言建模的标签（4 章）

思路 语言模型的标签是“紧跟在已输入内容后面的那个词”。



逐项 A、B **错误**：“今天”“天气”是已经输入的词，不是要预测的。C **正确**：图中绿色框紧接在已输入部分之后。D **错误**：“好”是再下一步（输入为“今天 天气 很”时）的标签。

常见错误 把标签当成“句子的最后一个词”。

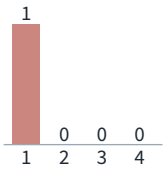
一句话总结 标签＝原句向左错开一位。

第 3 题　答案：D

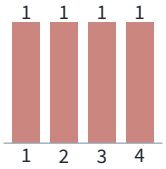
难度：基础 ｜ 考查：Softmax 的归一性（4.2 节）

计算 四个分数都是 1：$p_i = e^1/(4e^1) = 1/4 = 0.25$。分数相等时，Softmax 的输出一定是均匀分布。

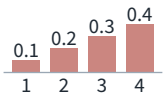
A：和为 1，但不均匀



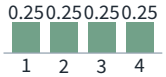
B：和为 4



C：和为 1，但不均匀



D：均匀且和为 1



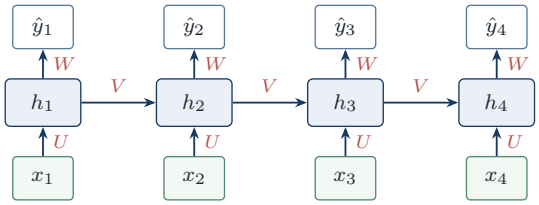
逐项 A **错误**：四个分数相同，没有理由让第一个独占全部概率。B **错误**：四根柱子之和是 4，不是概率分布——这是把“原始分数”直接当成了输出。C **错误**：和为 1，但柱子高低不同，而保序性要求“分数相等则概率相等”。D **正确**。

一句话总结 分数相同 ⇒ 概率均分。

第 4 题　答案：A

难度：基础 ｜ 考查：RNN 的共享权重（5.2 节）

思路 展开图中的四列是**同一个单元**在四个时间步的四次使用。



逐项 A **正确**：图上所有箭头只出现 U、V、W 三个字母，各时间步共用。B **错误**：没有“前两步一组、后两步一组”的划分。C **错误**：这是把 4 个时间步误当成 4 个不同的网络。D **错误**：无依据。

一句话总结 RNN 展开多少步都只有一组权重——这正是它能处理任意长度的原因。

第 5 题　答案：C

难度：基础 ｜ 考查：注意力权重矩阵的性质（第 6 章）

判据 每一行是一次 Softmax 的结果：**每个元素非负，每一行之和为 1**。列和没有要求。

A

0.6	0.6	和 1.2
0.5	0.5	和 1.0

B

1.2	−0.2	有负数
0.5	0.5	和 1.0

C

0.5	0.5	和 1.0
0.9	0.1	和 1.0

D

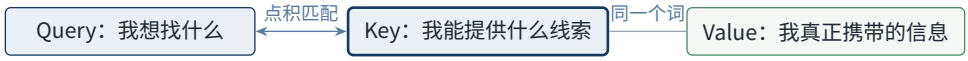
0.9	0.9	和 1.8
0.1	0.1	和 0.2

逐项 A **错误**：第 1 行之和为 1.2。B **错误**：第 1 行和虽为 1，但出现了负数和大于 1 的数，Softmax 的输出不可能为负。C **正确**：两行都非负且和为 1（它的列和是 1.4 与 0.6，并不妨碍）。D **错误**：这是列和为 1 的矩阵，行和分别为 1.8 与 0.2——把行列弄反了。

一句话总结 看行，不看列；非负，和为 1。

第 6 题　答案：B

难度：基础 ｜ 考查：Q、K、V 的含义（6.5 节）



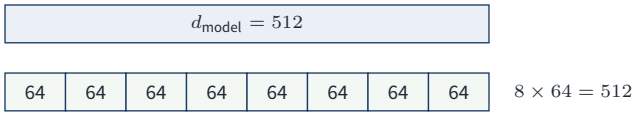
逐项 A **错误**：Query 是提问的一方。B **正确**：Key 是被拿来和 Query 比对的“线索”。C **错误**：Value 是匹配上之后被取回的内容，不参与匹配。D **错误**：位置编码提供顺序信息，不是 Q、K、V 之一。

一句话总结 Q 问，K 答“我在这儿”，V 才是被带走的内容。

第 7 题　答案：C

难度：基础 ｜ 考查：多头注意力的维数（6.9 节）

计算 $d_k = d_{\text{model}}/h = 512/8 = 64$。



逐项 A **错误**：8 是头数 h。B **错误**：$512/8 \neq 56$。C **正确**。D **错误**：4096 是 512×8，把除法做成了乘法。

一句话总结 每头维数＝总维数÷头数，拼接后恰好回到 d_{model}。

第 8 题　答案：D

难度：基础 ｜ 考查：位置编码的作用（6.10 节）

思路 自注意力只看词的内容之间的匹配，不看位置；打乱词序，每个词的输出不变（置换等变性）。



逐项 A **错误**：正弦位置编码不含参数，也不会减少其他参数。B **错误**：归一化由 Softmax 和 LayerNorm 负责。C **错误**：位置编码与词向量相加，句子长度不变。D **正确**。

一句话总结 注意力“认词不认座”，位置编码负责告诉它座位号。

第 9 题　答案：B

难度：基础 ｜ 考查：因果掩码（7.6 节）

判据 位置 i 只能看见位置 $1 \sim i$：第 i 行的前 i 格可见，形状为**下三角**（含对角线）。

A：上三角

✓	✓	✓	✓
×	✓	✓	✓
×	×	✓	✓
×	×	×	✓

B：下三角

✓	×	×	×
✓	✓	×	×
✓	✓	✓	×
✓	✓	✓	✓

C：只有对角线

✓	×	×	×
×	✓	×	×
×	×	✓	×
×	×	×	✓

D：全部可见

✓	✓	✓	✓
✓	✓	✓	✓
✓	✓	✓	✓
✓	✓	✓	✓

逐项 A **错误**：第 1 行能看见全部四个词——第一个位置就看到了全部“未来”，方向反了。B **正确**：第 1 行只看自己，第 4 行看全部。C **错误**：每个词只能看自己，连已经生成的前文都看不到，无法利用上下文。D **错误**：这是编码器（BERT）的情形，没有任何遮挡。

一句话总结 因果掩码＝下三角；右上角是“未来”，必须遮住。

第 10 题　答案：C

难度：基础 ｜ 考查：三种结构（第 8 章）



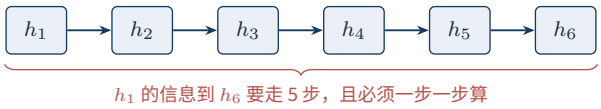
逐项 A **错误**：编码器＋解码器是原始 Transformer，用于翻译。B **错误**：只有编码器是 BERT，擅长理解。C **正确**：GPT 中的 T 是 Transformer，且只保留解码器（并去掉交叉注意力）。D **错误**：GPT 不使用循环结构。

一句话总结 BERT 取编码器，GPT 取解码器。

12.3 基础多选题解析（第 11–15 题）

第 11 题　答案：AB

难度：基础 ｜ 考查：RNN 的优缺点（5.4 节）



A **正确**。图中每个箭头都表示“后一步依赖前一步”，无法同时计算。

B **正确**。信息沿箭头逐级传递，每一级都有损失；训练时梯度也逐级衰减。

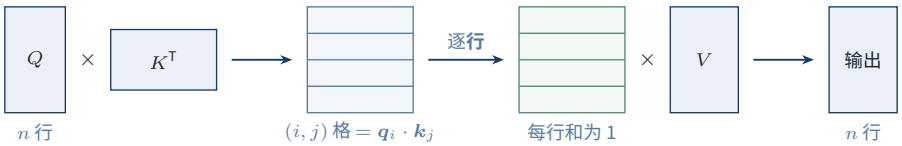
C **错误**。恰好相反：因为每步用同一个单元，RNN 可以处理任意长度的输入，这是它的优点。

D **错误**。所有时间步共享同一组权重，参数量与句长无关（见第 4 题）。

一句话总结 RNN 的毛病是“慢”和“忘”，不是“长度固定”或“参数多”。

第 12 题　答案：ACD

难度：基础 ｜ 考查：缩放点积注意力（6.5、6.6 节）



A **正确**。矩阵乘法的定义：第 i 行与第 j 列做点积。

B **错误**。图中横线把分数矩阵分成一行一行，Softmax 对每一行做，所以是行和为 1。

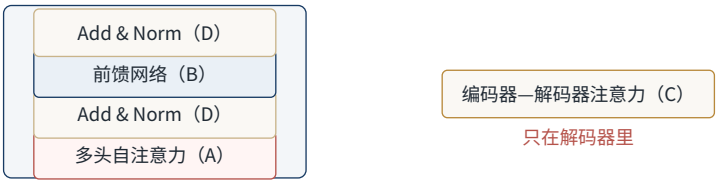
C **正确**。点积的方差为 d_k，除以标准差 $\sqrt{d_k}$ 后方差恒为 1。

D **正确**。图的两端都是 n 行：每个 query 对应一个输出。

一句话总结 一行一个提问者，一行一次 Softmax，一行一个输出。

第 13 题　答案：ABD

难度：基础 ｜ 考查：编码器层的组成（第 7 章）



A 正确，第一个子层。

B 正确，第二个子层。

C 错误。它需要用到“编码器的输出”，自然只能出现在解码器中。

D 正确，每个子层后面都有。

一句话总结 编码器一层＝自注意力＋FFN，各带 Add & Norm。

第 14 题　答案：ABC

难度：基础　|　考查：Softmax 的四条性质（4.2 节）

图上证据 以分数 $(2,1,0)$ 为基准，比较“全部加 5”与“全部乘 2”：



A 正确。指数恒为正。

B 正确。分子之和等于分母。

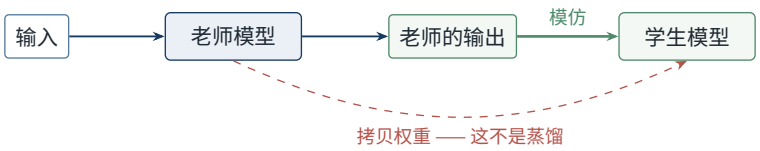
C 正确。中间一幅与左边完全相同：公因子 e^c 被约去。

D 错误。右边一幅明显更尖：乘 2 把分数差放大了一倍，概率之比由 e^1 变成 e^2。

一句话总结 Softmax 只认分数之差：平移不变，缩放会变。

第 15 题　答案：BCD

难度：基础　|　考查：蒸馏（9.3 节）



A 错误。图中红色虚线才是“拷贝权重”；蒸馏走的是上面那条路，只需要老师的输入和输出。

B 正确。绿色箭头：学生模仿老师的输出。

C 正确。老师更大更强，学生更小、更轻量。

D 正确。错误和偏见都包含在老师的输出里，学生会照着学。

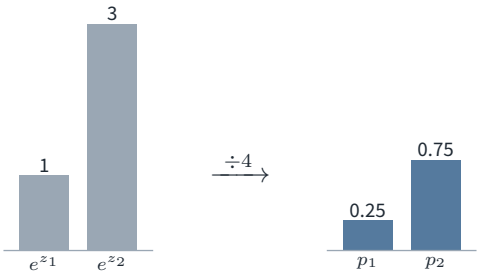
一句话总结 蒸馏抄的是“答卷”，不是“大脑”。

12.4　基础填空题解析（第 16–20 题）

第 16 题　答案：0.75（或 $3/4$）

难度：基础　|　考查：Softmax 的直接计算

过程 $e^{z_1}=e^0=1$，$e^{z_2}=e^{\ln 3}=3$；总和 $1+3=4$。所以 $p_1=1/4=0.25$，$p_2=3/4=0.75$。



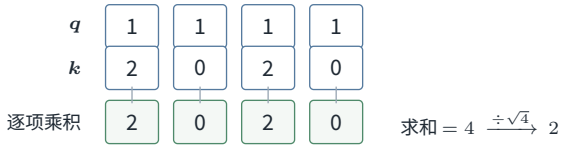
易错 把 $\ln 3$ 直接当作概率，或忘记取指数而算成 $\ln 3/(0+\ln 3)=1$。

等价答案 $3/4$、75%。

第 17 题　答案：4；2

难度：基础　|　考查：点积与缩放（6.4、6.6 节）

过程 $\boldsymbol{q}\cdot\boldsymbol{k}=1\times 2+1\times 0+1\times 2+1\times 0=4$；$\sqrt{d_k}=\sqrt{4}=2$；$4/2=2$。

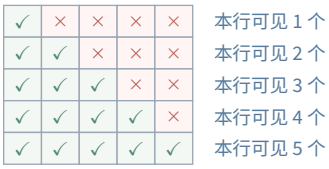


易错 除以 $d_k=4$ 得到 1；或把 d_k 当成向量里非零元素的个数。

第 18 题　答案：25；15

难度：基础　|　考查：权重矩阵的形状与因果掩码

过程 $n=5$，矩阵是 $5\times 5=25$ 个元素。加掩码后第 i 行只保留前 i 个：$1+2+3+4+5=15$。

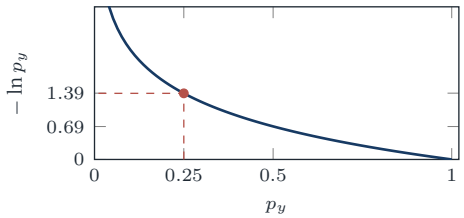


易错 忘记对角线（自己可以看自己），算成 $0+1+2+3+4=10$——那是被遮住的个数。

第 19 题　答案：1.39

难度：基础　|　考查：交叉熵损失（4.3 节）

过程 $L=-\ln 0.25=\ln 4=2\ln 2\approx 2\times 0.693=1.386\approx 1.39$。



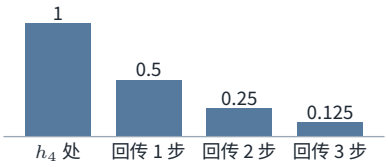
检查 $p_y=0.5$ 时损失为 0.69；概率减半，损失再加 0.69，与图中两条刻度一致。

易错 漏掉负号，或用以 10 为底的对数。

第 20 题　答案：0.125（或 $1/8$）

难度：基础　|　考查：梯度连乘（5.4 节）

过程 每步乘 0.5，回传 3 步：$0.5^3=0.125$。



易错 算成 $0.5 \times 3 = 1.5$（把连乘当成了相加）。

12.5 基础解答题解析（第 21–25 题）

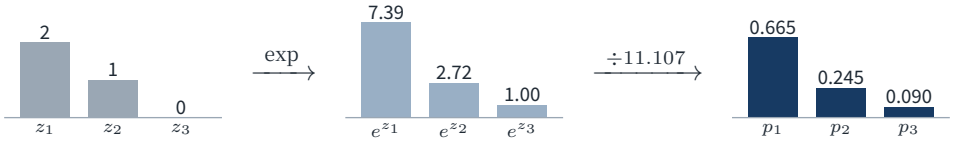
第 21 题　答案： $(0.665,\ 0.245,\ 0.090)$

难度：基础 ｜ 考查：Softmax 的完整计算

已知与求 分数 $z = (2, 1, 0)$，求 $\mathrm{softmax}(z)$。

公式 $p_i = e^{z_i} / \sum_j e^{z_j}$。

分步求解 ① 取指数：$e^2 \approx 7.389$，$e^1 \approx 2.718$，$e^0 = 1$。② 求和：$7.389 + 2.718 + 1 = 11.107$。③ 相除：$7.389/11.107 \approx 0.665$，$2.718/11.107 \approx 0.245$，$1/11.107 \approx 0.090$。



结果检查 $0.665 + 0.245 + 0.090 = 1.000$；顺序与分数一致；相邻两项之比约为 $e \approx 2.72$（$0.665/0.245 \approx 2.71$），因为分数依次相差 1。

易错 直接算 $2/(2+1+0)$。

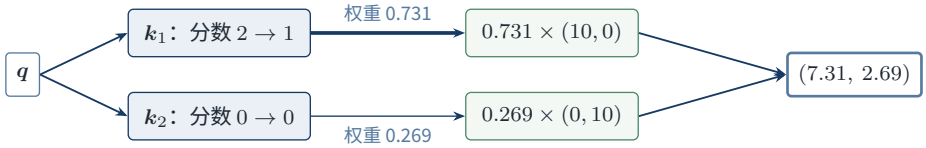
同类题怎么做 取指数、求和、相除、验和。

第 22 题　答案： $(7.31,\ 2.69)$

难度：基础 ｜ 考查：一次完整的注意力计算（6.5 节）

知识定位 点积 → 缩放 → Softmax → 对 V 加权。

分步求解 ① 点积：$\boldsymbol{q} \cdot \boldsymbol{k}_1 = 1 + 1 + 0 + 0 = 2$，$\boldsymbol{q} \cdot \boldsymbol{k}_2 = 0$。② 缩放：$\sqrt{d_k} = 2$，分数为 $(1,\ 0)$。③ Softmax：$e^1/(e^1 + e^0) = 2.718/3.718 \approx 0.731$，另一个为 0.269。④ 加权：$0.731 \times (10, 0) + 0.269 \times (0, 10) = (7.31,\ 2.69)$。



结果检查 输出的两个分量之和为 10，且更接近 $\boldsymbol{v}_1 = (10, 0)$，因为第 1 个词的权重更大；输出落在 $\boldsymbol{v}_1$ 与 $\boldsymbol{v}_2$ 的连线上（加权平均）。

易错 忘记除以 $\sqrt{d_k}$：那样得到 $\mathrm{softmax}(2, 0) = (0.881, 0.119)$，输出 $(8.81, 1.19)$。

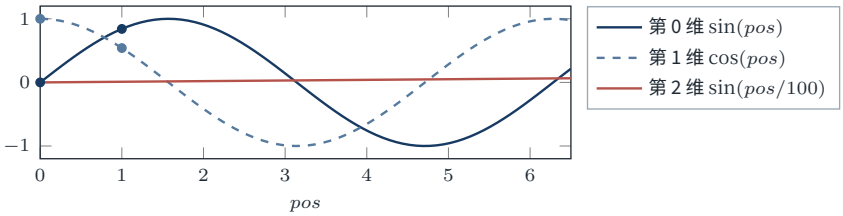
第 23 题　答案： $PE_0 = (0,\ 1,\ 0,\ 1)$；$PE_1 = (0.841,\ 0.540,\ 0.010,\ 1.000)$

难度：基础 ｜ 考查：正弦位置编码（6.10 节）

公式 $PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$，$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$。

分步求解 $d_{\text{model}} = 4$：$i = 0$ 时分母为 $10000^0 = 1$；$i = 1$ 时分母为 $10000^{2/4} = 100$。所以 $PE_{pos} = (\sin pos,\ \cos pos,\ \sin \frac{pos}{100},\ \cos \frac{pos}{100})$。

$pos=0$：$(\sin 0, \cos 0, \sin 0, \cos 0)=(0,1,0,1)$。　$pos=1$：$(\sin 1, \cos 1, \sin 0.01, \cos 0.01)\approx(0.841, 0.540, 0.010, 1.000)$。



结果检查 图中 $pos=0$、1 处的圆点就是前两维的取值；第 2 维在这两个位置几乎不动（红线贴近 0），说明它是慢变化的分量。

易错 把角度当成度数；或把分母写成 $10000^{i/d_{\text{model}}}$。

第 24 题　答案：$(-1,\ 1)$

难度：基础　|　考查：Add & Norm（7.3 节）

分步求解 ① Add：$x+\text{Sublayer}(x)=(1,3)+(1,1)=(2,4)$。② 均值 $\mu=(2+4)/2=3$。③ 标准差 $\sigma=\sqrt{((2-3)^2+(4-3)^2)/2}=\sqrt{1}=1$。④ 归一化：$\left(\frac{2-3}{1},\frac{4-3}{1}\right)=(-1,1)$。



结果检查 输出的均值为 0、标准差为 1，符合层归一化的作用。

易错 漏掉残差，直接对 $(1,1)$ 归一化——那样 $\sigma=0$，无法计算；或把相加写成拼接。

第 25 题　答案：见解析

难度：基础　|　考查：语言建模、自监督、交叉熵

（1） 4 个 token 可以构造 3 组：“猫”→“喜欢”；“猫 喜欢”→“吃”；“猫 喜欢 吃”→“鱼”。



（2） 标签（右列）就是原句向左错开一位得到的，不需要任何人工标注——监督信号来自数据自身，故称自监督。

（3） $L=-\ln 0.8\approx 0.223\approx 0.22$。

易错 数成 4 组（最后一个 token 后面没有词可作标签，除非另加结束符）。

同类题怎么做 T 个 token 给出 $T-1$ 组“前文→下一个”。

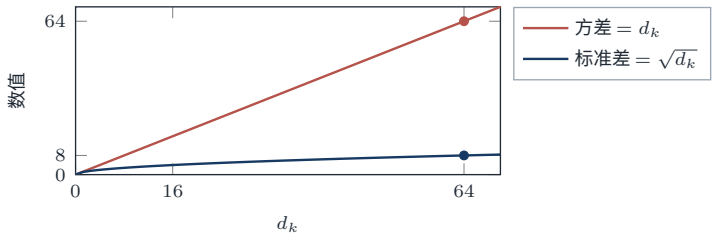
12.6　提升单选题解析（第 26–28 题）

第 26 题　答案：B

难度：中等偏下　|　考查：点积的方差与标准差（6.6 节）

知识链 方差 $=d_k$ → 标准差 $=\sqrt{d_k}$。

计算 $\text{Var}(\boldsymbol{q}\cdot\boldsymbol{k})=64$，标准差 $=\sqrt{64}=8$。



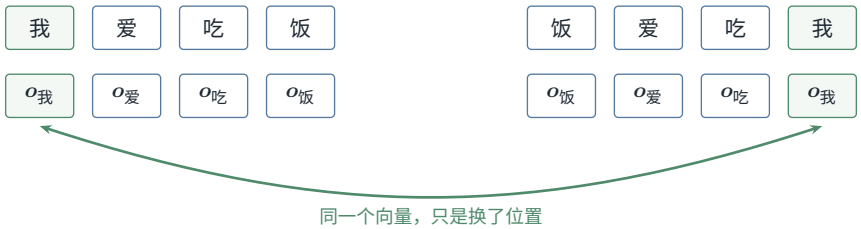
逐项 A **错误**：1 是缩放之后的标准差。B **正确**：图中蓝线在 $d_k = 64$ 处的值。C **错误**：64 是方差（红线），不是标准差。D **错误**：64^2，没有依据。

常见错误 混淆方差与标准差，这正是“为什么除以 $\sqrt{d_k}$ 而不是 d_k”的关键。

第 27 题　答案：C

难度：中等偏下 ｜ 考查：置换等变性（6.10 节）

思路 不加位置编码时，自注意力的输出随输入一同重排：每个词得到的向量只取决于“句子里有哪些词”，与它们的排列无关。



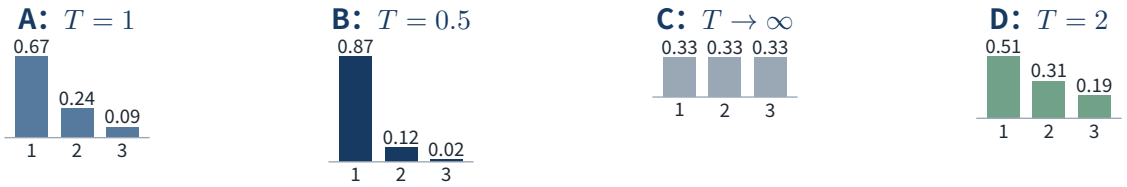
逐项 A **错误**：输出跟着词走，不跟着位置走；“我”不会拿到“饭”的输出。B **错误**：没有任何运算会把它变成零。C **正确**：图中两个绿色的 $o_{我}$ 相同。D **错误**：不同的词有不同的 q，输出各不相同。

本题最容易卡在哪里 以为“位置变了，输出就该变”。恰恰因为不变，才需要位置编码。

第 28 题　答案：D

难度：中等偏下 ｜ 考查：带温度的 Softmax（8.6 节）

计算 $T = 2$：分数变为 $(1,\ 0.5,\ 0)$；指数为 $2.718,\ 1.649,\ 1$，总和 5.367；概率 $\approx (0.51,\ 0.31,\ 0.19)$。



逐项 A **错误**：这是不加温度（$T = 1$）的 $\mathrm{softmax}(2, 1, 0)$，见第 21 题。B **错误**：比 A 更尖，对应把分数放大的低温 $T = 0.5$（即 $\mathrm{softmax}(4, 2, 0)$）。C **错误**：完全均匀只在 $T \to \infty$ 时出现；$T = 2$ 时三者仍有高低。D **正确**：比 A 平，但顺序不变。

一句话总结 温度升高，柱子变“矮胖”，但谁高谁低不变。

12.7　提升多选题解析（第 29–30 题）

第 29 题　答案：BCD

难度：中等偏下 ｜ 考查：交叉注意力的来源与形状（7.7 节）



A 错误。图中绿色箭头：Q 来自解码器——提问的是正在生成译文的一方。

B 正确。蓝色箭头：被查询、被取回的是编码器对原文的理解。

C 正确。行数＝Q 的个数＝3，列数＝K 的个数＝5。

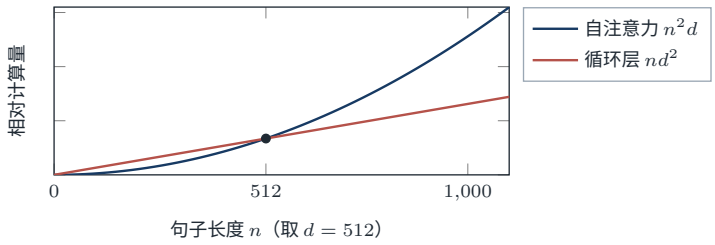
D 正确。每一行各做一次 Softmax。决定这一项成立的条件是“Softmax 按行进行”。

一句话总结 谁提问谁是行，被查的是列。

第 30 题　答案：ABD

难度：中等偏下 ｜ 考查：论文表 1（第 6 章开头）

层的类型	每层计算量	顺序操作数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环（RNN）	$O(n \cdot d^2)$	$O(n)$	$O(n)$



A 正确，表中第 1 行第 3 列。所有位置可以同时计算。

B 正确，表中第 2 行第 4 列。信息必须逐站传递。

C 错误。由图：$n < d$ 时蓝线在下方，自注意力更省；两线在 $n = d$ 处相交，之后蓝线（抛物线）迅速超过红线（直线）。句子越长，自注意力的计算量相对**越大**。

D 正确，表中第 1 行第 2 列。要算每一对 token 的分数，共 n^2 对。

一句话总结 自注意力赢在“并行”和“路径短”，代价是 n^2。

12.8　提升填空题解析（第 31–33 题）

第 31 题　答案：1 048 576

难度：中等偏下 ｜ 考查：多头注意力的参数量（6.9 节）

过程 一个投影矩阵：$512 \times 64 = 32\,768$。每个头 3 个、共 8 个头：$3 \times 8 \times 32\,768 = 786\,432$。$W^O$：$512 \times 512 = 262\,144$。合计 $786\,432 + 262\,144 = 1\,048\,576$。



检查 因为 $8 \times 64 = 512$，三类投影矩阵合起来各相当于一个 512×512 矩阵，加上 W^O 共 4 个：$4 \times 512^2 = 1\,048\,576$，两种算法一致。

易错 漏乘头数 8，或漏掉 W^O。

第 32 题　答案：6

难度：中等偏下　|　考查：因果掩码＋加权平均（7.6、6.7 节）

知识链 掩码决定“能看谁” → 分数相等则权重均分 → 输出是值的加权平均。

过程 第 3 个位置只能看位置 1、2、3；分数相等，权重各为 $1/3$；位置 4 的权重为 0。输出 $=\frac{1}{3}(3+6+9)+0\times 100=6$。

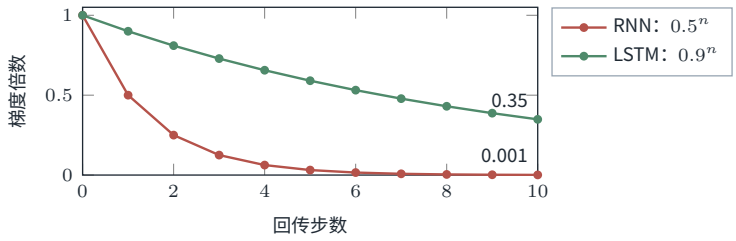


易错 把 $v_4=100$ 也算进去，得到 $(3+6+9+100)/4=29.5$。题目特意把 v_4 设得很大，就是为了让这个错误一目了然。

第 33 题　答案：$1/1024\approx 0.001$；0.35

难度：中等偏下　|　考查：RNN 与 LSTM 的梯度对比（第 5 章）

过程 RNN：$0.5^{10}=1/1024\approx 0.00098$。LSTM：$0.9^{10}\approx 0.349\approx 0.35$。



结果的意义 同样回传 10 步，LSTM 沿记忆通道保留的梯度约是 RNN 的 350 倍。但绿线仍在下降——忘记门小于 1 时衰减依然存在，只是慢得多，所以说 LSTM 是“缓解”而非“解决”。

12.9　提升综合大题解析（第 34–35 题）

第 34 题　答案：见解析

难度：中等偏下　|　考查：点积矩阵 → 缩放 → Softmax → 加权 → 掩码

1. 题目在问什么 把注意力公式对一个两词的句子完整算一遍，再看加掩码后有何变化。

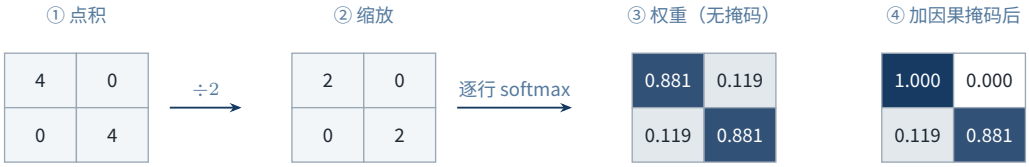
2. 已知整理 $Q=K=V=X=\begin{pmatrix}2&0&0&0\\0&2&0&0\end{pmatrix}$，$d_k=4$，$\sqrt{d_k}=2$。

3. 知识链 点积打分 → 缩放 → 逐行 Softmax → 乘 V →（解码器）掩码。

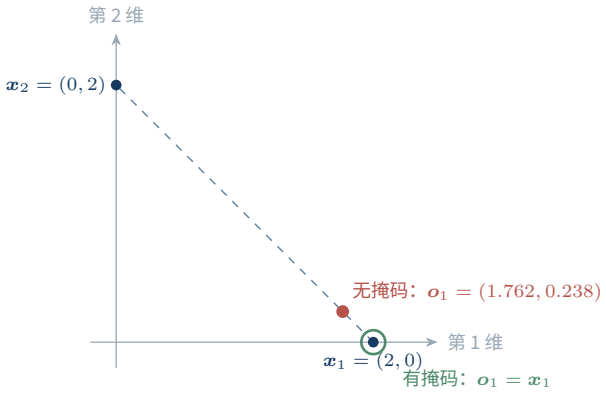
（1）点积矩阵 $(QK^{\mathsf{T}})_{ij}=\boldsymbol{x}_i\cdot\boldsymbol{x}_j$：$\boldsymbol{x}_1\cdot\boldsymbol{x}_1=4$，$\boldsymbol{x}_1\cdot\boldsymbol{x}_2=0$，$\boldsymbol{x}_2\cdot\boldsymbol{x}_2=4$。故 $QK^{\mathsf{T}}=\begin{pmatrix}4&0\\0&4\end{pmatrix}$。两个向量互相垂直，所以非对角元素为 0。

（2）权重矩阵 缩放后为 $\begin{pmatrix}2&0\\0&2\end{pmatrix}$。第 1 行：$\mathrm{softmax}(2,0)=\left(\frac{7.389}{8.389},\frac{1}{8.389}\right)\approx(0.881,\ 0.119)$；第 2 行同理为 $(0.119,\ 0.881)$。

（3）第 1 个 token 的输出 $\boldsymbol{o}_1=0.881\,\boldsymbol{x}_1+0.119\,\boldsymbol{x}_2=(1.762,\ 0.238,\ 0,\ 0)$。



（4）加因果掩码 第 1 个位置只能看自己，第 2 个分数改为 $-\infty$：$\mathrm{softmax}(2, -\infty) = (1,\ 0)$。于是 $\boldsymbol{o}_1 = 1 \times \boldsymbol{x}_1 = (2, 0, 0, 0)$，没有吸收任何来自 token 2 的信息。第 2 个位置可以看到两个词，权重不变。



结果验证 只画前两维（后两维恒为 0）。无掩码时 $\boldsymbol{o}_1$ 落在 $\boldsymbol{x}_1$ 与 $\boldsymbol{x}_2$ 的连线上，且靠近 $\boldsymbol{x}_1$——符合“加权平均、自己的权重大”；加掩码后它退回到 $\boldsymbol{x}_1$ 本身。

本题最容易卡在哪里 第（2）问忘记先除以 $\sqrt{d_k}$；第（4）问以为掩码影响所有位置（其实第 2 行不变）。

如果条件变化 若 $\boldsymbol{x}_2$ 改为与 $\boldsymbol{x}_1$ 相同，则四个点积都是 4，权重全为 0.5。

变式思考 把 d_k 的缩放去掉，第 1 行权重变成 $\mathrm{softmax}(4, 0) \approx (0.982, 0.018)$——分布更尖，这正是缩放要避免的。

第 35 题　答案：见解析

难度：中等偏下 ｜ 考查：Softmax → 贪心解码 → 交叉熵 → 温度

1. 题目在问什么 走完 GPT 生成与训练的一步：把分数变成概率、选词、算损失，再看温度的影响。

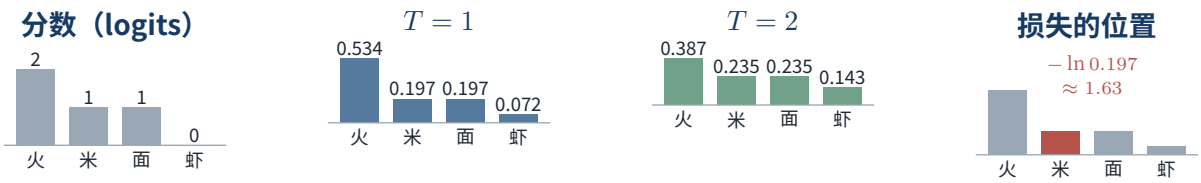
2. 知识链 Softmax（4.2）→ 贪心（8.5）→ 交叉熵（4.3）→ 温度（8.6）。

（1）$T = 1$ 的概率 指数：$e^2 \approx 7.389$，$e^1 \approx 2.718$，$e^1 \approx 2.718$，$e^0 = 1$；总和 13.825。概率：火锅 $7.389/13.825 \approx 0.534$，米饭 0.197，面条 0.197，小龙虾 $1/13.825 \approx 0.072$。

（2）贪心解码 选概率最高的：**火锅**。

（3）交叉熵 真实词是“米饭”，$p_y = 0.197$（更精确为 0.1966），$L = -\ln 0.1966 \approx 1.63$。

（4）$T = 2$ 分数除以 2：$(1,\ 0.5,\ 0.5,\ 0)$；指数：2.718，1.649，1.649，1；总和 7.016。概率：0.387，0.235，0.235，0.143。



读图 第二、三幅：温度从 1 升到 2，“火锅”从 0.534 降到 0.387，“小龙虾”从 0.072 升到 0.143，柱子更接近——输出**更随机**；但“火锅”仍是最高的，贪心解码的结果不变。第四幅：损失只看真实词“米饭”那根红柱的高度。

结果合理性检查 两组概率之和都为 1（0.534+0.197+0.197+0.072＝1.000；0.387+0.235+0.235+0.143＝1.000）；米饭与面条分数相同，概率也相同。

本题最容易卡在哪里 第（3）问用了“火锅”的概率；第（4）问把温度乘到分数上而不是除。

变式思考 若真实词恰好是“火锅”，损失为 $-\ln 0.534 \approx 0.63$，比 1.63 小得多——模型在这个位置猜得比较准。

12.10 跨学科迁移题解析（第 36–38 题）

第 36 题　答案：C

难度：基础～中等偏下 ｜ 考查：位置编码的频率（6.10 节）

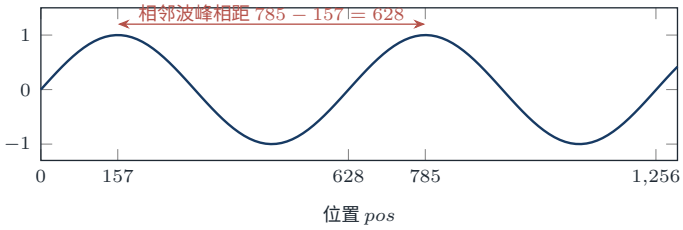
当前专题知识 位置编码每一维是关于位置的正弦函数，不同维频率不同。

借用的外部情境 大学物理中波长与角频率（波数）的关系 $\lambda = 2\pi/\omega$。

为什么不算超纲 该关系式已在题干中给出，只需代入。

迁移点 把“位置”当作波的自变量，位置编码的一维就是一列正弦波。

计算 $\sin(pos/100) = \sin(\omega\, pos)$，$\omega = 1/100$；$\lambda = 2\pi/\omega = 200\pi \approx 628$。



逐项 A **错误**：6.28 是 $\sin(pos)$（$\omega = 1$）的波长。B **错误**：100 是 $1/\omega$，漏乘了 2π。C **正确**：图中两个波峰在 $pos \approx 157$ 与 785。D **错误**：10000 是公式里的底数，不是波长。

一句话总结 分母越大，波长越长；这一维每 628 个位置才重复一次，适合区分“远近”。

第 37 题　答案：ACD

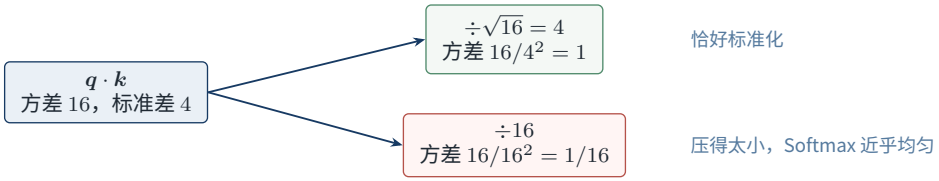
难度：中等偏下 ｜ 考查：为什么除以 $\sqrt{d_k}$（6.6 节）

当前专题知识 点积的方差随维数增长，缩放使其回到 1。

借用的外部情境 概率统计中独立随机变量之和的方差、$\mathrm{Var}(cX) = c^2\,\mathrm{Var}(X)$。

为什么不算超纲 两条性质都已在题干中给出。

迁移点 “把 16 项独立的量加起来”这一统计模型，正是点积的结构。



A **正确**。16 项独立、每项方差 1，和的方差为 16。

B **错误**。标准差是方差的平方根：$\sqrt{16} = 4$。

C **正确**。$c = 1/4$，$c^2 \times 16 = 1$。

D **正确**。$c = 1/16$，$c^2 \times 16 = 1/16$。这一项本身是正确的陈述，它同时说明了“若除以 d_k 会缩放过头”。

一句话总结 除以标准差 $\sqrt{d_k}$ 刚刚好，除以 d_k 就过头了。

第 38 题　答案： 4

难度：基础　|　考查：自注意力的计算量（论文表 1）

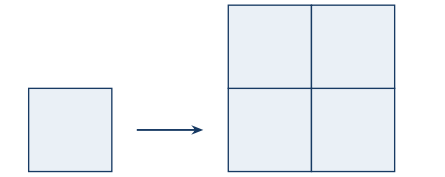
当前专题知识 自注意力要为每一对 token 计算分数，共 n^2 对。

借用的外部情境 数据结构与算法课中“运算次数与 n^2 成正比”的增长规律。

为什么不算超纲 “n 变为 c 倍则 n^2 变为 c^2 倍”已在题干说明。

迁移点 注意力权重矩阵的格子数，就是算法分析里的“基本运算次数”。

计算 n 由 500 变为 1000，$c=2$；计算量变为 $c^2=4$ 倍（$1000^2/500^2=4$）。



易错 答成 2 倍（那是与 n 成正比的循环层的情形）。

单位 无单位，是倍数。

一句话总结 长度翻倍，注意力的账单翻两番——这是长文本处理的主要成本来源。

参考资料

原始教学依据

- 课件《第四章 语言大模型》（“第三节课件_Transformer”，62 页）。
- 课堂手写笔记 4 页（Memo No. 444–447，2026 年 10 月 3 日）。

核心定义与公式核验

- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin. Attention Is All You Need. NIPS 2017.（本讲义第 6、7 章的公式、超参数、表 1–3 的数据均逐项对照这份原文。）
- S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. Neural Computation, 1997.
- J. L. Ba, J. R. Kiros, G. E. Hinton. Layer Normalization. 2016.
- K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR 2016.
- I. Goodfellow, Y. Bengio, A. Courville. Deep Learning. MIT Press, 2016.（Softmax、交叉熵、RNN 与梯度消失的标准表述。）
- 邱锡鹏.《神经网络与深度学习》. 机械工业出版社, 2020.

模型与训练方法核验

- J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. 2018.
- A. Radford, K. Narasimhan, T. Salimans, I. Sutskever. Improving Language Understanding by Generative Pre-Training. 2018.
- G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. 2015.

- L. Ouyang 等. Training Language Models to Follow Instructions with Human Feedback. 2022.
- R. Rafailov 等. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023.

学科史资料

- J. L. Elman. Finding Structure in Time. Cognitive Science, 1990.
- D. Bahdanau, K. Cho, Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. 2014.
- T. Brown 等. Language Models are Few-Shot Learners. 2020.（GPT-3）

> **教学补全 · 关于核验范围的说明** 第 6、7 章凡标注“论文”的内容，均与用户提供的论文原文逐处对照；全部算例与 38 道练习的答案均经程序复算。其余参考文献按作者记忆中的标准出处列出，制作本讲义时未能联网逐条复核书目细节（卷期、页码等从略）；课件中 2024 年以后的模型型号、笔记中的产品参数未作核验，正文已如实注明。讲义中标为“示意”的数值仅用于说明概念，不代表任何真实模型的输出。